

**MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND
COMMUNICATION TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING**



**GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY**

(HIGH-QUALITY PROGRAM)

**BUILDING A FASHION
E-COMMERCE APPLICATION
WITH RECOMMENDATION AND
IMAGE-BASED PRODUCT SEARCH**

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, December 2025

**MINISTRY OF EDUCATION AND TRAINING
CAN THO UNIVERSITY
COLLEGE OF INFORMATION AND COMMUNICATION
TECHNOLOGY
DEPARTMENT OF SOFTWARE ENGINEERING**



**GRADUATION THESIS
BACHELOR OF ENGINEERING IN
INFORMATION TECHNOLOGY**

(HIGH-QUALITY PROGRAM)

**BUILDING A FASHION
E-COMMERCE APPLICATION
WITH RECOMMENDATION AND
IMAGE-BASED PRODUCT SEARCH**

Student: Nguyen Thanh Phat
Student ID: B2005853
Class: DI20V7F1 (K46)
Advisor: Dr. Tran Cong An

Can Tho, 12/2025

**XÁC NHẬN CHỈNH SỬA LUẬN VĂN
THEO YÊU CẦU CỦA HỘI ĐỒNG**

Tên luận văn (tiếng Việt và tiếng Anh):

Building a Fashion Ecommerce Application with Recommendation and Image-based Product Search

Phát triển ứng dụng thương mại điện tử bán thời trang tích hợp gợi ý và tìm kiếm sản phẩm bằng hình ảnh

Họ tên sinh viên: Nguyễn Thanh Phát

MASV: B2005853

Mã lớp: DI20V7F1

Đã báo cáo tại hội đồng ngành: Công nghệ Thông tin

Ngày báo cáo: 24/12/2025

Hội đồng báo cáo gồm:

1. TS. Phạm Thế Phi
2. TS. Thái Minh Tuấn
3. TS. Trần Công Ân

Luận văn đã được chỉnh sửa theo góp ý của Hội đồng.

Cần Thơ, ngày tháng năm 2026

Giáo viên hướng dẫn
(Ký và ghi họ tên)

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and extend across the width of the page. There are no margins, text, or other markings on the paper.

ACKNOWLEDGEMENTS

I wish to express my deepest gratitude and sincere thanks to my advisor, Dr. Tran Cong An, who dedicated immense time, patience, and effort to guide me through every challenge of this thesis. His invaluable support was the cornerstone of this work. I would also like to thank the lecturers at Can Tho University for their invaluable knowledge and guidance throughout my studies.

Most importantly, I am profoundly grateful to my family for being my constant companions, offering their unwavering love, care, and support throughout this journey. I am also thankful to my friends for their continuous encouragement and companionship.

Sincerely,

Can Tho, December 2025

Nguyen Thanh Phat

TABLE OF CONTENTS

EVALUATION OF ADVISOR	IV
ACKNOWLEDGEMENTS	V
TABLE OF CONTENTS	VI
LIST OF FIGURES	X
LIST OF TABLES	XIV
LIST OF ABBREVIATIONS	XVII
ABSTRACT	XVIII
TÓM TẮT	XIX
PART 1: INTRODUCTION	2
I. Context and Motivation	2
II. Problem Statement	2
III. Objectives	3
IV. Scope and Limitations	4
V. Research Methodology	4
VI. Thesis Outline	5
PART 2: THESIS CONTENT	6
CHAPTER 1. BACKGROUND AND RELATED WORK	6
1.1 Fashion E-Commerce	6
1.2 Content-Based Image Retrieval	6
1.2.1 Visual Search Concepts	7
1.2.2 The Semantic Gap	7
1.2.3 Mathematical Foundations of Embeddings	7
1.3 Machine Learning Models	8
1.3.1 Convolutional Neural Networks	8
1.3.2 Vision Transformers	11
1.3.3 CLIP and Fashion-CLIP	13
1.3.4 Model Selection and Justification	16
1.4 Vector Databases	18
1.4.1 The Search Challenge	18
1.4.2 Approximate Nearest Neighbour Search	18
1.4.3 HNSW: Hierarchical Navigable Small World	19
1.4.4 IVFFlat: Inverted File with Flat Compression	19
1.4.5 pgvector: Vector Search in PostgreSQL	20
1.4.6 Architectural Decision and Trade-offs	20
1.5 Platform Architecture and Technology Stack	21
1.5.1 Architectural Patterns	21
1.5.2 .NET Backend	22

1.5.3	Vue.js Frontend	23
1.5.4	PostgreSQL and pgvector	23
1.5.5	Redis Caching	23
1.5.6	Python ML Sidecar	24
1.5.7	Hangfire Background Jobs	24
1.5.8	Identity, Authentication, and Authorisation	25
1.5.9	Benchmark Framework	25
1.5.10	Technology Stack Summary	26
1.6	Related Work and Research Gap	26
1.6.1	Academic Research	26
1.6.2	Commercial Systems	27
1.6.3	Contribution Differentiators	28
CHAPTER 2. DESIGN AND IMPLEMENTATION		29
2.1	Requirements Specification	29
2.1.1	Functional Requirements	29
2.1.2	Non-Functional Requirements	33
2.1.3	Feature Classification	34
2.2	System Modeling	35
2.2.1	System Actors	35
2.2.2	Functional Decomposition	36
2.2.3	Use Case Overview	36
2.2.4	Administrator Use Cases	39
2.2.5	Customer Use Cases	54
2.2.6	System Use Cases	64
2.3	System Architecture & Design	66
2.3.1	System Overview	67
2.3.2	Domain-Driven Design	68
2.3.3	C4 Architecture	72
2.3.4	Database Design	77
2.3.5	API Design	84
2.3.6	Security Design	86
2.4	Implementation	87
2.4.1	Technology Stack	88
2.4.2	Vertical Slice Architecture Core	89
2.4.3	Data Persistence Architecture	90
2.4.4	ML Sidecar and CBIR Search	92
2.4.5	Frontend Applications	95
CHAPTER 3. TESTING AND ASSESSMENT		118
3.1	Goal of Testing	118
3.2	Testing Strategy	119
3.2.1	Unit Testing	119
3.2.2	Integration Testing	119
3.2.3	End-to-End Testing	120

3.3	Scenario of Testing	120
3.3.1	Testing Environment	120
3.4	Result of Testing	120
3.4.1	Visual Search (CBIR) – UC-STR-SRC	120
3.4.2	ML Embedding Pipeline	121
3.4.3	Shopping Cart and Checkout – UC-STR-CRT, UC-STR-CHK	122
3.4.4	Admin Product Management – UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG	122
3.4.5	Summary	123
3.5	Benchmark Protocol and Experimental Setup	123
3.5.1	Models Benchmarked	123
3.5.2	Performance Metrics	124
3.5.3	Methodology	124
3.5.4	Statistical Analysis	125
3.5.5	Hardware Environment	125
3.6	Retrieval Performance and Accuracy	125
3.6.1	Ground-Truth Sensitivity	127
3.7	Computational Efficiency and Resource Trade-offs	129
3.8	Synthesis, Deployment Strategy, and Limitations	131
3.8.1	Accuracy-Efficiency Trade-off	131
3.8.2	Deployment Recommendations	132
3.8.3	Limitations	132
3.8.4	Summary	133
PART 3: CONCLUSION AND FUTURE WORK		135
I.	Summary of Work	135
II.	Contributions	136
III.	Limitations	136
IV.	Future Work	137
V.	Requirements Traceability	138
REFERENCES		140
APPENDIX A. FULL BENCHMARK RESULTS		144
A.1	Category-Only Ground Truth (5 Categories)	144
A.2	Category + Colour Ground Truth	145
A.3	Category + Colour + Pattern Ground Truth	145
A.4	Operational Efficiency (3-Fold CV)	146
A.5	Per-Fold Variability	147
APPENDIX B. DATASET COMPOSITION		148
B.1	Source and Selection	148
B.2	Category Distribution	148
B.3	Ground-Truth Label Schemes	149
B.4	Image Preprocessing	149

APPENDIX C. HARDWARE SPECIFICATIONS	150
C.1 Compute Hardware	150
C.2 Software Stack	150
C.3 Precision and Reproducibility	150

LIST OF FIGURES

Figure 1	ResNet architecture with residual skip connections	9
Figure 2	EfficientNet-B0 architecture: input image to feature vector	10
Figure 3	DINOv2 architecture: self-attention over image patches	12
Figure 4	CLIP ViT-B/16 dual-tower architecture	14
Figure 5	Fashion-CLIP dual-tower architecture with shared 512-dimensional embeddings	15
Figure 6	Work breakdown of the 26 use cases into the Administration, Storefront, and Background Services work areas.	37
Figure 7	Use case diagram for Product Management (UC-ADM-PROD). .	40
Figure 8	Use case diagram for Variant and Pricing (UC-ADM-VAR). . . .	41
Figure 9	Use case diagram for Image and Embedding Management (UC-ADM-IMG).	42
Figure 10	Use case diagram for Taxonomy and Classification (UC-ADM-TAX).	43
Figure 11	Use case diagram for Option Type Configuration (UC-ADM-OPT).	44
Figure 12	Use case diagram for Order Lifecycle (UC-ADM-ORD, UC-ADM-ORD-ITEMS).	45
Figure 13	Use case diagram for Payment Processing (UC-ADM-PAY). . . .	47
Figure 14	Use case diagram for Payment Method Configuration (UC-ADM-PAY-METHOD).	48
Figure 15	Use case diagram for Stock Location Management (UC-ADM-LOC).	48
Figure 16	Use case diagram for Stock Item Management (UC-ADM-STK).	49
Figure 17	Use case diagram for User Management (UC-ADM-USR).	51
Figure 18	Use case diagram for Role and Permission Governance (UC-ADM-ROL).	52
Figure 19	Use case diagram for Shipping Method Configuration (UC-ADM-SHP).	53
Figure 20	Use case diagram for Reference Data Management (UC-ADM-REF).	54
Figure 21	Use case diagram for Catalog Browsing (UC-STR-BRW).	54
Figure 22	Use case diagram for Search (UC-STR-SRC).	56
Figure 23	Use case diagram for Cart Management (UC-STR-CRT).	57
Figure 24	Use case diagram for Checkout Flow (UC-STR-CHK).	58
Figure 25	Use case diagram for Order History (UC-STR-OHI).	59
Figure 26	Use case diagram for Payment Processing (UC-STR-PAY).	60
Figure 27	Use case diagram for Authentication (UC-STR-AUT).	61
Figure 28	Use case diagram for Session Management (UC-STR-SES).	62

Figure 29	Use case diagram for Profile and Preferences (UC-STR-PRF). . .	63
Figure 30	Use case diagram for Embedding Operations (UC-SYS-EMB). .	64
Figure 31	ML embedding pipeline: image upload to normalised vector response.	65
Figure 32	Use case diagram for Background Maintenance (UC-SYS- MNT).	66
Figure 33	Bounded Context Map: eight contexts with Published Language identifiers.	68
Figure 34	Order checkout state machine showing forward-only stages and cancellation paths.	71
Figure 35	Payment intent lifecycle and terminal states.	72
Figure 36	System Context diagram: ReSys.Shop boundary, user roles, external integrations.	73
Figure 37	Containers: Vue 3 SPAs, .NET 10 API, ML sidecar, PostgreSQL/ pgvector, Redis.	74
Figure 38	Component diagram: API Backend internals and three-layer Python ML sidecar.	75
Figure 39	Deployment topology showing containerized orchestration with horizontal scaling.	76
Figure 40	Core entity-relationship model across eight bounded contexts. . .	78
Figure 41	Catalog schema: product aggregate and taxonomy tree.	79
Figure 42	Identity schema: ASP.NET Identity user aggregate.	80
Figure 43	Ordering schema: order aggregate with line items and adjustments.	80
Figure 44	Payment schema: payment capture aggregate and gateway configuration.	81
Figure 45	Inventory schema: stock item, reservation, and transfer aggregates.	82
Figure 46	Shipping schema: shipment aggregate with methods, rates, and zones.	82
Figure 47	Location schema: country and state reference data.	83
Figure 48	Profile schema: user profile and wishlist aggregates.	83
Figure 49	CBIR search sequence: upload, embedding extraction, pgvector search, results.	94
Figure 50	Visual search empty state: drop zone with format note (max 10 MB).	96
Figure 51	Upload state: selected-image preview with Search Similar Products button.	97
Figure 52	Search parameters panel: Fashion-CLIP model selector and three sliders.	97
Figure 53	Results state: product grid with similarity badges; query image in sidebar.	98

Figure 54	Storefront home: hero banner, collection cards, featured carousel.	99
Figure 55	Catalogue browsing: sidebar category tree, paginated product grid, search bar.	99
Figure 56	Product detail: gallery, size/colour pickers with stock badges, Add to Cart.	100
Figure 57	Cart: line items with quantity controls and sticky Order Summary panel.	101
Figure 58	Empty cart: bag icon with Continue Shopping link back to catalogue.	101
Figure 59	Address step: stepper wizard, saved-address picker with inline add form.	102
Figure 60	Delivery step: shipping methods with carriers, delivery dates, and rates.	102
Figure 61	Payment step: stepper wizard with Credit Card and Cash on Delivery options.	102
Figure 62	Confirm step: read-only summary with totals and Place Order action.	103
Figure 63	Complete step: confirmation with generated order number and summary.	103
Figure 64	Order history: stacked order cards with status badges, expandable line items.	104
Figure 65	Order detail: status header, state timeline, line items, addresses, totals.	104
Figure 66	Login page: email/password card with register link.	105
Figure 67	Registration page: account form with password strength indicator.	105
Figure 68	Session management: active sessions with device, IP, last activity; logout-all.	106
Figure 69	Address book: address cards with edit/delete and inline add-new form.	106
Figure 70	Wishlists: cards with item counts, privacy toggles, product thumbnails.	107
Figure 71	Notification preferences: Email/SMS toggles per notification category.	107
Figure 72	Admin product listing: paginated table with SKU, status, VND price, toolbar.	108
Figure 73	Product creation form: basic info, fashion metadata, SEO, option assignment.	108
Figure 74	Variant management: Master radio, SKU, options, inventory toggle, pricing.	109
Figure 75	Variant pricing tab: Current/Sale/Scheduled history with inline add form.	109

Figure 76	Images tab: sortable grid with Fashion-CLIP embedding status and regeneration.	109
Figure 77	Order management: table with checkout/payment/shipment statuses and filters.	110
Figure 78	Order detail: status header, state timeline, line items, payment actions.	110
Figure 79	Payment detail: Payment Intent ID, gateway, timeline, refund/void actions.	111
Figure 80	Payment methods: gateways with active toggles and supported currencies.	111
Figure 81	Admin dashboard: order/revenue/inventory summary cards. . . .	112
Figure 82	Inventory stock: On Hand/Reserved/Available per variant, low-stock badge.	112
Figure 83	Stock movements: audit log with +/- quantity deltas, reasons, and filters.	113
Figure 84	Stock locations: table with name, address, active badge, item count.	113
Figure 85	Restock dialog: variant, quantity, unit cost, reason, and notes. . .	113
Figure 86	Transfer dialog: source/destination locations, variant, quantity. .	114
Figure 87	User list with status toggle and roles; search and filter toolbar. .	114
Figure 88	User edit form: name, email, enabled toggle, role checkboxes. .	114
Figure 89	Roles: table with name, description, user count badge, Edit/Delete.	115
Figure 90	Role editor: permissions tree with domain.category.resource.action toggles.	115
Figure 91	Shipping methods: carrier, delivery estimate, active toggle, rates count.	115
Figure 92	Rates: table with zone, weight range, value range, rate, active toggle.	116
Figure 93	Hangfire dashboard overview: job metrics with realtime and history graphs.	116
Figure 94	Queues: queue depths, next jobs, and per-state job counts.	117
Figure 95	Job detail: method, parameters table, state history timeline.	117
Figure 96	mAP comparison across six benchmarked models, with Fashion-CLIP leading at 0.9336.	125
Figure 97	Precision at K = 5, 10, and 20, with Fashion-CLIP highest across all six models.	126
Figure 98	Inference latency across the six benchmarked models.	129
Figure 99	Throughput across the six benchmarked models.	129

LIST OF TABLES

Table 1	What different CNN layers learn to detect in fashion images	9
Table 2	CNN-based models benchmarked	11
Table 3	DINOv2 model specifications assessed in this project	12
Table 4	CLIP variants benchmarked	16
Table 5	Candidate pre-trained embedding models assessed for fashion product retrieval	16
Table 6	pgvector vs specialised vector databases	20
Table 7	Comparison of architectural patterns	21
Table 8	Technology stack of the ReSys.Shop platform	26
Table 9	Comparison of commercial visual search products	27
Table 10	Feature areas classified as Core Research vs Supporting Infrastructure.	34
Table 11	All 26 use cases with actor, module, and requirement traceability.	38
Table 12	Manage Products.	40
Table 13	Manage Variants.	41
Table 14	Manage Images and Embeddings.	42
Table 15	Manage Taxonomies and Classification.	43
Table 16	Manage Option Types.	44
Table 17	Manage Orders.	45
Table 18	Manage Order Details.	47
Table 19	Manage Stock Locations.	48
Table 20	Manage Stock.	49
Table 21	Manage Users.	51
Table 22	Manage Roles and Permissions.	52
Table 23	Browse and Search Catalog.	54
Table 24	Visual Search.	56
Table 25	Manage Cart.	57
Table 26	Checkout.	58
Table 27	Order History.	60
Table 28	Authentication.	61
Table 29	Session Management.	62
Table 30	Embedding Operations.	64
Table 31	System services with technology stacks and HTTP contracts.	67
Table 32	Bounded contexts with aggregate roots, key entities, and MediatR dispatch.	67
Table 33	Bounded context responsibilities and Published Language identifiers.	68
Table 34	Ubiquitous Language Glossary across domain contexts.	70

Table 35	ReSys.Shop API contract: 262 Carter endpoints across eight modules.	85
Table 36	Principal technologies grouped by architectural role with pinned versions.	88
Table 37	Sequential execution flow within the CreateProduct command handler.	89
Table 38	Schema-per-context mapping with UUID references, no cross-schema foreign keys.	90
Table 39	Concurrency control mechanisms across system domains.	91
Table 40	Registered embedding models with output dimensionality and architectural family.	92
Table 41	Embedding generation pipeline stages.	93
Table 42	ML sidecar API endpoints requiring <i>X-API-Key</i> authentication. . .	93
Table 43	Visual search UI state model.	95
Table 44	Testing objectives focused on core research features.	118
Table 45	Visual search test cases: normal flow, edge cases, and fault recovery.	120
Table 46	ML embedding pipeline test cases across four model architectures.	121
Table 47	Shopping cart and checkout test cases.	122
Table 48	Admin product, variant, image, and taxonomy management test cases.	122
Table 49	Six benchmarked models representing CNN, ViT, CLIP, and domain-tuned architectures.	123
Table 50	Performance metrics and definitions.	124
Table 51	Hardware environment. All benchmarks executed on CPU without GPU acceleration.	125
Table 52	Aggregate Retrieval Metrics, 3-Fold Cross-Validation (Category-Only Ground Truth)	126
Table 53	Key Caveats for Retrieval Performance	127
Table 54	mAP Under Three Ground-Truth Definitions (6 Models, 3-Fold CV)	127
Table 55	Efficiency Metrics, 3-Fold Cross-Validation	129
Table 56	Combined Accuracy and Efficiency Comparison	131
Table 57	Deployment Scenario Recommendations	132
Table 58	Research Question Answers	135
Table 59	Requirements traceability mapping objectives and research questions to chapters.	138
Table 60	Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean $\pm$ SD)	144
Table 61	Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean $\pm$ SD)	145

Table 62	Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean $\pm$ SD)	146
Table 63	Operational Efficiency, All Models (3-Fold CV, Mean $\pm$ SD) . . .	146
Table 64	Per-Fold Breakdown, Category-Only Ground Truth	147
Table 65	Dataset Category Distribution	148
Table 66	Benchmark Workstation Configuration	150
Table 67	Benchmark Software Environment	150

LIST OF ABBREVIATIONS

Abbreviation	Description
API	Application Programming Interface
ANN	Approximate Nearest Neighbor
CBIR	Content-Based Image Retrieval
CNN	Convolutional Neural Network
CQRS	Command Query Responsibility Segregation
DDD	Domain-Driven Design
DSR	Design Science Research
EF Core	Entity Framework Core
HNSW	Hierarchical Navigable Small World
JWT	JSON Web Token
mAP	Mean Average Precision
P@K	Precision at K
R@K	Recall at K
REST	Representational State Transfer
SPA	Single Page Application
ViT	Vision Transformer
VSA	Vertical Slice Architecture

ABSTRACT

E-commerce platforms rely primarily on text-based search, yet fashion products are inherently visual. Patterns, silhouettes, and textures resist keyword description. This thesis presents a fashion e-commerce platform with integrated Content-Based Image Retrieval (CBIR) that enables customers to search by uploading images. The system implements a modular architecture with a .NET backend, Vue.js frontend, and a Python ML sidecar for embedding generation.

A systematic benchmark compares six pre-trained models spanning CNN, vision-transformer, and CLIP architectures on both retrieval accuracy and operational efficiency. Fashion-CLIP, fine-tuned on over 700,000 fashion images, achieved the highest observed mean Average Precision at 0.9336 (SD 0.0060); the nearest competitor, DINOv2 ViT-S/14, scored 0.9299 (SD 0.0058). With only three folds, the 0.40% gap falls within measurement uncertainty. EfficientNet-B0 delivered inference at 42.6 ms (SD 5.6), a 2.67x speed advantage over Fashion-CLIP (113.6 ms) for a 2.86% accuracy trade-off.

Domain-specific pre-training may provide retrieval advantages, though the margin is small and within measurement uncertainty. The pluggable model architecture, switchable via a single environment variable, allows deployments to select the optimal accuracy-speed profile for their infrastructure.

Keywords: e-commerce, visual search, deep learning, modular architecture, computer vision, benchmarking

TÓM TẮT

Các sàn thương mại điện tử phụ thuộc nhiều vào tìm kiếm văn bản, một cơ chế kém hiệu quả trong lĩnh vực thời trang, nơi hoa văn, chất liệu và kiểu dáng khó diễn đạt qua từ khóa. Hệ thống được xây dựng theo hướng module: backend .NET, frontend Vue.js, và dịch vụ Python trích xuất đặc trưng hình ảnh từ mô hình học sâu tiền huấn luyện.

Thực nghiệm đánh giá chéo ba lần, so sánh sáu mô hình embedding trên chất lượng truy xuất (mAP, P@K, R@K) và hiệu năng vận hành (độ trễ, thông lượng, dung lượng). Fashion-CLIP, tinh chỉnh trên hơn 700.000 ảnh thời trang, đạt mAP quan sát cao nhất 0,9336 (SD 0,0060); mô hình gần nhất, DINOv2 ViT-S/14, đạt 0,9299 (SD 0,0058). Với ba lần đánh giá chéo, biên 0,40% nằm trong phạm vi không chắc chắn đo lường. EfficientNet-B0 đạt tốc độ suy luận nhanh nhất 42,6 ms (SD 5,6), nhanh gấp 2,67 lần Fashion-CLIP (113,6 ms) với mức đánh đổi 2,86% về chất lượng.

Huấn luyện chuyên biệt theo lĩnh vực có thể mang lại lợi thế truy xuất, tuy nhiên biên lợi thế nhỏ và nằm trong phạm vi không chắc chắn đo lường. Kiến trúc mô hình hoán đổi linh hoạt qua biến môi trường cho phép hệ thống thích ứng với nhiều cấu hình triển khai.

Từ khóa: thương mại điện tử, tìm kiếm hình ảnh, học sâu, kiến trúc module, thị giác máy tính, đánh giá hiệu năng

PART 1: INTRODUCTION

I. CONTEXT AND MOTIVATION

Global fashion e-commerce revenue exceeded **770 billion USD** in 2024, with projections surpassing **one trillion by 2030** [1]. However, keyword search does not work well for fashion products. Fashion items are defined by silhouette, drape, print density, and colour. These attributes are hard to describe in text. Shoppers who cannot find what they want often leave the session instead of rewriting their query [2].

Content-Based Image Retrieval (CBIR) solves this problem by comparing images directly instead of using text labels. Products are indexed using **dense vector embeddings** computed from images, and similarity is measured through mathematical distance functions, instead of using labels written by humans. For example, if a user uploads an image of a dress with a specific neckline and print pattern, the system can find visually similar products without needing any text search. Pre-trained convolutional neural networks [3], [4], vision transformers [5], and fashion-specific models [6] have greatly improved this capability in recent years.

This thesis focuses on system architecture, not on creating new algorithms. It studies how to add existing CBIR methods into a practical e-commerce system built with conventional web technologies, and provides experimental data showing which embedding models give the best balance between accuracy, speed, and resource use. The work connects two different technology stacks: the **Python machine learning stack** and the **.NET web stack**. This must be done while keeping real-time speed for interactive search.

II. PROBLEM STATEMENT

Fashion search that relies only on keywords has four main problems.

Catalogue vocabulary mismatch. Different vendors use different words to describe the same product. This can cause relevant products to be excluded from search results.

Visual attributes are hard to describe in text. Attributes such as fabric drape, texture, silhouette shape, and pattern style are difficult to search for using text.

Cold-start invisibility. New products do not yet have interaction history. Visual feature extraction allows these products to be found as soon as they are added to the catalogue.

Polyglot integration cost. The Python deep learning ecosystem does not work directly with .NET. To keep search speed under one second, the machine learning part of the system must be kept separate from the main application.

III. OBJECTIVES

This project builds a working fashion e-commerce platform with image-based search, and it benchmarks pre-trained deep learning models within this system. The main contribution is an **engineering demonstration** of adding existing models into a conventional web application stack.

Technical Objectives

- **Model integration.** Integrate pre-trained vision models into a PostgreSQL and .NET e-commerce stack. This can serve as an example for teams that already use similar web technology.
- **Polyglot architecture.** Architect a polyglot system in which a dedicated Python sidecar handles AI inference while the .NET backend manages transactional logic, business rules, and API routing.
- **Vector storage validation.** Validate **pgvector** (an open-source PostgreSQL extension) as the sole vector storage and retrieval layer, assessing whether it meets real-time search latency requirements at catalogue scales representative of small-to-medium fashion retailers.
- **Empirical benchmarking.** Benchmark multiple embedding models spanning convolutional and transformer architectures on shared hardware, giving practical guidance for choosing a model when resources are limited.

Research Questions

Three questions guide the investigation; Chapter 3 answers each experimentally.

RQ1: Model comparison. How do fashion-specific embedding models compare with general-purpose CNN and ViT architectures on fashion product retrieval?

RQ2: Accuracy-speed trade-off. What trade-offs exist between retrieval accuracy and inference latency, and which model offers the best balance for real-time search?

RQ3: Architecture viability. Can a service-oriented architecture with a dedicated AI sidecar separate image inference from the main application while maintaining interactive response times?

Tasks Completed

- **Build AI service.** Python FastAPI service loading pre-trained embedding models for vector generation within interactive latency bounds.
- **Set up vector search.** PostgreSQL with pgvector for high-dimensional embedding storage and similarity queries.
- **Connect services.** .NET backend orchestrating image upload, embedding generation, vector database query, and result assembly.
- **Create user interface.** Vue.js storefront with drag-and-drop image upload and similarity-scored results grid.
- **Assess results.** Systematic benchmark measuring retrieval accuracy, inference speed, and operational trade-offs across models.

IV. SCOPE AND LIMITATIONS

In scope: visual search via image upload, embedding-based product recommendation delivered through the **Similar Products** feature (a visual-similarity mechanism, Section 2.3), core e-commerce (catalogue, cart, checkout), and multi-model comparison across CNN and transformer architectures. Out of scope: real payment processing, shipping and logistics, social login, mobile applications, and custom model training.

Known Limitations

Four limitations define the boundaries of this work.

- **Dataset.** 5,000 fashion product images [7]. Controlled benchmarking is feasible at this scale but results may not apply directly to production catalogues containing millions of items.
- **Hardware.** Consumer-grade (Intel i7-1165G7, 16 GB RAM), all inference on CPU. Latency and throughput figures are relative to this profile; GPU acceleration would improve both metrics.
- **Assessment.** Exclusively quantitative: accuracy, latency, throughput. No formal user study, so the relationship between these metrics and actual user satisfaction is still an open question.
- **Model training.** All models used as published. Domain-specific fine-tuning, particularly for models pre-trained on generic corpora, might improve quality but was beyond scope.

V. RESEARCH METHODOLOGY

This section describes the methodology and tools used to implement and assess the system.

Development Methodology

The project follows **Design Science Research** (DSR) [8], [9] across four phases: Research and Planning (literature review, model and tool selection), Design (technology stack, system architecture, schema design), Implementation (.NET backend with VSA, Python FastAPI sidecar, Vue 3 storefront), and Testing and Assessment (mAP accuracy with cross-validation, inference latency, throughput across six models).

Technologies Used

The system is built using a modular stack designed for performance and scalability:

- **Backend:** .NET 10 with Carter, MediatR, FluentValidation.
- **AI Service:** Python 3.12 with FastAPI, PyTorch, Hugging Face Transformers.
- **Frontend:** Vue 3 with TypeScript, Vite, Pinia.
- **Database:** PostgreSQL with pgvector for relational and vector data in a single ACID database.

The system is assessed using quantitative metrics: Mean Average Precision (mAP) with 3-fold cross-validation for retrieval accuracy, per-image inference latency and throughput (images/second) for efficiency, across six models spanning CNN, vision-transformer, and CLIP architectures and the Fashion Product Images Dataset [7] (5,000 images). Detailed results appear in Chapter 3.

VI. THESIS OUTLINE

This thesis is organized into three parts.

Part 1: Introduction (this part) establishes the research context, problem statement, objectives, research questions, scope, methodology, and this outline.

Part 2: Thesis Content contains three chapters:

- **Chapter 1: Background and Related Work.** Surveys vector embeddings, neural architectures, vector databases, prior work in fashion image retrieval, and the technology stack.
- **Chapter 2: Design and Implementation.** Functional and non-functional requirements, system architecture (DDD, C4, database, API, security), and concrete implementation (.NET backend, Python ML sidecar, Vue storefront).
- **Chapter 3: Testing and Assessment.** Systematic benchmark comparing retrieval accuracy and inference efficiency across embedding models using cross-validation on 5,000 fashion images.

Part 3: Conclusion and Future Work synthesizes findings, assesses contributions and limitations, and proposes future work.

PART 2: THESIS CONTENT

1 BACKGROUND AND RELATED WORK

This chapter surveys the theoretical foundations and technical prerequisites for the project.

- **Fashion E-commerce.** Market context, semantic gap, business case.
- **Content-Based Image Retrieval.** Embeddings, cosine similarity, CBIR pipeline.
- **Machine Learning Models.** CNN, ViT, CLIP, Fashion-CLIP architectures.
- **Vector Databases.** ANN search, HNSW, IVFFlat, pgvector.
- **Platform Architecture and Technology Stack.** Modular monolith, vertical slice, .NET, Vue, PostgreSQL, Redis, Python sidecar, orchestration, auth, benchmarks.
- **Related Work and Research Gap.** Academic research, commercial systems, contribution differentiators.

1.1 FASHION E-COMMERCE

Global fashion e-commerce reached **770 billion USD** in 2024 and is projected to surpass **one trillion USD by 2030** [1]. However, text-based search bars have difficulty matching what users actually want to find, since keyword queries often fail to capture the user's visual intent [2].

Fashion is fundamentally visual. Attributes like silhouette, drape, color harmony, and pattern density do not translate easily into search terms. A customer can easily recognize a garment in an image but often fails to find it using keywords.

Major platforms like ASOS, Zalando, and Shopee invest in visual search because their catalogs contain millions of items, and the text descriptions written by different vendors are often inconsistent. The same pattern can carry different labels from different suppliers, so keyword search misses relevant products. Visual search avoids this metadata problem and can reduce lost sales caused by failed searches.

1.2 CONTENT-BASED IMAGE RETRIEVAL

Content-Based Image Retrieval replaces text queries with image queries. Rather than indexing products by human-authored labels, CBIR systems encode images into dense vector representations and measure similarity through mathematical

distance functions. This section introduces the core concepts and mathematical foundations of CBIR as applied to fashion product search.

1.2.1 Visual Search Concepts

Content-Based Image Retrieval (CBIR) replaces text queries with image queries. Instead of asking users to describe what they want in words, CBIR allows them to search using an example image.

The system encodes a query image into a **dense vector embedding**: a fixed-length sequence of numbers that captures shape, texture, colour, and pattern. It then retrieves catalog items whose embeddings are nearest in vector space. Visually similar products produce similar vectors; dissimilar products produce distant ones.

This approach bypasses the need for consistent textual labels. A photograph of a dress with a distinctive neckline retrieves visually similar products regardless of how the catalog describes them.

1.2.2 The Semantic Gap

The semantic gap, introduced in Section 1.1, is the difference between how visually detailed a garment is and how well a user can describe that detail in keywords. CBIR closes this gap by operating directly on visual content rather than on human-authored metadata. The embedding acts as a general description that captures attributes such as fabric texture, silhouette proportion, and colour gradients automatically, without any keyword translation step.

1.2.3 Mathematical Foundations of Embeddings

An embedding model processes an input image and transforms its visual content into a fixed-length numerical array:

$$\mathbf{e} = [e_1, e_2, e_3, \dots, e_d]^T \in \mathbb{R}^d$$

For a model with a 512-dimensional output, this vector $\mathbf{e}$ contains 512 continuous numbers. These arrays are called **vector embeddings**. Visually similar items yield nearby numerical values across these dimensions, converting visual comparison into a standard mathematical distance calculation.

1.2.3.1 The Latent Space

Embeddings act as coordinates within a high-dimensional continuous space known as the **latent space**. A 512-dimensional vector occupies a coordinate system with 512 orthogonal axes.

Within this space:

- Visually and semantically similar items sit close to one another.

- Dissimilar items lie further apart.
- The term “latent” indicates that individual axes rarely map directly to single human visual labels like “stripes” or “red.” Instead, each dimension captures abstract, non-linear feature combinations learned by the model during training.

1.2.3.2 Measuring Similarity: Cosine Similarity

To measure how closely two images match, the system measures the directional angle between their corresponding embedding vectors A and B using **cosine similarity**:

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\|_2 \times \|B\|_2}$$

Where $A \cdot B$ represents the vector dot product, and $\|A\|_2$ and $\|B\|_2$ denote the Euclidean norms (L_2 norms) of each vector.

Cosine similarity produces values ranging from +1.0 (identical vector orientation) to 0.0 (orthogonal vectors) down to −1.0 (opposite orientations). For normalized fashion embeddings, scores above 0.70 generally correspond to strong visual similarity that shoppers can see.

1.2.3.3 The CBIR Pipeline

A standard Content-Based Image Retrieval system operates through four core sequential stages:

1. **Image Input:** The user uploads or selects a target query photograph.
2. **Embedding Generation:** A deep learning model processes the image to output its feature vector.
3. **Vector Comparison:** The database compares the query vector against pre-indexed catalog vectors using cosine distance or cosine similarity.
4. **Ranking & Retrieval:** The system orders products by their similarity scores and renders the top nearest neighbors to the user.

1.3 MACHINE LEARNING MODELS

Visual feature extraction uses three families of architectures: convolutional neural networks, vision transformers, and multimodal CLIP-based models. Each subsection explains how the architecture works, introduces the specific variants assessed, and notes their trade-offs for fashion retrieval; the section ends with the model selection decision.

1.3.1 Convolutional Neural Networks

CNNs have dominated computer vision since AlexNet (2012). Variants benchmarked: ResNet-50, ResNet-101, EfficientNet-B0, and EfficientNet-B4.

1.3.1.1 Hierarchical Feature Extraction

A CNN processes an image through a stack of learned filters. Each filter is a small window (typically 3 by 3 pixels) sliding across the image detecting local patterns [3], building increasingly complex representations:

Table 1: What different CNN layers learn to detect in fashion images

Layer	What It Detects
Early layers	Simple patterns: edges, colours, corners
Middle layers	Combinations: textures, shapes, parts (lapels, sleeves)
Late layers	High-level features: garment categories, styles

Local patterns cascade into global understanding: edges become textures in middle layers, garments in late layers. CNNs have a strong **inductive bias** toward local patterns: they excel at texture and colour but may miss relationships between distant image regions.

1.3.1.2 ResNet and Skip Connections

Deeper networks capture richer features but suffer from **vanishing gradients**: training signals decay as they propagate backward through many layers. ResNet solves this with **skip connections**: identity paths that bypass convolutional blocks and add the block input directly to its output [3]. This lets gradients flow unimpeded, enabling 50-, 101-, or 152-layer networks to train effectively.

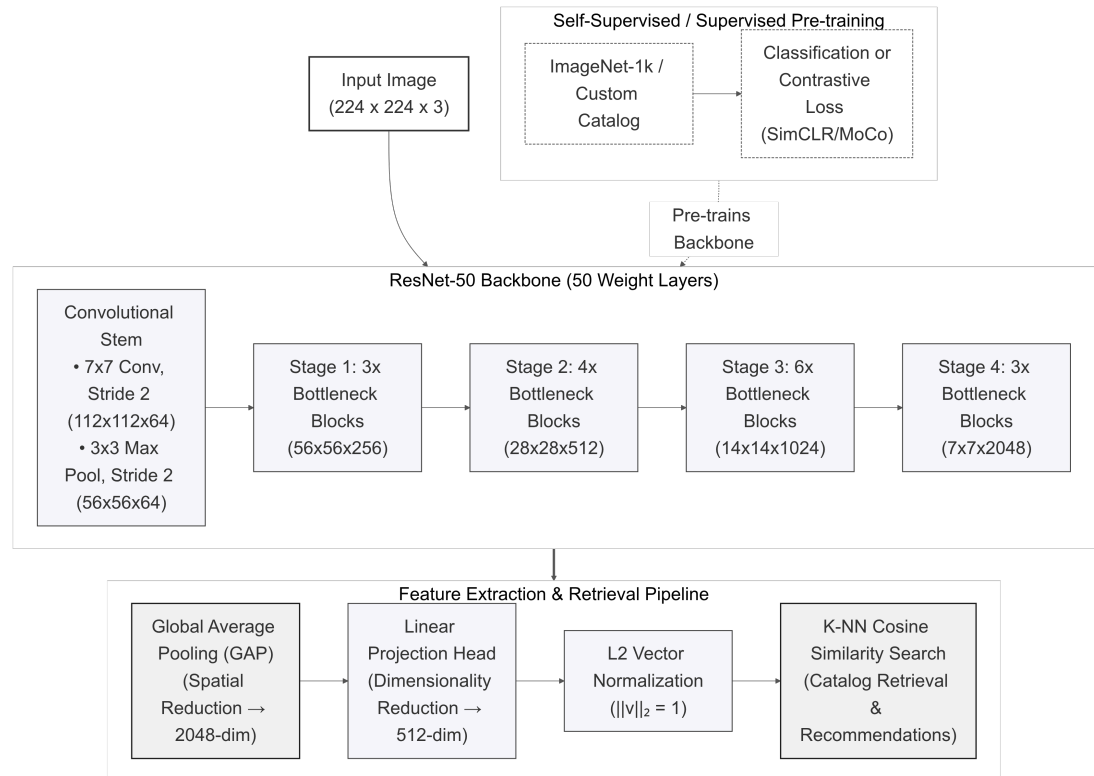


Figure 1: ResNet architecture with residual skip connections

ResNet-50 (25.6M parameters, 2,048-dim embeddings) remains a strong baseline for image retrieval. ResNet-101 (44.5M parameters) provides additional depth for comparison.

1.3.1.3 EfficientNet and Compound Scaling

Traditional scaling enlarges a network along one dimension: depth, width, or resolution. EfficientNet introduces **compound scaling**, balancing all three simultaneously using a learned coefficient [4]. This produces models (B0 through B7) with competitive accuracy and far fewer parameters.

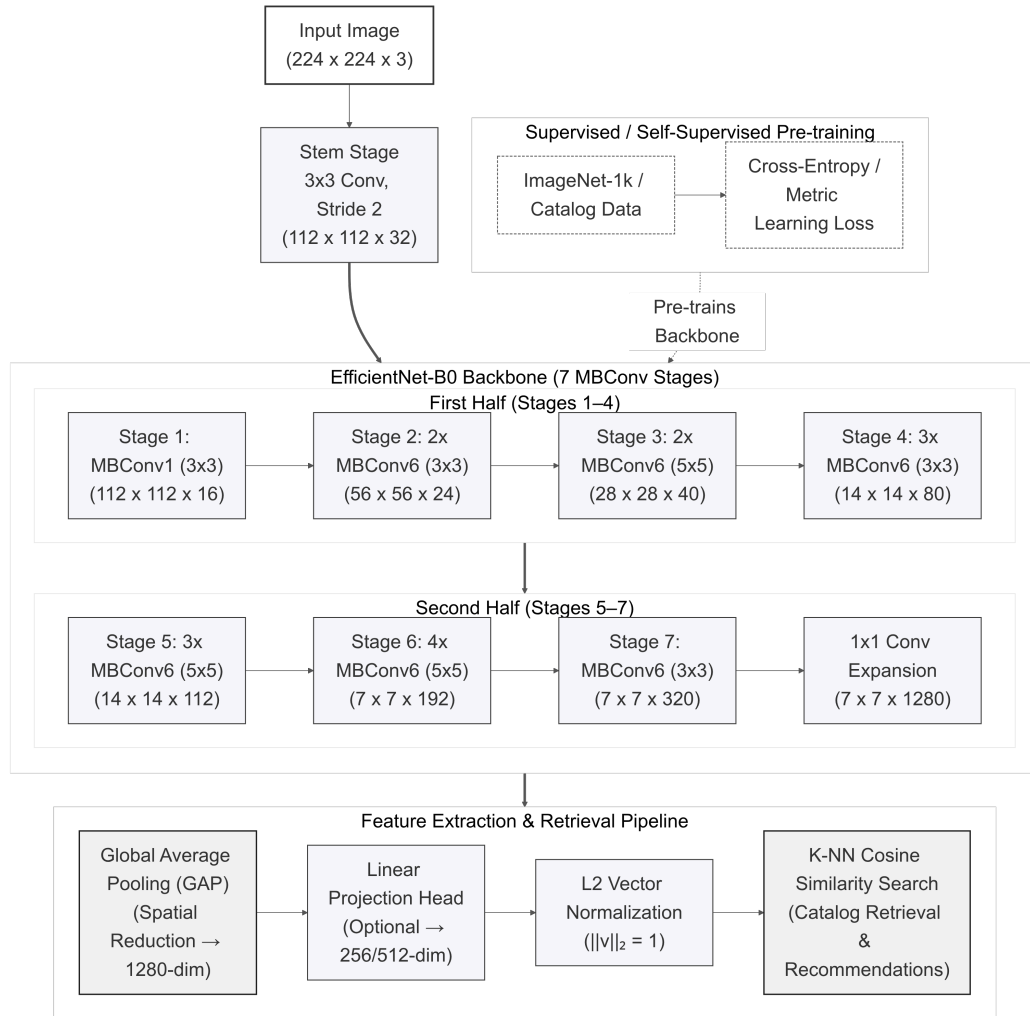


Figure 2: EfficientNet-B0 architecture: input image to feature vector

EfficientNet-B0 uses 5.3M parameters and 1,280-dim embeddings, well suited to CPU-only deployments. EfficientNet-B4 (19.3M parameters, 1,792-dim embeddings) provides higher capacity.

1.3.1.4 Benchmarked CNN Variants

Table 2: CNN-based models benchmarked

Family	Model	Parameters	Embedding Dim	Training Data
CNN	ResNet-50	25.6M	2048	ImageNet (1.2M images)
CNN	ResNet-101	44.5M	2048	ImageNet (1.2M images)
CNN	EfficientNet-B0	5.3M	1280	ImageNet (1.2M images)
CNN	EfficientNet-B4	19.3M	1792	ImageNet (1.2M images)

1.3.2 Vision Transformers

Vision Transformers (ViTs) apply the transformer architecture from NLP to images, replacing convolutional filters with self-attention over image patches.

1.3.2.1 Patch Embedding and Tokenization

Transformers were originally developed for NLP tasks such as translation and text generation. Their key innovation is **self-attention**, allowing the model to consider relationships between all parts of the input simultaneously. In 2020, researchers showed this approach also works for images [10]:

- Split the image into a grid of fixed-size patches (e.g., 14 by 14 or 16 by 16 pixels).
- Flatten each patch into a vector and treat it as a token, analogous to a word in a sentence.
- Add position encodings to preserve spatial information.
- Pass the sequence through transformer layers with multi-head self-attention.

1.3.2.2 Global Context via Self-Attention

Unlike CNNs, which focus on local patterns, self-attention captures relationships across the entire image from the first layer. A ViT can directly compare any two patches, even if far apart. For fashion, this means understanding that collar and cuffs match across opposite sides of an image: valuable for garment retrieval where silhouette and drape matter as much as local texture.

1.3.2.3 DINOv2 and Self-Supervised Pre-Training

Where supervised learning requires expensive human labels, **self-supervised learning** learns from the images themselves. DINOv2 uses a student-teacher self-distillation framework [11]:

- Take an image and create two different views (different crops, slightly different colours).
- Pass them through student and teacher networks with identical architecture.
- Train the student to match the teacher’s output.
- Update the teacher as an exponential moving average of the student weights.

- Repeat across 142 million uncured images.

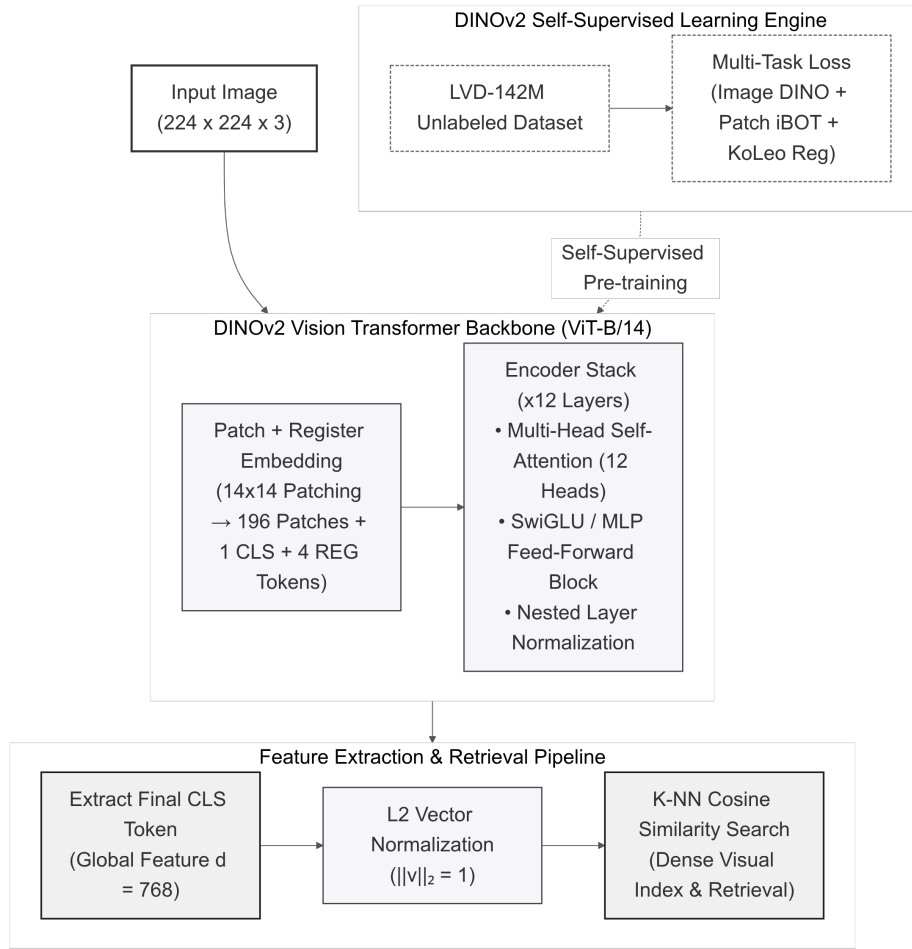


Figure 3: DINOv2 architecture: self-attention over image patches

DINOv2 produces features with strong object-level structure: silhouettes, part geometry, and garment boundaries without category labels, making it adaptable to fashion domains where curated labels are scarce.

1.3.2.4 Structural Fidelity for Fashion Retrieval

DINOv2 excels at **structural fidelity**: shapes, silhouettes, and proportions. It matches garments by cut (A-line, fitted, oversized), by proportion (cropped vs. full-length), and separates shape from colour; useful for finding a dress in a different colour with the same shape.

1.3.2.5 DINOv2 Model Specifications

Table 3: DINOv2 model specifications assessed in this project

Property	DINOv2 ViT-S/14	DINOv2 ViT-B/14
Architecture	Vision Transformer (Small)	Vision Transformer (Base)

Parameters	21M	86M
Patch size	14 by 14 pixels	14 by 14 pixels
Embedding dimension	384	768
Training data	142M images	142M images
Training method	Self-supervised	Self-supervised

1.3.2.6 Trade-offs and Limitations

Vision Transformers trade off differently than CNNs:

Advantages:

- Better at understanding global structure and long-range relationships.
- Learned without human labels, so potentially more general.
- Strong structural fidelity: captures shapes and silhouettes.

Disadvantages:

- Slower than CNNs (more computation required).
- Require larger input images for best results.
- May be overkill for simple colour/pattern matching.

1.3.3 CLIP and Fashion-CLIP

CLIP (Contrastive Language-Image Pre-training) connects vision and language, enabling search using both visual and textual queries. Fashion-CLIP, its domain-specialized variant, is the primary model for visual search.

1.3.3.1 Contrastive Language-Image Pre-Training

Traditional image models classify images into fixed categories (“cat,” “dog,” “dress”). CLIP learns to match images with text descriptions instead [5].

During training, CLIP processed 400 million image-text pairs from the public web (e.g., a floral dress with “colourful floral summer dress,” sneakers with “white running shoes on grass”). From these pairs, the model learned to:

- Convert images into vectors (image encoder).
- Convert text into vectors (text encoder).
- Make matching image-text pairs produce similar vectors.

1.3.3.2 Dual-Tower Architecture

CLIP has two separate towers:

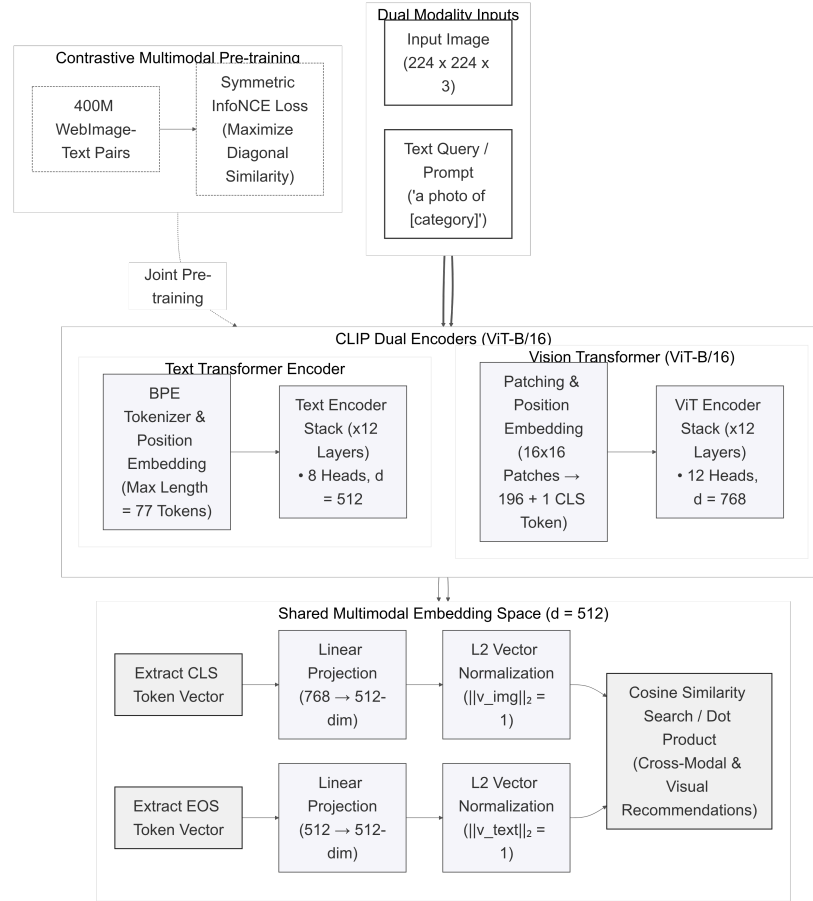


Figure 4: CLIP ViT-B/16 dual-tower architecture

- **Image Tower.** Processes the image using a Vision Transformer (ViT-B/16, ViT-B/32, or ViT-L/14).
- **Text Tower.** Processes text using a transformer.

Both towers output vectors of the same size (512 dimensions for ViT-B variants, 768 for ViT-L), so images and text can be directly compared using cosine similarity.

1.3.3.3 Multimodal Embedding Space

CLIP’s natural language understanding enables fashion concepts like “bohemian style” or “minimalist design,” matching images to abstract descriptions (“something for a casual Friday”), and capturing semantic meaning beyond visual appearance. However, general CLIP was trained on diverse internet images, not specifically fashion; it may not distinguish “A-line dress” from “sheath dress” or “Bohemian” from “vintage.”

1.3.3.4 Multimodal Query Capabilities

The dual-tower design enables query modalities unavailable in vision-only models such as DINOv2 or EfficientNet:

- **Text-to-image search.** A user types “red floral summer dress”; the text encoder maps the description into the same embedding space as catalog images.
- **Hybrid queries.** An uploaded photo combined with textual refinement (“like this, but in blue”) by encoding both modalities and merging the results.

This flexibility makes CLIP-based models the primary choice for the visual search feature.

1.3.3.5 Fashion-CLIP and Domain-Specific Fine-Tuning

Fashion-CLIP further trains CLIP on over 700,000 fashion product images paired with detailed descriptions covering garment categories, fabric textures, style descriptors, and occasion labels [6]. This specialization helps Fashion-CLIP understand:

- Fashion-specific vocabulary (“A-line,” “empire waist,” “distressed denim”).
- Style categories (“streetwear,” “preppy,” “athleisure”).
- Occasion suitability (“office wear,” “cocktail party,” “beach vacation”).

Fashion-CLIP inherits the ViT-B/16 architecture, producing 512-dimensional embeddings. The original paper reports an improvement on fashion retrieval over general CLIP; the benchmark analysis in Chapter 3 (§3.6) measured a 1.46% relative mAP advantage under category-only relevance [6].

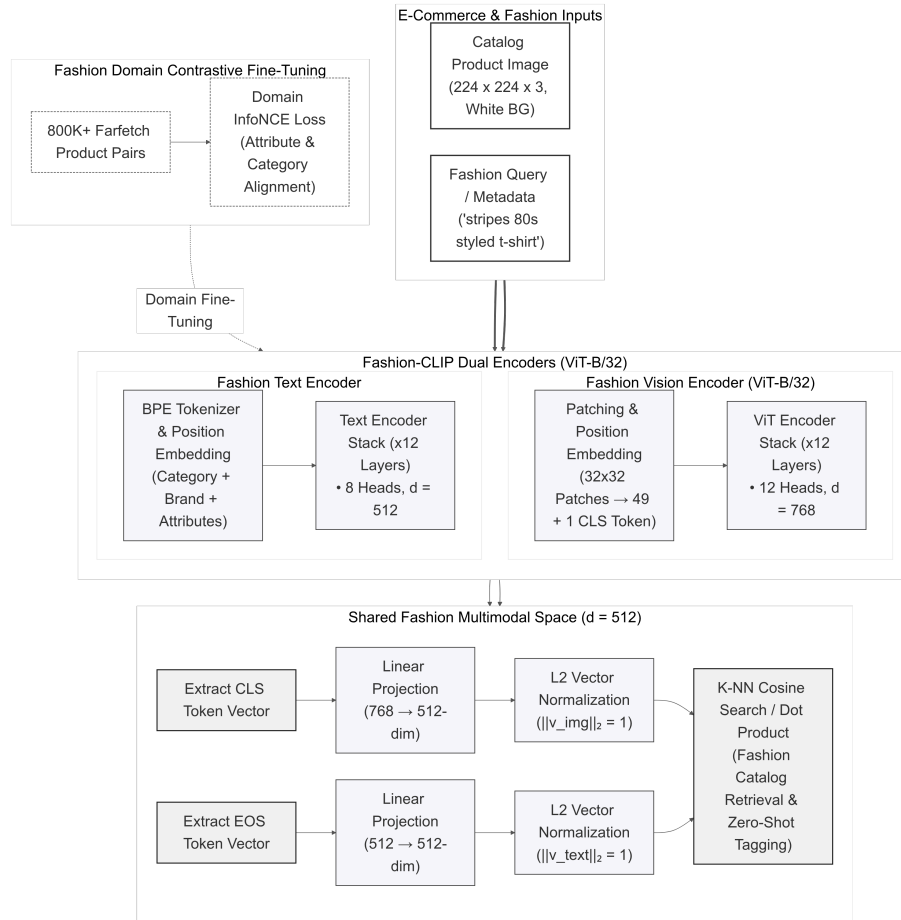


Figure 5: Fashion-CLIP dual-tower architecture with shared 512-dimensional embeddings

1.3.3.6 Benchmarked CLIP Variants

Table 4: CLIP variants benchmarked

Model	Architecture	Training	Domain
CLIP ViT-B/32	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-B/16	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
CLIP ViT-L/14	Dual-tower (ViT + text transformer)	Contrastive (400M pairs)	General
Fashion-CLIP	Dual-tower (ViT + text transformer)	Contrastive, fine-tuned on 700K fashion images	Fashion-specific

1.3.4 Model Selection and Justification

The rationale for selecting Fashion-CLIP as the primary model for visual search is presented below.

1.3.4.1 Candidate Models

Eleven candidate pre-trained models spanning three architectural families were considered; six representative models were benchmarked. The subset was chosen to cover every architectural family (CNN, ViT, CLIP, fashion-specific) with at least one representative per family, while keeping the total number of benchmarks manageable on consumer-grade hardware. Within each family, the model with the best published accuracy-to-compute ratio was preferred: ResNet-50 and EfficientNet-B0 for CNNs, DINOv2 ViT-S/14 for ViT, CLIP ViT-B/16 and CLIP ViT-B/32 for generic CLIP, and Fashion-CLIP for the fashion-specific category.

Table 5: Candidate pre-trained embedding models assessed for fashion product retrieval

Model	Architecture	Dim	Parameters	Training Method
ResNet-50	CNN	2048	25.6M	Supervised (ImageNet)
ResNet-101	CNN	2048	44.5M	Supervised (ImageNet)
ResNet-152	CNN	2048	60.2M	Supervised (ImageNet)
EfficientNet-B0	CNN	1280	5.3M	Supervised (ImageNet)
EfficientNet-B4	CNN	1792	19.3M	Supervised (ImageNet)
DINOv2 ViT-S/14	ViT	384	21M	Self-supervised (142M images)

Model	Architecture	Dim	Parameters	Training Method
DINOv2 ViT-B/14	ViT	768	86M	Self-supervised (142M images)
CLIP ViT-B/32	ViT + Text	512	151M	Contrastive (400M pairs)
CLIP ViT-B/16	ViT + Text	512	150M	Contrastive (400M pairs)
CLIP ViT-L/14	ViT + Text	768	428M	Contrastive (400M pairs)
Fashion-CLIP	ViT + Text	512	150M	Fine-tuned (700K fashion)

1.3.4.2 Benchmark Methodology

All models were assessed under identical conditions: 5,000 fashion product images from the Fashion Product Images dataset, split into training and query sets. Hardware was consumer-grade: Intel i7-1165G7 CPU with 16 GB RAM, all inference on CPU.

Metrics included Mean Average Precision ($\text{mAP}@10$) as the primary measure of overall retrieval quality, Precision at K ($\text{P}@K$) for top-ranked accuracy, Recall at K ($\text{R}@K$) for coverage of relevant items, and inference latency in milliseconds. A retrieved product was considered relevant if it belonged to the same category as the query image. The full benchmark protocol, results, and cross-validation methodology are presented in Chapter 3.

1.3.4.3 Weighted Selection Criteria

Model selection was based on four criteria: retrieval quality ($\text{mAP}@10$ and $\text{P}@K$ scores), inference latency (sub-300 ms total response time), multimodal capability (search by image and text), and hardware compatibility within memory and compute constraints of commodity hardware.

1.3.4.4 Selection Decision

Fashion-CLIP was selected as the primary embedding model for the visual search feature. Three factors drove this decision.

First, retrieval quality: Fashion-CLIP achieved the highest mAP among the assessed models, outperforming generic CLIP ViT-B/16 on category-only relevance [6] (see Chapter 3 for the quantitative comparison).

Second, multimodal capability: Fashion-CLIP’s dual-tower architecture enables search by image, by text description, and by hybrid image-plus-text queries, unavailable in vision-only models such as DINOv2 and EfficientNet.

Third, domain specialization: fine-tuning on 700,000 fashion images gives Fashion-CLIP an understanding of fashion-specific vocabulary, styles, and garment attributes that general-purpose models lack.

Fashion-CLIP provides the strongest overall combination of retrieval quality, search flexibility, and inference performance for the target deployment scenario, though EfficientNet-B0 offers faster CPU inference and DINOv2 excels at structural fidelity for silhouette-based matching.

1.3.4.5 Alternative Deployment Scenarios

For different deployment contexts, alternative models may be preferred. EfficientNet-B0 provides the fastest inference at 5.3 million parameters, trading off 2.86% lower mAP with no text-to-image capability. DINOv2 excels at shape and silhouette matching but lacks multimodal capability. General CLIP variants suit multi-category marketplaces with lower fashion accuracy. CLIP ViT-L/14 offers the largest capacity at 428 million parameters but requires substantial GPU VRAM.

The complete numerical comparison and error analysis across all models are presented in Chapter 3.

1.4 VECTOR DATABASES

Once images are converted to embeddings, those vectors must be stored and searched efficiently. The following subsections explain the challenge of vector similarity search at scale, introduce two indexing algorithms, and describe pgvector, the PostgreSQL extension used in this project.

1.4.1 The Search Challenge

When a user uploads a query image, the system must compare its embedding vector against every product vector in the catalog. This is **nearest neighbour search**.

A naive brute-force approach scales linearly with catalog size:

- 10,000 products = 10,000 comparisons per search.
- 100,000 products = 100,000 comparisons.
- 1,000,000 products = 1,000,000 comparisons.

For real-time search (under one second), this is too slow.

1.4.2 Approximate Nearest Neighbour Search

The solution is **Approximate Nearest Neighbour** (ANN) search [12]. Instead of checking every vector, ANN algorithms build index structures (graphs or clusters) that let a query go directly to the group of likely matches, skipping irrelevant vectors.

The key trade-off: we trade perfect accuracy for speed. If the true top match has similarity 0.95 and the returned result has 0.93, that is acceptable

for product search. ANN algorithms typically achieve 97 to 99% recall of the true top matches while reducing search time by orders of magnitude [12]. Two algorithms are used in this project: **HNSW** for production search and **IVFFlat** for rapid benchmarking [13].

1.4.3 HNSW: Hierarchical Navigable Small World

HNSW is the preferred index for production-scale vector search [12]. It builds a multi-layered graph where each vector is a node connected to its nearest neighbours.

- **Structure.** Multiple layers form a hierarchy. Top layers are sparse and connect distant regions for long-range jumps. Bottom layers are dense and refine the search locally.
- **Search.** Start at a top-layer node, greedily traverse toward the nearest neighbour, descend one layer, and repeat. Cost scales logarithmically with catalog size.
- **Recall.** Reported at over 95% with query latencies under 10 ms, sustained across catalog scales of up to 10 million vectors [12].
- **Build cost.** Graph construction is computationally expensive. At tens of thousands of vectors, build time is measured in minutes.

Configuration parameters:

- **M.** Connections per node. Default 16. Higher improves recall at cost of memory and build time.
- **ef_construction.** Candidates assessed during index build. Higher produces better index at cost of build time.

HNSW's logarithmic query cost makes it suitable for interactive fashion retrieval at millions of catalog items [12], where sub-100 ms latency is required.

1.4.4 IVFFlat: Inverted File with Flat Compression

IVFFlat is a simpler ANN algorithm used during model benchmarking. It partitions the embedding space into clusters via **k-means** and stores each vector in its nearest cluster [13].

- **Search.** Compute distance to all cluster centroids, select the nearest few clusters, and search exhaustively within only those clusters.
- **Recall.** Moderate: 65 to 72% at sub-10 ms for catalog-scale data, per pgvector documentation [13]. Recall degrades as the catalog grows because each cluster contains more candidates.
- **Build cost.** Low. Clustering completes in under one second for 5,000 vectors with minimal memory overhead.

Configuration parameters:

- **lists.** Number of clusters. More lists means smaller clusters and faster search, but risks missing relevant vectors.
- **probes.** Number of clusters searched per query. More probes improve recall at linear cost to query latency.

IVFFlat is used for the model comparison benchmarks in Chapter 3, where the objective is ranking embedding models rather than optimising index performance. Its fast build time and simple configuration suit rapid benchmarking. For production, HNSW is the designated long-term index.

1.4.5 pgvector: Vector Search in PostgreSQL

pgvector is an open-source PostgreSQL extension adding vector operations to standard SQL [13], storing vectors alongside regular product data.

Key features:

- **Vector column.** *vector* columns store embeddings and support arbitrary dimensionality; the platform stores an untyped *vector* column whose dimension is fixed per model (e.g., 512 for Fashion-CLIP).
- **Similarity operators.** `<=>` for cosine distance, `<->` for Euclidean.
- **Indexing.** HNSW and IVFFlat for fast approximate search.

The critical advantage is transactional consistency: vectors and product metadata share the same ACID boundary, avoiding the inconsistency problems that occur when a vector store and a relational database are kept separate. Combined queries can search for visually similar products filtered by category and price range in one query plan.

```
CREATE INDEX ON variant_image_embeddings USING hnsw ((vector::vector(512))
vector_cosine_ops)
WHERE model_name = 'fashion_clip';
SELECT id, name, 1 - (image_embedding <=> query_vec) AS similarity
FROM products ORDER BY image_embedding <=> query_vec LIMIT 10;
```

1.4.6 Architectural Decision and Trade-offs

Specialised vector databases (Pinecone, Milvus, Weaviate) exist for large-scale search. pgvector was selected for simplicity and transactional integration.

Table 6: pgvector vs specialised vector databases

Feature	Specialised Vector DB	pgvector
Setup	Moderate to high	Low (extension only)
Consistency	Separate database	Same transaction as product data
Query language	Custom API	Standard SQL
Cost	Often paid service	Free, open source

Scale limit	Billions of vectors	Millions of vectors
-------------	---------------------	---------------------

For thousands to tens of thousands of products, pgvector’s simplicity outweighs scaling advantages. Limitations: not designed for billion-vector deployments, fewer features than dedicated vector databases, no native multi-server distribution. For this project’s 5,000-product scope, these are acceptable.

1.5 PLATFORM ARCHITECTURE AND TECHNOLOGY STACK

Building a working e-commerce platform with machine learning built in requires careful architecture and technology choices. The following subsections describe each technology in the ReSys.Shop stack and the reasons for each choice.

1.5.1 Architectural Patterns

An application’s architecture determines code partitioning and component communication [14]. Table 7 compares four patterns governing system-level structure.

Table 7: Comparison of architectural patterns

Pattern	Structure	Strengths	Limitations	Suited For
Monolith [15]	Single deployable unit; one codebase, build, and deployment	No network overhead; simple development and debugging	Components entangle over time; full restart on any deployment	Small applications; one team; few business domains
Microservices [16]	Independent services, each owning one business capability; network communication	Autonomous teams with different stacks; independent scaling and deployment	Network latency; service discovery; partial failure handling; distributed transaction coordination	Large organisations; independently evolving teams and domains
Modular Monolith [15]	Independent modules in a single process; in-process message bus; compile-time cross-module isolation	Module independence without networking cost; one deployment and one database; simple migration path to microservices	Single database may limit extreme-scale growth; all modules restart on any deployment	Single-team projects requiring logical separation without distributed infrastructure
VSA [17]	Self-contained feature folders with handler, validation, data	All code for one feature co-located; features added, modified,	Cross-cutting concerns, such as authentication and logging, require	Numerous independent features evolving at different cadences

Pattern	Structure	Strengths	Limitations	Suited For
	access, and endpoint definition	or removed in isolation	explicit extraction across slices	

1.5.1.1 Architectural Decision

ReSys.Shop combines three patterns:

- **Modular monolith.** Nine business modules in a single .NET process communicate through MediatR CQRS [18]. Compile-time rules prevent direct cross-module references.
- **Vertical slice architecture** within each module [17]. Each feature is a self-contained folder with handler, endpoint, and validator.
- **Python sidecar.** Embedding generation runs in a dedicated FastAPI service over HTTP, isolated from the .NET runtime.

This combination gives the simplicity of monolith deployment, code isolation similar to microservices, and machine learning capability, without the extra operational cost of a full microservices setup [15].

1.5.2 .NET Backend

The backend is built on .NET 10, a high-performance runtime with ahead-of-time compilation and native asynchronous I/O [19]. Its architecture is organised around five core libraries:

- **Carter** extends ASP.NET minimal APIs with module-based endpoint registration. Each business module declares its own routes independently, keeping endpoint definitions co-located with their handlers rather than centralised in a startup file.
- **MediatR** implements CQRS, routing commands (writes) and queries (reads) to handlers through an in-process message bus [18]. Handlers are discovered at startup and dispatched by request type, with no direct coupling between modules.
- **Entity Framework Core** maps C# domain objects to PostgreSQL tables, including pgvector column types for embedding storage [20]. Migrations are version-controlled and applied at startup.
- **FluentValidation** enforces input rules at the application boundary. Each request type has an associated validator that runs before the handler executes, rejecting invalid data before it reaches business logic.
- **Vertical slice architecture** [17] (described in Section 1.5.1) organises each feature as a self-contained folder containing the handler, request, response,

endpoint, and validator. This colocation keeps feature concerns together rather than spread across technical layers.

1.5.3 Vue.js Frontend

The frontend uses Vue 3 with TypeScript and the Vite build tool [21]. Two surfaces share a common component library and state management:

- **Storefront.** Product catalog browsing with category trees, faceted filters (price, size, colour), and paginated result grids. Visual search with image upload, similarity-ranked results displaying thumbnail, price, and similarity score. Cart management with session-based guest support and a multi-step checkout flow spanning address entry, delivery selection, payment confirmation, and order completion.
- **Administration interface.** Complete lifecycle management for products, variants, taxonomies, and option types. Order processing with fulfilment status tracking. User and role administration with permission assignment. Analytics overview summarising sales and inventory metrics.
- **Pinia**, a state management library, organises client-side state through typed stores. Each bounded context has its own store (catalog, cart, auth, orders), mirroring the backend module boundaries.

1.5.4 PostgreSQL and pgvector

PostgreSQL 17 hosts both relational business data and vector embeddings in a single database [13].

- **Relational schema.** Tables for products, variants, orders, users, and supporting entities are organised by bounded context. Foreign keys reference entities within the same context; cross-context references use identifier columns without database-level constraints, preserving logical isolation.
- **Performance.** Composite indexes on query-critical combinations (user status, session status, product slug) optimise frequent access patterns. Query plans combine vector similarity and relational filtering in a single execution.

The pgvector extension, HNSW indexing, and transactional consistency between product data and embeddings are described in detail in Section 1.4.

1.5.5 Redis Caching

Redis 7 operates alongside the .NET **HybridCache** abstraction in a two-tier arrangement [22].

- **L1: in-process.** Frequently accessed data (taxonomy trees, front-page product lists) is held in application memory with sub-millisecond read latency. Cache entries expire on a configurable sliding window, typically five minutes.
- **L2: Redis.** The shared tier synchronises cache across application instances. Redis also backs Hangfire job queues and guest session storage. On cache miss at L1, the value is retrieved from Redis and promoted to L1, ensuring subsequent hits stay in-process.

1.5.6 Python ML Sidecar

The machine learning capability runs as a dedicated Python 3.12 service, isolated from the .NET backend due to incompatible runtime dependencies (PyTorch requires Python, .NET requires the CLR) [23].

- **Framework.** FastAPI provides async HTTP endpoints with automatic OpenAPI schema generation. Uvicorn is the ASGI runtime.
- **Model management.** A singleton **ModelManager** lazy-loads models from the HuggingFace hub on first request. Once loaded, models persist in GPU memory (or CPU, if no GPU is available) for the lifetime of the service. The manager supports multiple architectures through a common embedding interface.
- **API surface.** *POST /embeddings* accepts raw image bytes (JPEG, PNG, WebP) and returns a JSON array of floating-point values. *GET /health* reports the currently loaded model, its embedding dimension, and the most recent inference latency.
- **Security.** Requests require an *X-API-Key* header validated at the middleware layer. The sidecar listens only on the internal Docker network, inaccessible from the public internet.

1.5.7 Hangfire Background Jobs

Hangfire processes operations that should not block HTTP requests [24]. Jobs are persisted in Redis for resilience across application restarts.

- **Cart expiry.** A recurring job runs hourly, removing carts with no activity for seven days. This prevents abandoned carts from accumulating indefinitely.
- **Embedding queue.** When an admin uploads new product images, embedding generation tasks are enqueued for asynchronous processing. The upload endpoint returns immediately; the embedding appears in search results once the job completes.

1.5.8 Identity, Authentication, and Authorisation

- **ASP.NET Identity** provides user account management: password hashing with salted PBKDF2, email confirmation workflows, and optional two-factor authentication via TOTP [19].
- **JWT tokens.** Access tokens carry claims (user ID, roles, permissions) and expire after 15 minutes [25]. Refresh tokens enable silent renewal: the client exchanges a refresh token for a new access token, and each refresh token is single-use. Reuse of an already-consumed refresh token triggers revocation of all tokens for that user, mitigating token theft.
- **Guest sessions.** Anonymous users receive a signed session cookie. This cookie links to a server-side session backed by Redis, enabling cart operations and product browsing without authentication. On registration or login, the guest session is transferred to the authenticated user context.
- **Permission model.** Authorisation uses granular claims in the format `domain.category.resource.action` (e.g., `admin.catalog.products.create`). Roles aggregate commonly used permission sets. Endpoint-level attributes resolve claims at the middleware layer, so handler code contains no authorisation logic.

1.5.9 Benchmark Framework

The benchmark framework is a Python 3.12 pipeline for systematic comparison of embedding models [23], operating separately from the application codebase. Experimental results are reported in Chapter 3.

- **Modes.** One-shot comparison, three-fold cross-validation with stratified category splits (default), and pgvector pipeline mode measuring end-to-end latency.
- **Models.** The benchmarking harness supports 11 architectures: CNN (ResNet-50, ResNet-101, ResNet-152, EfficientNet-B0, EfficientNet-B4), ViT (DINOv2 ViT-S/14, DINOv2 ViT-B/14), CLIP (ViT-B/32, ViT-B/16, ViT-L/14, Fashion-CLIP). Each has a thin adapter implementing *generate_embeddings*. Of these, six representative models are benchmarked in Chapter 3: Fashion-CLIP, DINOv2 ViT-S/14, CLIP ViT-B/16, CLIP ViT-B/32, ResNet-50, and EfficientNet-B0.
- **Caching.** Embeddings cached per model, fold, and split to avoid recomputation.
- **Metrics.** Retrieval accuracy: mAP, Precision at K, Recall at K, nDCG. Efficiency: inference latency, throughput, model load time, storage, RAM.
- **Outputs.** JSON, CSV, Markdown, and Typst table formats embed directly into the thesis without manual transcription.

- **Multi-label pipeline.** Enriched-dataset mode benchmarks three label schemes (category; category+colour; category+colour+pattern).

1.5.10 Technology Stack Summary

The preceding sections introduced the principal technologies that compose the ReSys.Shop platform. Table 8 consolidates the complete stack.

Table 8: Technology stack of the ReSys.Shop platform

Layer	Technology	Role
Frontend	Vue 3, TypeScript, Vite	Customer storefront and administration interface
Backend API	.NET 10, Carter, MediatR	REST endpoints via minimal APIs with CQRS across business modules
Database	PostgreSQL, pgvector	Relational data and vector embeddings with HNSW-indexed similarity search
Caching	Redis, Hybrid-Cache	Two-tier cache (L1 in-memory, L2 Redis); Hangfire job queue backing store
ML Sidecar	Python 3.12, FastAPI, PyTorch	Embedding generation with lazy model loading and GPU acceleration
Background Jobs	Hangfire	Persistent job processing for cart expiry, embedding queue, and maintenance
Auth and Identity	JWT, ASP.NET Identity	Access tokens, refresh rotation, permission-based authorisation
Benchmarking	Python 3.12, PyTorch	Systematic six-model comparison (of 11 supported architectures) across retrieval accuracy and efficiency

1.6 RELATED WORK AND RESEARCH GAP

This section compares the ReSys.Shop platform to existing academic research and commercial visual search systems, and explains the specific gap this thesis addresses.

1.6.1 Academic Research

The **DeepFashion** dataset, introduced by Liu et al., established the foundational benchmark for fashion recognition and retrieval with over 800,000 images annotated with attributes, landmarks, and in-shop-to-consumer photo pairs [26]. This dataset led to much of the later work in fashion AI.

FashionIQ extended retrieval to the conversational setting, where users modify queries through natural language feedback (“like this dress but shorter”) [27]. This is an interesting approach, but building an interactive dialogue system

requires infrastructure beyond the scope of this project, which focuses on single-turn visual and text queries.

The **Fashion-CLIP** work demonstrated that domain-specific fine-tuning of CLIP on 700,000 fashion images improves retrieval over the general model [6]. This thesis follows that approach, using pre-trained models without custom training, and extends the comparison to additional architectures (ResNet, EfficientNet, DINOv2) for systematic comparison. The benchmark in Chapter 3 (§3.6) measured a 1.46% relative mAP advantage of Fashion-CLIP over generic CLIP ViT-B/16 under category-only relevance.

More recently, unified multimodal embedding models such as ImageBind [28] and BLIP-2 [29] have pushed beyond the image-text pairing of CLIP, binding several modalities into shared latent spaces. These advances underpin the cross-modal text-plus-image retrieval direction discussed in this thesis’s future work (Part 3), which would reuse the CLIP-family shared latent space for hybrid queries; they were outside the scope of the single-query benchmark presented here.

1.6.2 Commercial Systems

Several platforms have deployed visual search at production scale.

Table 9: Comparison of commercial visual search products

Product	Key Strength	Limitation
Google Lens	Massive scale, general-domain coverage	Closed ecosystem, not customisable
Pinterest Lens	Over 600M monthly searches, style-aware	Proprietary, requires Pinterest integration
ASOS Style Match	Fashion-specific accuracy	Restricted to ASOS catalog only
ViSenze	API-based, good accuracy	Paid service with recurring per-query costs

These products share some common limitations for independent projects:

- They are proprietary, so they cannot be studied or changed.
- API access costs money based on how many queries are made.
- Relying on external services creates dependency on that provider.

This thesis shows that similar functionality can be built using open-source tools. This provides both a reference implementation and a lower-cost alternative for smaller deployments.

1.6.3 Contribution Differentiators

This project differs from prior work by addressing the **engineering gap** between model research and production systems. Five contributions define this gap:

1. Polyglot architecture. Python’s machine learning ecosystem (PyTorch, HuggingFace) does not natively interoperate with the .NET stack common in enterprise e-commerce. This thesis presents a modular monolith with a dedicated AI sidecar. It combines .NET’s type safety and transactional integrity with Python’s access to state-of-the-art vision models, without the operational overhead of a full microservices deployment.

2. Vector-native consistency. By using pgvector within PostgreSQL, embeddings and product metadata share the same transactional boundary. Product updates, image replacements, and index maintenance occur atomically, eliminating stale-index bugs that arise when a vector store and relational database have independent consistency guarantees.

3. Commodity hardware benchmarking. Commercial visual search runs on cloud TPU clusters. This thesis benchmarks six models on consumer-grade hardware, establishing that production-quality visual search is achievable without specialised infrastructure, lowering the barrier for small to medium e-commerce platforms.

4. Applied model comparison. Instead of chasing the highest possible benchmark scores, this thesis compares models under realistic deployment constraints (inference latency budget, memory limits, storage cost). The resulting accuracy-efficiency trade-off data, presented in Chapter 3, provides a pragmatic guide for practitioners selecting embedding models.

5. Reference CBIR implementation. The pipeline integrates pre-trained embedding models (PyTorch, FastAPI) into a production-style e-commerce platform (.NET 10, Vue 3, PostgreSQL pgvector), demonstrating that open-source tools can support competitive visual search without proprietary APIs or specialised hardware.

2 DESIGN AND IMPLEMENTATION

This chapter translates requirements into a concrete system design and describes the implementation of the principal components.

- **Requirements Specification.** Functional requirements in 27 groups, non-functional requirements, feature classification.
- **System Modeling.** Actors, work breakdown structure, 26 use cases with traceability to requirements.
- **System Architecture and Design.** Domain-driven design, C4 diagrams, database schema, API design, security model.
- **Implementation.** Technology stack, vertical slice architecture, data persistence with pgvector, ML sidecar and CBIR search flow, frontend applications.

2.1 REQUIREMENTS SPECIFICATION

The platform organises its functional requirements into 27 requirement groups across eight **business modules**. Each group enforces related domain invariants, expressed through entity validation rules and application-layer checks. Five non-functional quality dimensions define performance, security, modularity, observability, and reliability targets. These targets shaped the architectural decisions made throughout the design. Feature classification separates three **core research** contributions, detailed in Sections 2.3 and 2.4, from four **supporting infrastructure** modules. These modules provide the realistic context needed for the testing described in Section 3.2.

- **Functional Requirements.** Traceable per module: Catalog, Identity, Inventory, Ordering, Payment, Shipping, Profile, and Location.
- **Non-Functional Requirements.** Five quality dimensions with measurable, atomic constraints.
- **Feature Classification.** Core Research versus Supporting Infrastructure: scope of the thesis contribution.

2.1.1 Functional Requirements

Functional requirements are organized by business module and grouped into related clusters. Each group carries a unique identifier (for example, CAT-GRP-01) that is used throughout the design, implementation, and testing chapters to trace each use case back to the capability it relies on.

2.1.1.1 Catalog Module

Manages product lifecycle, classification, image handling, and CBIR infrastructure.

Product Lifecycle (CAT-GRP-01). Products are created with name, description, URL slug, and SEO metadata, enriched with fashion attributes such as

style code, season, material composition, care instructions, fit notes, department, and gender target. A unique slug is enforced at the database constraint level, each product progresses through Draft, Active, and Archived states with configurable availability dates, and one variant is designated as the master representation for catalog displays.

Variants & Options (CAT-GRP-02). Sellable variants combine option values into attribute configurations, each carrying its own SKU, barcode, physical dimensions, weight, and independent time-bound pricing with currency specification. Reusable option types (Size, Color, Material) define ordered values that are assigned, synchronized, retrieved, and revoked on a per-product basis.

Images & Embeddings (CAT-GRP-03). Variant images support upload with automatic thumbnail generation, multiple images per variant with display ordering, and full CRUD for metadata such as alt text. Embeddings are generated for uploaded images by the ML sidecar and stored in pgvector with model metadata; regeneration runs when the active model configuration changes or corrupted embeddings are detected.

Visual Search (CAT-GRP-04). Content-based image retrieval accepts query images, extracts embeddings, and queries pgvector via HNSW indexing with cosine distance to return ranked results. Minimum similarity thresholds and maximum result limits are configurable, embedding models are switchable via environment variables with results filtered by active model, per-variant availability is exposed through storefront endpoints, and visually similar products can be retrieved for a target product.

Taxonomy & Classification (CAT-GRP-05). Products are organized through hierarchical taxonomies with nested taxon trees; business rules on taxon nodes support update, synchronization, and retrieval, and products are automatically classified into taxons when their attributes change.

2.1.1.2 Identity Module

Authentication, authorization, and user management with JWT access tokens and refresh token rotation.

Authentication & Sessions (IDN-GRP-01). Customer accounts are registered with email, password, and basic profile information, and authenticated via email and password with a 15-minute JWT access token and refresh token. Refresh tokens rotate on each issuance and a previously consumed token revokes the whole token family, guest sessions operate through signed cookies, and users can inspect their sessions and terminate them via logout.

Credential & Email Lifecycle (IDN-GRP-02). Passwords are reset through single-use, time-limited email verification tokens and changed after

verifying the current credential; email addresses are confirmed via verification tokens with support for modification requests.

Roles & Permissions (IDN-GRP-03). Role-based access control is enforced with permission claims formatted as `domain.category.resource.action` at middleware boundaries; roles are created, updated, deleted, and listed with per-role permission assignments, and roles are assigned and revoked per user with direct permission overrides.

User Governance (IDN-GRP-04). Administrative endpoints manage user accounts, role bindings, and permission grants, and user accounts can be enabled or disabled without purging user record history.

2.1.1.3 Inventory Module

Stock tracking, checkout reservations, and audit ledgers.

Stock & Locations (INV-GRP-01). Physical stock locations are defined with address metadata and active flags; on-hand and reserved quantities are tracked per variant per location with full CRUD, bulk adjustments, and CSV imports, and inventory falling below configured thresholds drives replenishment notifications.

Reservations & Overselling (INV-GRP-02). Inventory is reserved during checkout and at cart level with status inspection and automatic release on cart expiry, cancellation, timeout, or abandonment; order confirmation is rejected when requested quantities exceed unreserved stock, and stale holds expire after a configurable timeout (default: 15 minutes).

Transfers (INV-GRP-03). Stock movements between locations are recorded with origin, destination, quantity, and timestamp, progressing through Created, In-Transit, Received, and Cancelled states with paged listings.

Audit & Movement (INV-GRP-04). An immutable audit log records stock changes with operator identity and reason codes, and detailed, paged audit trails of stock movements are maintained as first-class domain entities.

2.1.1.4 Ordering Module

Shopping cart through checkout state transitions to completed orders.

Cart (ORD-GRP-01). Guest and customer carts support variant item addition, quantity adjustment, creation, clearing, and deletion; guest cart contents merge into customer accounts upon authentication without data loss, and abandoned carts are purged after 7 days of inactivity by scheduled background maintenance.

Checkout Pipeline (ORD-GRP-02). Checkout enforces strict sequential state transitions (Address, Delivery, Payment, Confirm, Complete) with valida-

tion of inventory levels, address completeness, and shipping method selection prior to payment; applicable shipping rates are selected within cart boundaries before payment is finalized.

Order Accounting & State (ORD-GRP-03). Monetary balances are calculated independently (item total + adjustments + shipping = grand total), order numbering generates unique sequential references within database isolation transactions, finalized orders are locked as immutable, and parallel, decoupled state machine counters track payment and fulfillment state.

Order Management (ORD-GRP-04). Orders can be cancelled before fulfillment confirmation, releasing inventory holds and voiding payments; administrators approve, complete, resume, and modify pending orders through administrative surfaces, and customers access paged order histories with line-item detail, payment status, and tracking information.

2.1.1.5 Payment Module

Payment intent lifecycles across Stripe and internal test gateways.

Payment Intents (PAY-GRP-01). Payment intents specify amount, currency, order reference, and gateway target, and are confirmed to authorize fund capture during checkout finalization; Stripe SetupIntents securely save customer payment methods for future checkouts.

Capture, Refund & Void (PAY-GRP-02). Capture, void, and refund operations execute with idempotency keys; validation prevents refund amounts from exceeding total captured funds, and authorized-but-uncaptured payments are voided on cancelled orders to release authorization holds.

Gateway Integration & State (PAY-GRP-03). Incoming gateway webhooks are verified with HMAC cryptographic signatures and processed with idempotency; local payment state records run parallel to external gateway records for offline queries, and a mock gateway simulates transaction lifecycles without external API calls.

Method Configuration (PAY-GRP-04). Administrators create, update, toggle, and delete configured payment gateway methods through admin panels.

2.1.1.6 Shipping Module

Delivery options, geographic zones, and rate calculation.

Delivery Methods & Rates (SHP-GRP-01). Shipping methods are configured with carrier details, rate rules, and geographic zones; administrative endpoints create, update, activate, deactivate, and delete methods, and rate matrices map to methods, zones, and weight/value tiers.

Rate Calculation (SHP-GRP-02). Shipping costs are calculated dynamically from address zone, method, cart weight, and total value, and eligible methods and rates are calculated and displayed during storefront checkout.

Shipment Tracking (SHP-GRP-03). Active shipments progress through Pending, Ready, Shipped, and Delivered states via administrative status updates.

2.1.1.7 Profile Module

Customer preferences, addresses, and saved items.

Address Book & Saved Items (PRF-GRP-01). Shipping and billing addresses are managed with default selections and custom labels, and named wishlists contain targeted product variants with notes.

Notification Preferences (PRF-GRP-02). Channel-specific notification preferences (email, SMS) are managed with opt-in flags.

2.1.1.8 Location Module

Reference data for countries and states used in address validation and shipping zones.

Country & State Directories (LOC-GRP-01). Country reference entries carry ISO 3166-1 alpha-2 codes, names, and active flags; state and province entries carry ISO 3166-2 codes linked to parent countries. Both support full CRUD by database ID or ISO code, with states listed and filtered by parent country.

2.1.2 Non-Functional Requirements

Five quality dimensions define production-readiness constraints with atomic, measurable targets. Each dimension forms a single requirement group whose identifier (for example, NFR-GRP-01) is used when a design decision refers back to these targets. Verification is assessed in Chapter 3.

Performance (NFR-GRP-01). End-to-end CBIR search workflows complete within one second per query, non-search REST endpoints respond within 200 milliseconds under standard load, and asynchronous I/O is enforced across all data-access and HTTP pipelines.

Security (NFR-GRP-02). JWT access tokens [25] expire after 15 minutes with single-use refresh token rotation, and fine-grained claims (domain.category.resource.action) are enforced at the API middleware boundary. Per-IP fixed-window rate limiting protects authentication, registration, password reset, payment, and webhook endpoints; uploads are validated via magic-byte header inspection with extension allowlists and a 10 MB cap; and security headers (CSP, X-Frame-Options, X-Content-Type-Options, Referrer-

Policy, Permissions-Policy) are injected by middleware, with HSTS applied by the reverse-proxy layer in deployment.

Modularity (NFR-GRP-03). All business modules compile into one Module assembly with forward-only dependencies (Shared $\rightarrow$ Module $\rightarrow$ Api); inter-module behaviour routes through MediatR in-process dispatch where pipeline value (validation, logging, transactions) applies, while direct service calls and navigations are used where they fit the feature slice. Each module is independently testable without initialising foreign contexts.

Observability (NFR-GRP-04). OpenTelemetry trace contexts propagate across the .NET API and the Python ML sidecar, structured logging attaches a unique correlation identifier to every log entry, and health monitoring exposes /health/live and /health/ready endpoints verifying database and cache connectivity, alongside the sidecar’s own /health and /alive endpoints.

Reliability (NFR-GRP-05). Background tasks run on Hangfire [24] backed by Redis [22] so they survive process restarts; scheduled maintenance purges carts inactive for 7 days on an hourly job and releases their associated inventory reservations, Stripe webhooks are guarded by idempotency keys against duplicate state updates, and unconfirmed checkout inventory holds expire after 15 minutes of inactivity.

2.1.3 Feature Classification

Platform features are categorized into Core Research contributions and Supporting Infrastructure to define thesis scope. Core contributions are detailed in Sections 2.3 and 2.4; supporting modules provide a realistic e-commerce environment for the benchmark experiments (Chapter 3).

Table 10: Feature areas classified as Core Research vs Supporting Infrastructure.

Feature Area	Classification	Rationale
Visual Search (CBIR)	Core Research	Primary Contribution: Integrated multi-model CBIR system [30] with a pluggable architecture for image-based product discovery across CNNs [3], ViTs [10], and CLIP-based architectures [5], [6].
ML Embedding Pipeline	Core Research	Operational Backbone: Automated pipeline processing product images, generating vector embeddings via a PyTorch sidecar [23], and managing pgvector HNSW indexing [12], [13].
Model Benchmark System	Core Research	Secondary Contribution: Systematic benchmarking of retrieval accuracy and latency across

Feature Area	Classification	Rationale
		six embedding models, providing model selection guidelines for deployment.
Product Catalog	Supporting	Benchmark Domain: Structured fashion product data serving as the vector search target.
Order System	Supporting	Conversion Tracking: Shopping workflows capturing cart additions and checkouts to validate visual search utility.
Inventory	Supporting	Availability Constraints: Filters search queries by real-time stock levels.
Authentication	Supporting	Security Boundary: Protects administrative surfaces and user session state.

2.2 SYSTEM MODELING

Three actors interact with the platform across 26 use cases in total. These are organised into a functional work breakdown structure (WBS) and a summary matrix with detailed scenario specifications. The use case specifications provide full traceability to the functional requirements defined in Section 2.1.

- **System Actors.** Customer, Administrator, and System: roles, responsibilities, interaction surfaces.
- **Functional Decomposition.** WBS: three functional areas with constituent modules and sub-functions.
- **Use Cases.** 26 specifications: summary matrix, detailed scenarios with traceability.

2.2.1 System Actors

Three categories of actors interact with the platform, separated by access level and interaction surface. Individual use cases may additionally reference supporting external systems (ML Service, Payment Gateway, Email Service) under a Support field, a standard UML convention distinct from the three primary actors.

2.2.1.1 Customer

The **Customer** accesses the browser-based **storefront**.

- **Discovery.** Browse catalog with faceted filters, keyword search, and visual CBIR (Section 2.3.3). Guests browse and cart without registration; session promoted on login.
- **Purchase.** Persistent cart, multi-step checkout (address, delivery, payment, confirm).

- **Account.** Register or log in; manage profile, addresses, wishlists, order history.

2.2.1.2 Administrator

The **Administrator** operates a separate administration interface.

- **Catalog.** CRUD products with fashion metadata; variants and pricing; image uploads triggering embedding pipeline; taxonomies, option types, product classification.
- **Operations.** Order review, payment capture/refund, shipment management; real-time stock monitoring per location with audit history.
- **Governance.** User CRUD; role assignment (Customer, Administrator); permission grants (*domain.category.resource.action*).

2.2.1.3 System

The **System** actor runs automated background jobs via **Hangfire** [24] and **Redis** [22].

- **Embedding Generation.** Process uploaded images via ML sidecar.
- **Cart Expiry.** Hourly job: delete carts inactive 7 days, release inventory.
- **Inventory Reservation.** Hold stock during checkout; expire after 15-minute inactivity.
- **Maintenance.** Per-model HNSW index initialisation at startup; async payment webhook validation.

2.2.2 Functional Decomposition

The platform decomposes into three functional areas, whose work breakdown is illustrated by Figure 6 in Section 2.2.3 (with each area’s use cases enumerated in the summary matrix Table 11):

- **Administration.** Seven modules: Catalog, Ordering, Payment, Inventory, Identity, Shipping, Location.
- **Storefront.** Three modules: Product Discovery (browse, search, visual search), Purchase Flow (cart, checkout, payment, order history), Account Management (authentication, session, profile).
- **Background Services.** Five processes: Embedding Generation, Cart Expiry, Inventory Reservation, Payment Webhook Processing, Index Optimisation.

2.2.3 Use Case Overview

The platform’s 26 use cases are organised around three actors and grouped by business module, as summarised in Table 11 and structured by the work breakdown in Figure 6. The Administrator interacts with seven modules (Catalog, Ordering, Payment, Inventory, Identity, Shipping, Location); the Customer in-

teracts with five modules (Catalog, Ordering, Payment, Identity, Profile); and the System performs background operations through the System Services module.

These modules map onto the feature classification of Section 2.1.3. The Core Research contributions (Visual Search, the ML embedding pipeline, and the model benchmark system) are embodied by UC-STR-SRC (Visual Search), UC-SYS-EMB (Embedding Operations), and UC-ADM-IMG (Image and Embedding Management). The Supporting infrastructure areas (Product Catalog, Order System, Inventory, Authentication) retain full detail across their catalog, ordering, inventory, and identity use cases. Peripheral modules absent from that classification (Payment, Shipping, Profile, Location, and System Maintenance) are specified in condensed form in the following sections.

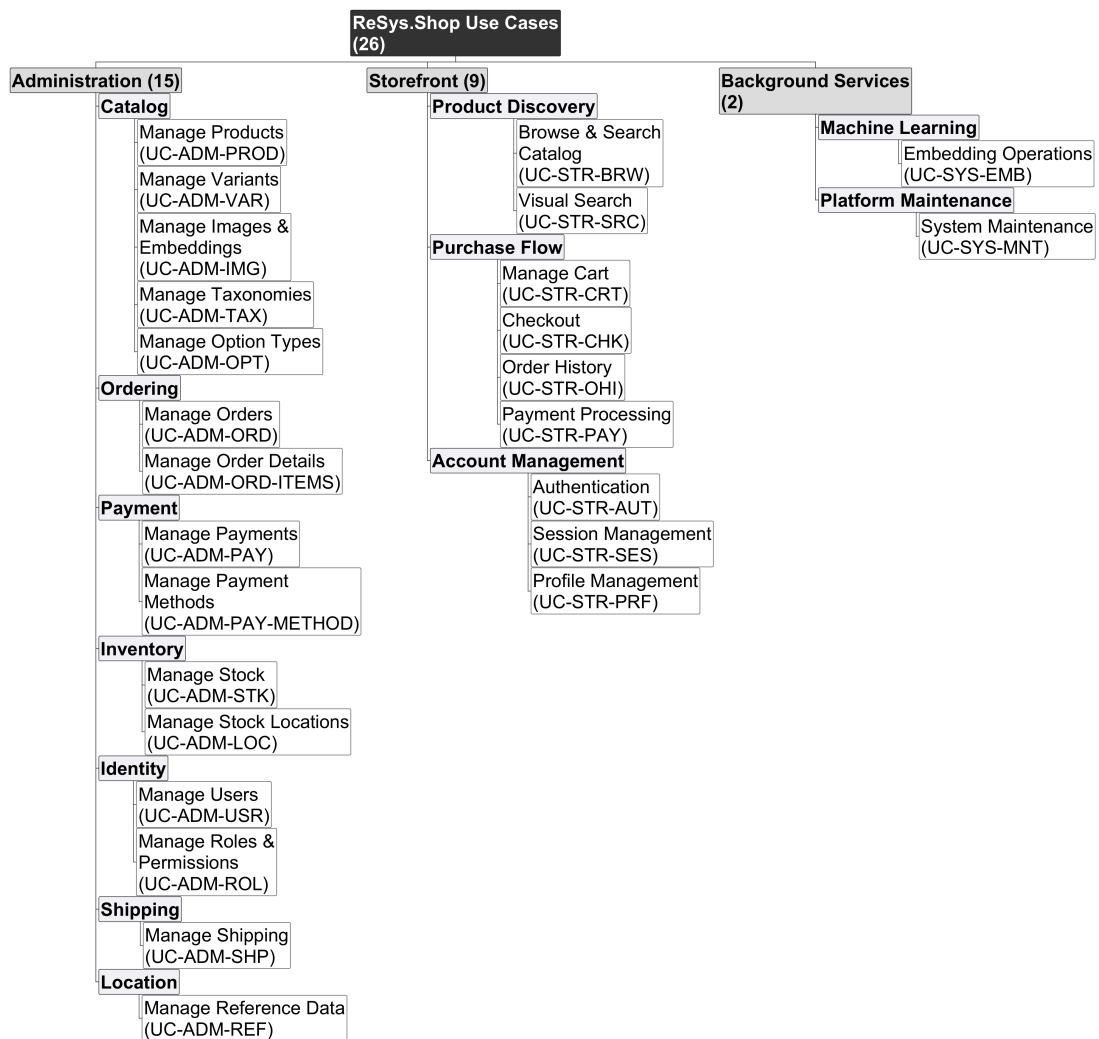


Figure 6: Work breakdown of the 26 use cases into the Administration, Storefront, and Background Services work areas.

The work breakdown above decomposes the 26 use cases into the three actor work areas (Administration, Storefront, Background Services); Table 11 below

provides the traceability from every use case to the functional requirements defined in Section 2.1.

Table 11: All 26 use cases with actor, module, and requirement traceability.

UC-ID	Use Case	Actor	Module	Related FR group
UC-ADM-PROD	Manage products	Admin	Catalog	CAT-GRP-01, CAT-GRP-02
UC-ADM-VAR	Manage variants	Admin	Catalog	CAT-GRP-02
UC-ADM-IMG	Manage images and embeddings	Admin	Catalog	CAT-GRP-03
UC-ADM-TAX	Manage taxonomies and classification	Admin	Catalog	CAT-GRP-05
UC-ADM-OPT	Manage option types	Admin	Catalog	CAT-GRP-02
UC-ADM-ORD	Manage orders	Admin	Ordering	ORD-GRP-02, ORD-GRP-03, ORD-GRP-04
UC-ADM-ORD-ITEMS	Manage order details	Admin	Ordering	ORD-GRP-04
UC-ADM-PAY	Manage payments	Admin	Payment	PAY-GRP-02, PAY-GRP-03
UC-ADM-PAY-METHOD	Manage payment methods	Admin	Payment	PAY-GRP-04
UC-ADM-STK	Manage stock	Admin	Inventory	INV-GRP-01, INV-GRP-03, INV-GRP-04
UC-ADM-LOC	Manage stock locations	Admin	Inventory	INV-GRP-01
UC-ADM-USR	Manage users	Admin	Identity	IDN-GRP-04
UC-ADM-ROL	Manage roles and permissions	Admin	Identity	IDN-GRP-03

UC-ID	Use Case	Actor	Module	Related FR group
UC-ADM-SHP	Manage shipping	Admin	Shipping	SHP-GRP-01
UC-ADM-REF	Manage reference data	Admin	Location	LOC-GRP-01
UC-STR-BRW	Browse and search catalog	Customer	Catalog	CAT-GRP-01, CAT-GRP-02, CAT-GRP-04, CAT-GRP-05
UC-STR-SRC	Visual search	Customer	Catalog	CAT-GRP-04
UC-STR-CRT	Manage cart	Customer	Ordering	ORD-GRP-01
UC-STR-CHK	Checkout	Customer	Ordering	ORD-GRP-02, ORD-GRP-03
UC-STR-OHI	Order history	Customer	Ordering	ORD-GRP-04
UC-STR-PAY	Payment processing	Customer	Payment	PAY-GRP-01
UC-STR-AUT	Authentication	Customer	Identity	IDN-GRP-01, IDN-GRP-02
UC-STR-SES	Session management	Customer	Identity	IDN-GRP-01
UC-STR-PRF	Profile management	Customer	Profile	PRF-GRP-01, PRF-GRP-02
UC-SYS-EMB	Embedding operations	System	Embedding	CAT-GRP-03
UC-SYS-MNT	System maintenance	System	Infrastructure	CAT-GRP-04, ORD-GRP-01, INV-GRP-02, PAY-GRP-03

2.2.4 Administrator Use Cases

The Administrator actor manages data and daily operations across all business modules through a dedicated administration interface. Each specification includes the actor's goal, trigger, preconditions, postconditions, a numbered

main success scenario, alternative and exception flows, and related functional requirements.

2.2.4.1 Product Management

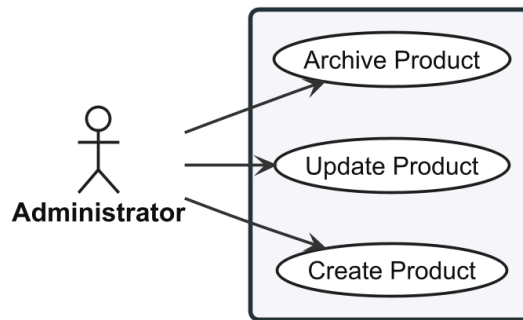


Figure 7: Use case diagram for Product Management (UC-ADM-PROD).

2.2.4.2 UC-ADM-PROD: Manage Products

Table 12: Manage Products.

Use Case	UC-ADM-PROD: Manage Products
Actor	Administrator
Goal	Create, update, and archive products in the catalog.
Pre/Post	Pre: authenticated with catalog management permissions. Post: product catalog reflects the performed operation; status transitions logged for audit.
Scenario	Create Product 1. Selects create option. 2. System presents form. 3. Enters product name, description, slug, and fashion metadata (style code, season, material, department, gender target). 4. Defines at least one variant with SKU and price as master variant. 5. Assigns product to relevant taxons. 6. Submits; system validates slug uniqueness, persists, confirms creation with Draft status. , Update Product 1. Selects product from catalog listing. 2. System displays edit form with current values. 3. Modifies fields (name, description, slug, fashion metadata, SEO attributes, or status). 4. Submits; system validates, persists, confirms update. , Archive Product 1. Selects product, chooses archive option. 2. System requests confirmation. 3. Confirms; product status changes to Archived, hidden from storefront.

	,
Alternatives	1. A1. Slug not unique (Create/Update) → system rejects, prompts for different slug. 2. A2. No master variant (Create) → system rejects, instructs to designate one. 3. A3. Product has active orders (Archive) → system warns, requests explicit confirmation.
Exceptions	1. E1. System fails to persist → reports failure, retains form data for retry.
Requirements	CAT-GRP-01, CAT-GRP-02

2.2.4.3 Variant and Pricing

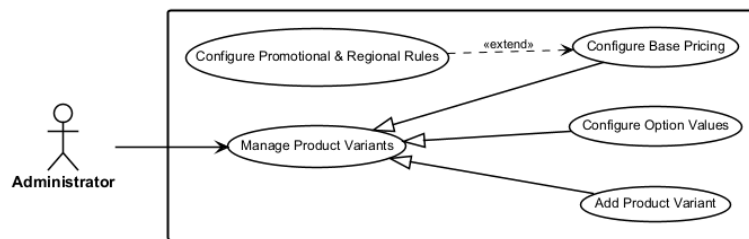


Figure 8: Use case diagram for Variant and Pricing (UC-ADM-VAR).

2.2.4.4 UC-ADM-VAR: Manage Variants

Table 13: Manage Variants.

Use Case	UC-ADM-VAR: Manage Variants
Actor	Administrator
Goal	Add and manage product variants including option-value configuration and pricing.
Pre/Post	Pre: authenticated with catalog permissions; parent product exists. Post: variant created or updated with option assignments and pricing.
Scenario	Add Variant 1. Navigates to product detail, selects add variant. 2. System presents variant creation form. 3. Enters variant details (SKU, barcode, dimensions, weight, position). 4. Assigns option values (e.g. Size M, Colour Red) from available option types. 5. Submits; system validates SKU uniqueness and option combination, creates variant, confirms. Configure Options 1. Selects variant from product detail page. 2. System displays variant detail with current option assignments.

	3. Selects option type and chooses value, repeats for additional types. 4. Saves; system validates combination, persists, confirms change. , Configure Pricing 1. Navigates to pricing management for product. 2. System displays current prices for all variants grouped by currency. 3. Selects variants and specifies new price with currency and optional validity dates. 4. Submits; system validates non-negative prices and valid currency, persists, confirms. ,
Alternatives	1. A1. SKU not unique → system rejects, prompts for different SKU. 2. A2. Duplicate option combination → system rejects, highlights conflict. 3. A3. Zero price → system accepts but warns variant appears as free. 4. A4. Overlapping date ranges for same variant and currency → system rejects.
Exceptions	1. E1. Persistence failure → system reports, retains form data for retry.
Requirements	CAT-GRP-02

2.2.4.5 Image and Embedding Management

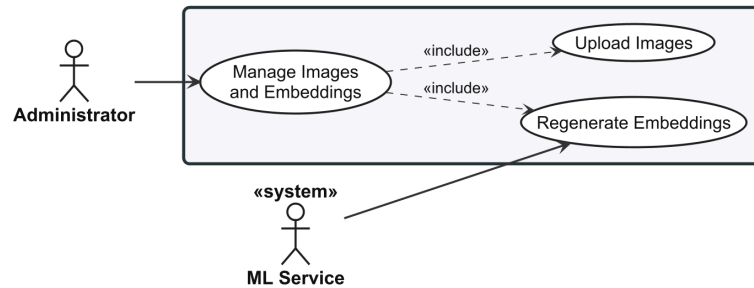


Figure 9: Use case diagram for Image and Embedding Management (UC-ADM-IMG).

2.2.4.6 UC-ADM-IMG: Manage Images and Embeddings

Table 14: Manage Images and Embeddings.

Use Case	UC-ADM-IMG: Manage Images and Embeddings
Actor	Administrator
Support	ML Service
Goal	Upload variant images and manage embedding generation.

Pre/Post	Pre: authenticated with image management permissions; variant exists. Post: image stored and associated with variant; embeddings available for visual search.
Scenario	Upload Images 1. Selects product variant, initiates image upload. 2. Selects image file from local file system. 3. System validates format (JPEG, PNG, WebP) and size (max 10 MB). 4. Provides alt text and display order position. 5. Confirms, submits; system stores image, generates thumbnails, schedules embedding generation, confirms upload. Regenerate Embeddings 1. Navigates to image management, selects images. 2. Initiates regenerate embeddings action. 3. System displays confirmation with affected image count. 4. Confirms; system sends each image to ML service for embedding generation, stores embeddings with model metadata, reports completion with success count.
Alternatives	1. A1. Unsupported format → system rejects, lists accepted formats. 2. A2. Exceeds max size → system rejects, displays size constraint. 3. A3. No images selected for regeneration → system disables action, prompts to select.
Exceptions	1. E1. ML service unavailable → system reports failure, suggests retry when operational. 2. E2. Processing cannot be scheduled → system stores image, notifies search will exclude it until processing succeeds.
Requirements	CAT-GRP-03

2.2.4.7 Taxonomy and Classification



Figure 10: Use case diagram for Taxonomy and Classification (UC-ADM-TAX).

2.2.4.8 UC-ADM-TAX: Manage Taxonomies and Classification

Table 15: Manage Taxonomies and Classification.

Use Case	UC-ADM-TAX: Manage Taxonomies and Classification
Actor	Administrator

Goal	Create and modify taxonomy structures and assign product classifications.
Pre/Post	Pre: authenticated with taxonomy management permissions. Post: taxonomy structure and product classifications updated.
Scenario	Manage Taxonomies 1. Navigates to taxonomy management. 2. System displays taxonomy tree with existing taxons. 3. Creates new taxonomy root or selects existing taxonomy. 4. Adds, edits, reorders, or removes taxon nodes, optionally defines business rules. 5. Saves; system persists updated taxonomy, confirms. Classify Products 1. Selects products from catalog listing. 2. Opens classification panel. 3. System displays taxonomy tree with current classifications. 4. Selects taxons to assign or deselects to remove. 5. Saves; system validates each taxon path, persists associations, confirms.
Alternatives	1. A1. Delete taxon with children → system prompts to cascade-delete or reassign. 2. A2. Delete taxon with products → system warns products lose classification. 3. A3. All taxons removed from product → system warns product will not appear in category browsing.
Exceptions	1. E1. Concurrent modification → system refreshes data, asks to retry.
Requirements	CAT-GRP-05

2.2.4.9 Option Type Configuration

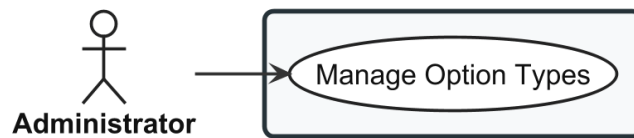


Figure 11: Use case diagram for Option Type Configuration (UC-ADM-OPT).

2.2.4.10 UC-ADM-OPT: Manage Option Types

Table 16: Manage Option Types.

Use Case	UC-ADM-OPT: Manage Option Types
Actor	Administrator
Goal	Create, modify, and remove option types and their associated values.

Pre/Post	Pre: authenticated with option type management permissions. Post: option types updated and available for product configuration.
Scenario	1. Navigates to option type management. 2. System displays all option types with their values. 3. Creates new option type with name and presentation style. 4. Adds ordered option values (e.g. S, M, L, XL for Size). 5. Optionally edits, reorders, or removes existing types and values. 6. Saves; system validates name uniqueness and non-empty values, persists, confirms.
Alternatives	1. A1. Delete type in use by products → system warns, asks for confirmation. 2. A2. Remove value in use by variants → system warns, asks for confirmation.
Exceptions	1. E1. Concurrent modification → system refreshes data, asks to retry.
Requirements	CAT-GRP-02

2.2.4.11 Order Lifecycle

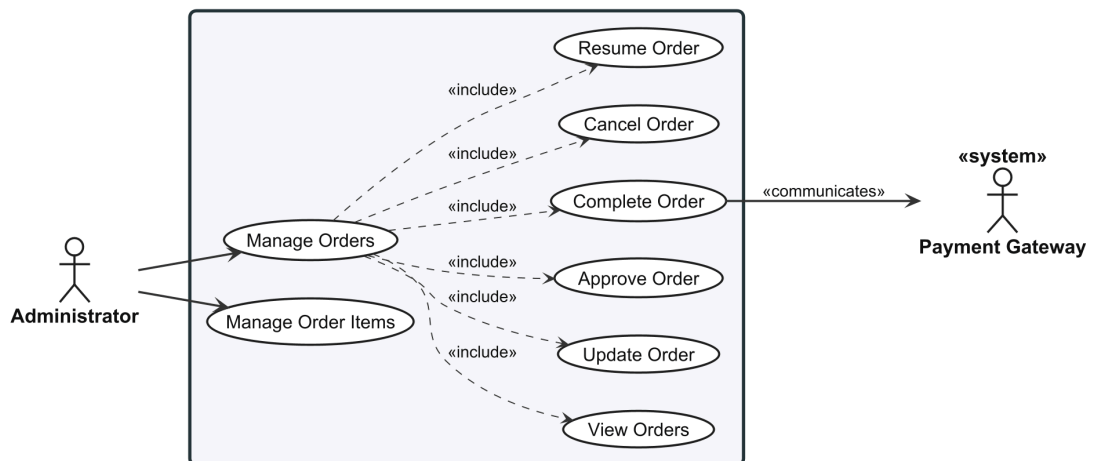


Figure 12: Use case diagram for Order Lifecycle (UC-ADM-ORD, UC-ADM-ORD-ITEMS).

2.2.4.12 UC-ADM-ORD: Manage Orders

Table 17: Manage Orders.

Use Case	UC-ADM-ORD: Manage Orders
Actor	Administrator
Support	Payment Gateway
Goal	View, modify, and manage the lifecycle of customer orders.
Pre/Post	Pre: authenticated with order management permissions. Post: order state transitions performed; status logged for audit.

Scenario	View Orders 1. Navigates to order management. 2. System displays order list sorted by most recent. 3. Applies optional filters (status, date range, customer email). 4. Selects order to view detail with line items, pricing, payment, shipment, addresses, timeline. , Update Order 1. Opens order, initiates edit. 2. System presents editable form with current values. 3. Modifies line items, addresses, or shipping method. 4. Submits; system validates modifications, recalculates totals, persists, confirms. , Approve Order 1. Opens pending order, verifies payment capture and inventory availability. 2. Selects approve; system validates, transitions order to approved, confirms. , Complete Order 1. Opens order in fulfilment state. 2. Verifies shipment dispatched. 3. Selects complete. 4. System displays confirmation. 5. Confirms; system decrements on-hand inventory, transitions to completed, logs, confirms. , Cancel Order 1. Opens order, selects cancel. 2. System displays confirmation with consequences summary. 3. Provides cancellation reason, confirms. 4. System releases reserved inventory, voids or refunds payment, transitions to cancelled, confirms. , Resume Order 1. Locates paused or stalled order. 2. Resolves underlying issue. 3. Selects resume; system validates prerequisites, transitions back to pending, confirms. ,
Alternatives	1. A1. No orders match → system displays empty message, suggests broadening filters. 2. A2. Payment not captured (Approve) → system prevents, suggests capturing first. 3. A3. Payment gateway unreachable (Cancel) → system cancels order, releases inventory, queues payment action. 4. A4. Prerequisites not met (Resume) → system prevents, displays issues to resolve. 5. A5. Concurrent state change → system refreshes, notifies.
Exceptions	1. E1. Order became immutable concurrently → system refreshes, notifies order is no longer editable. 2. E2. Payment gateway state mismatch → system prevents, advises verifying with gateway.

	3. E3. Inventory decrement data conflict → system reports, suggests verifying stock levels.
Requirements	ORD-GRP-02, ORD-GRP-03, ORD-GRP-04

2.2.4.13 UC-ADM-ORD-ITEMS: Manage Order Details

Table 18: Manage Order Details.

Use Case	UC-ADM-ORD-ITEMS: Manage Order Details
Actor	Administrator
Goal	Manage line items, shipping address, and billing address on existing orders.
Pre/Post	Pre: authenticated with order update permissions; order is in a mutable state. Post: order details updated with recalculated totals; change logged.
Scenario	1. Opens order from listing. 2. System displays order detail. 3. Initiates edit action. 4. System presents editable form with current values. 5. Modifies line items (add, update, remove), shipping address, or billing address. 6. Submits; system validates modifications (stock, address completeness), recalculates totals, persists, confirms.
Alternatives	1. A1. Quantity exceeds stock → system rejects, shows max available. 2. A2. All line items removed → system rejects, warns at least one is required. 3. A3. Address incompatible with shipping method → system warns, prompts method change.
Exceptions	1. E1. Order became immutable concurrently → system refreshes, notifies order is no longer editable.
Requirements	ORD-GRP-04

2.2.4.14 Payment Processing

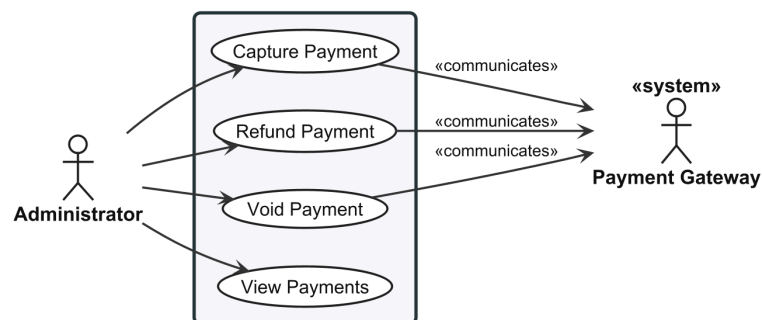


Figure 13: Use case diagram for Payment Processing (UC-ADM-PAY).

2.2.4.15 UC-ADM-PAY: Manage Payments

Goal: Capture, refund, void, and review payments. **Trigger:** the administrator opens a payment detail view for an order. **Related requirements:** PAY-GRP-02, PAY-GRP-03. The flow supports partial or full capture and refund through the gateway using idempotency keys, voiding of authorised-but-uncaptured payments, and filtered payment review; alternatives handle state mismatches and already-captured voids, and exceptions cover gateway rejection or unavailability with safe retry.

2.2.4.16 Payment Method Configuration

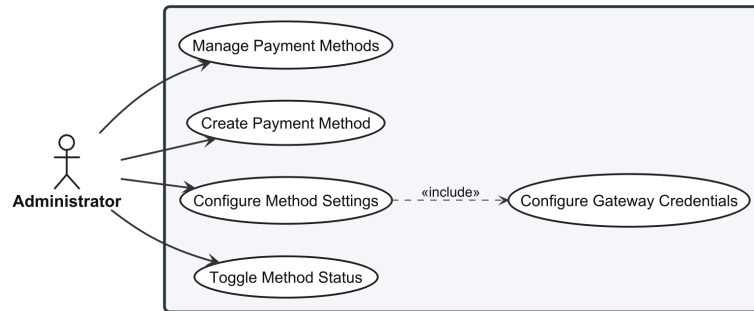


Figure 14: Use case diagram for Payment Method Configuration (UC-ADM-PAY-METHOD).

2.2.4.17 UC-ADM-PAY-METHOD: Manage Payment Methods

Goal: Create, update, activate, and deactivate payment methods. **Trigger:** the administrator opens payment method configuration. **Related requirements:** PAY-GRP-04. The flow creates, edits, activates, deactivates, or removes methods with name and gateway validation; alternatives warn when a method is in use or has pending payments, and an exception handles concurrent modification.

2.2.4.18 Stock Location Management

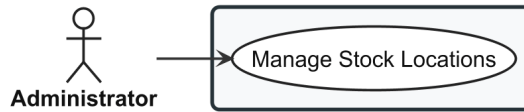


Figure 15: Use case diagram for Stock Location Management (UC-ADM-LOC).

2.2.4.19 UC-ADM-LOC: Manage Stock Locations

Table 19: Manage Stock Locations.

Use Case	UC-ADM-LOC: Manage Stock Locations
Actor	Administrator
Goal	Create, update, and remove stock locations.

Pre/Post	Pre: authenticated with location management permissions. Post: location configuration updated.
Scenario	1. Navigates to stock location management. 2. System displays existing stock locations with addresses and active status. 3. Creates new location with name, address, and active status flag. 4. Optionally designates new location as default for stock intake. 5. Optionally edits, deactivates, or removes existing locations. 6. Saves; system validates location name uniqueness, persists, confirms.
Alternatives	1. A1. Delete location with active stock → system prevents, requires transfer first. 2. A2. Deactivate location → system prevents new intake but allows existing movements. 3. A3. Delete last location → system prevents; at least one must remain.
Exceptions	1. E1. Concurrent modification → system refreshes, asks to retry.
Requirements	INV-GRP-01

2.2.4.20 Stock Item Management

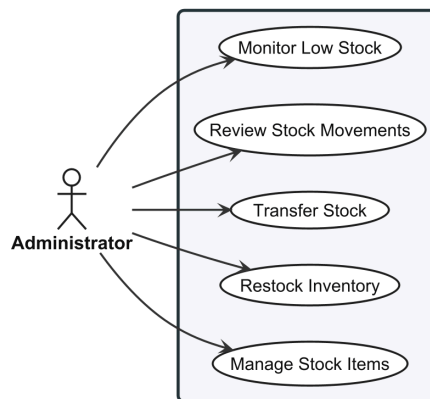


Figure 16: Use case diagram for Stock Item Management (UC-ADM-STK).

2.2.4.21 UC-ADM-STK: Manage Stock

Table 20: Manage Stock.

Use Case	UC-ADM-STK: Manage Stock
Actor	Administrator
Goal	Create, update, restock, transfer, and monitor stock levels.
Pre/Post	Pre: authenticated with stock management permissions. Post: stock quantities updated; changes logged for audit.
Scenario	Manage Stock Items 1. Navigates to stock item management.

	2. System displays stock items with variant, location, on-hand, and reserved quantities. 3. Creates stock item by selecting variant and location, enters initial on-hand quantity. 4. Alternatively selects existing stock item and updates on-hand quantity. 5. Provides reason for adjustment. 6. Saves; system validates variant-location uniqueness, persists, records audit log, confirms. , Restock Inventory 1. Locates stock item in management interface. 2. Enters restock quantity, provides reference and notes. 3. Confirms; system increments on-hand quantity, records restock event, confirms updated quantities. , Transfer Stock 1. Navigates to stock transfer, initiates new transfer. 2. Selects source location, destination, variant, quantity. 3. System validates sufficient stock at source. 4. Submits; system creates transfer record pending, decrements source. 5. Upon arrival, confirms receipt; system increments destination, transitions to completed, confirms. , Review Stock Movements 1. Navigates to stock movement audit interface. 2. System displays all stock movements in reverse chronological order with pagination. 3. Applies optional filters (date range, variant, location, movement type). 4. Selects movement to view full detail. , Monitor Low Stock 1. Navigates to low stock monitoring view. 2. System displays stock items below configured threshold with variant, location, on-hand, threshold, days since last restock. 3. Reviews list, identifies items needing replenishment. ,
Alternatives	1. A1. Bulk adjustment via file upload → system processes, validates, reports success/failure counts. 2. A2. Reduce below reserved quantity → system rejects, shows current reserved. 3. A3. Transfer exceeds available stock → system rejects, shows maximum. 4. A4. Cancel pending transfer → system returns deducted quantity to source, logs cancellation. 5. A5. No items below threshold (Low Stock) → system displays message that all stock levels are sufficient.
Exceptions	1. E1. Variant-location pair already exists → system rejects, suggests editing existing. 2. E2. Concurrent modification → system refreshes, asks to re-enter. 3. E3. Retrieval failure → system displays error, offers retry.

Requirements	INV-GRP-01, INV-GRP-03, INV-GRP-04
---------------------	------------------------------------

2.2.4.22 User Management

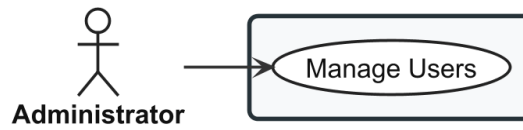


Figure 17: Use case diagram for User Management (UC-ADM-USR).

2.2.4.23 UC-ADM-USR: Manage Users

Table 21: Manage Users.

Use Case	UC-ADM-USR: Manage Users
Actor	Administrator
Goal	Create, update, enable, and disable user accounts.
Pre/Post	Pre: authenticated with user management permissions. Post: user account created or modified; status changes take effect on next authentication.
Scenario	1. Navigates to user management. 2. System displays user list with default sorting and pagination. 3. Applies optional filters (email, name, role, account status). 4. Creates new user account by entering email, name, assigning roles. 5. Alternatively selects existing user to view detail or edit. 6. Modifies profile fields, enables/disables account, or adjusts role assignments. 7. Saves; system validates email uniqueness, persists, confirms; if account was disabled, all active sessions are revoked.
Alternatives	1. A1. Disable account → system revokes all active sessions immediately. 2. A2. Re-enable disabled account → system restores active status. 3. A3. Email already registered → system rejects, prompts for different email.
Exceptions	1. E1. Persistence failure → system reports, retains form data for retry.
Requirements	IDN-GRP-04

2.2.4.24 Role and Permission Governance

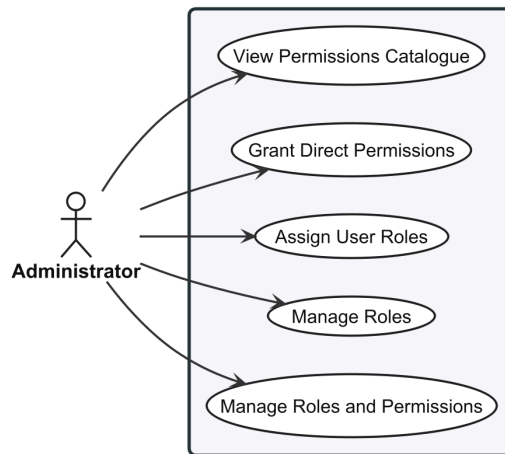


Figure 18: Use case diagram for Role and Permission Governance (UC-ADM-ROL).

2.2.4.25 UC-ADM-ROL: Manage Roles and Permissions

Table 22: Manage Roles and Permissions.

Use Case	UC-ADM-ROL: Manage Roles and Permissions
Actor	Administrator
Goal	Create and manage roles, assign permissions to roles, and grant roles to users.
Pre/Post	Pre: authenticated with role and permission management rights. Post: role and permission configuration updated; affected users receive updated permissions on next token.
Scenario	Manage Roles 1. Navigates to role management. 2. System displays list of roles with name, description, permission count. 3. Creates new role with name and description. 4. Assigns permissions from permissions catalogue. 5. Optionally edits existing role's name, description, or permission set. 6. Saves; system validates role name uniqueness, persists, confirms. Assign User Roles 1. Opens user's detail page. 2. Opens role assignment panel. 3. System displays all available roles with checkboxes for current assignments. 4. Selects roles to assign and deselects to revoke. 5. Saves; system persists, displays updated effective permissions. Grant Direct Permissions 1. Opens user's detail page. 2. Opens direct permission panel.

	3. System displays permissions catalogue with role-inherited and direct-grant indicators. 4. Selects permissions to grant directly and deselects to revoke. 5. Saves; system persists, recalculates effective permissions. , View Permissions Catalogue 1. Navigates to permissions catalogue. 2. System displays all permission claims grouped by domain and module. 3. Applies optional filters (domain, category, keyword search). 4. Reviews permission matrix to plan role designs or audit assignments. ,
Alternatives	1. A1. Remove role assigned to users → system warns affected users lose permissions. 2. A2. Revoke permission from role → system warns all users with this role lose it on next token. 3. A3. Revoke last role from user → system warns user loses all role-derived permissions. 4. A4. Grant already inherited from role → system accepts; effective permission unchanged but direct grant recorded.
Exceptions	1. E1. Concurrent modification or role deleted → system refreshes, asks to retry.
Requirements	IDN-GRP-03

2.2.4.26 Shipping Method Configuration

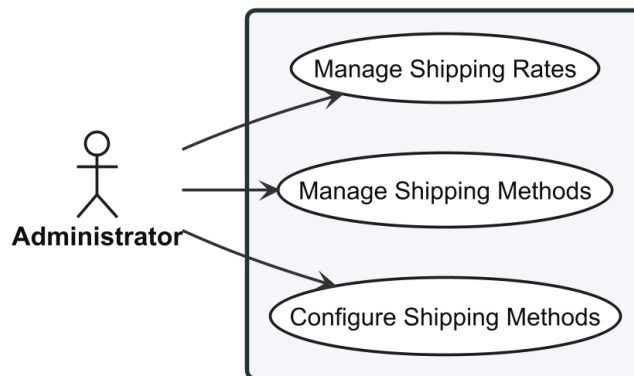


Figure 19: Use case diagram for Shipping Method Configuration (UC-ADM-SHP).

2.2.4.27 UC-ADM-SHP: Manage Shipping

Goal: Configure delivery methods and their associated shipping rates. **Trigger:** the administrator opens shipping method management. **Related requirements:** SHP-GRP-01. The flow creates, edits, activates, deactivates, or removes shipping methods and their rate tiers (with zone, weight, and cart-value ranges); alternatives warn about methods in active checkouts, methods with no zone, and overlapping rate tiers, and an exception handles concurrent modification.

2.2.4.28 Reference Data Management

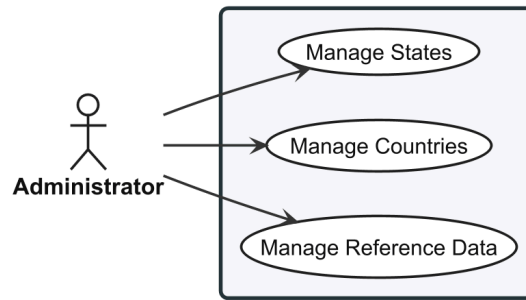


Figure 20: Use case diagram for Reference Data Management (UC-ADM-REF).

2.2.4.29 UC-ADM-REF: Manage Reference Data

Goal: Create and update country and state reference data. **Trigger:** the administrator opens country or state management. **Related requirements:** LOC-GRP-01. The flow creates and edits ISO-coded country and state records with active-status flags; alternatives warn about deactivating or deleting records referenced elsewhere (states, shipping zones, addresses), and an exception handles duplicate ISO codes.

2.2.5 Customer Use Cases

The Customer actor interacts through the browser-based storefront for product discovery, purchase, and account management. Each specification follows the same structure as the Administrator use cases.

2.2.5.1 Catalog Browsing

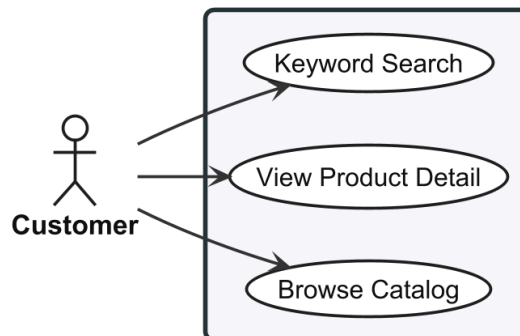


Figure 21: Use case diagram for Catalog Browsing (UC-STR-BRW).

2.2.5.2 UC-STR-BRW: Browse and Search Catalog

Table 23: Browse and Search Catalog.

Use Case	UC-STR-BRW: Browse and Search Catalog
----------	---------------------------------------

Actor	Customer
Goal	Browse the product catalog, view product details, and search by keyword.
Pre/Post	Pre: catalog data is published and searchable. Post: product listing or detail displayed with pricing and availability.
Scenario	Browse Catalog 1. Visits storefront home page or category landing page. 2. System displays taxonomy tree with top-level categories. 3. Selects category or subcategory from taxonomy navigation. 4. System retrieves products associated with selected taxon and its descendants. 5. System displays paginated grid of product cards showing thumbnail, name, price, availability. 6. Applies optional facets to filter by option types (e.g. size, colour) and other attributes. 7. System refreshes listing with filtered results. , View Product Detail 1. Clicks product card from catalog listing, search result, or recommendation. 2. System displays product detail page with primary image, name, description, price range. 3. System shows fashion metadata (style code, season, material, department, gender target). 4. System displays taxonomy breadcrumb showing category path. 5. Browses product images in gallery. 6. Selects variant option values from available options. 7. System updates displayed price, availability, variant-specific images based on selection. , Keyword Search 1. Enters search keyword or phrase in storefront search bar. 2. System queries catalog for products matching name, description, or fashion attributes. 3. System ranks results by relevance, displays paginated grid of matching product cards. 4. Refines search with additional keywords or applies filters. ,
Alternatives	1. A1. No products in category → system displays empty state suggesting other categories. 2. A2. Selected variant out of stock → system shows out-of-stock status, disables add-to-cart. 3. A3. No products match keyword → system displays suggestions to check spelling or browse categories. 4. A4. Very short query (single character) → system prompts to enter at least two characters.
Exceptions	1. E1. Retrieval or search failure → system displays error, offers retry.
Requirements	CAT-GRP-01, CAT-GRP-02, CAT-GRP-04, CAT-GRP-05

2.2.5.3 Search

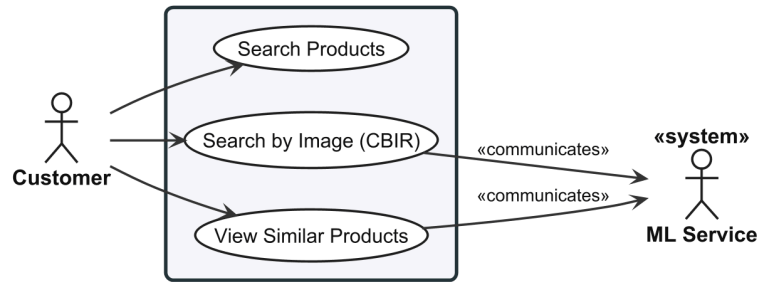


Figure 22: Use case diagram for Search (UC-STR-SRC).

2.2.5.4 UC-STR-SRC: Visual Search

Table 24: Visual Search.

Use Case	UC-STR-SRC: Visual Search
Actor	Customer
Support	ML Service
Goal	Search for products by uploading a reference image and discover visually similar items.
Pre/Post	Pre: catalog has products with embeddings; ML service is operational. Post: visually similar products displayed with similarity scores above configured threshold.
Scenario	Search by Image (CBIR) 1. Navigates to visual search interface. 2. Selects or drags image file from local file system. 3. System validates image format (JPEG, PNG, WebP) and size constraints. 4. System sends image to ML service for embedding generation. 5. System performs similarity search against image embeddings index. 6. System retrieves matching products ranked by similarity score above minimum threshold. 7. System displays results as grid of product thumbnails with scores, linking to detail pages. , View Similar Products 1. Opens product detail page with image embeddings available. 2. System retrieves product's primary image embedding. 3. System performs similarity search against other product embeddings. 4. System displays horizontal carousel or grid of visually similar product cards. 5. Clicks on similar product card to navigate to its detail page. ,
Alternatives	1. A1. Invalid image format → system rejects, lists accepted formats.

	2. A2. No products above threshold → system displays empty result suggesting different image or keyword search. 3. A3. No images or embeddings on detail page → system hides similar products section. 4. A4. ML service unavailable → system gracefully hides section without affecting detail page.
Exceptions	1. E1. ML service unavailable → system displays error, suggests retrying later. 2. E2. Embedding index empty → system reports visual search not yet available.
Requirements	CAT-GRP-04

2.2.5.5 Cart Management

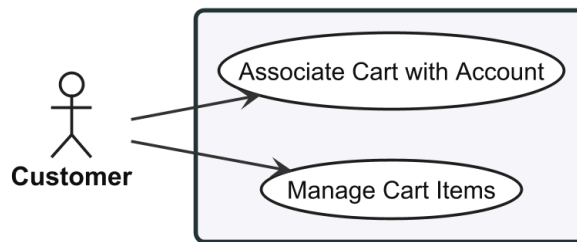


Figure 23: Use case diagram for Cart Management (UC-STR-CRT).

2.2.5.6 UC-STR-CRT: Manage Cart

Table 25: Manage Cart.

Use Case	UC-STR-CRT: Manage Cart
Actor	Customer
Goal	Add, update, and remove items in a shopping cart; associate guest cart with account.
Pre/Post	Pre: product variant exists and is available. Post: cart persisted across page navigation; guest carts survive browser sessions.
Scenario	Manage Cart Items 1. On product detail page, selects variant and specifies quantity. 2. Clicks Add to Cart; system validates variant and quantity. 3. System adds variant to customer's cart, displays confirmation. 4. Opens cart to view all items with quantities, prices, subtotal. 5. Updates quantity of item or removes item. 6. System recalculates cart subtotal, updates display. , Associate Cart with Account 1. Browses storefront as guest, adds items to cart. 2. Logs in or registers for account. 3. System detects guest cart exists, retrieves any existing user cart. 4. System merges guest and user carts: matching variants increase quantity, unique variants are added.

	5. System associates merged cart with user account, invalidates guest cart cookie.
Alternatives	1. A1. Quantity exceeds stock → system rejects, shows max available. 2. A2. Same variant already in cart → system increments existing quantity. 3. A3. Guest customer → system assigns session-based cart identifier in signed cookie. 4. A4. No existing user cart → system transfers guest cart to user account directly. 5. A5. Merge exceeds available stock → system caps at max available, notifies customer.
Exceptions	1. E1. Variant deactivated or archived → system rejects, suggests refreshing product page. 2. E2. Merge fails due to data conflict → system creates user cart with guest items, notifies to review.
Requirements	ORD-GRP-01

2.2.5.7 Checkout Flow

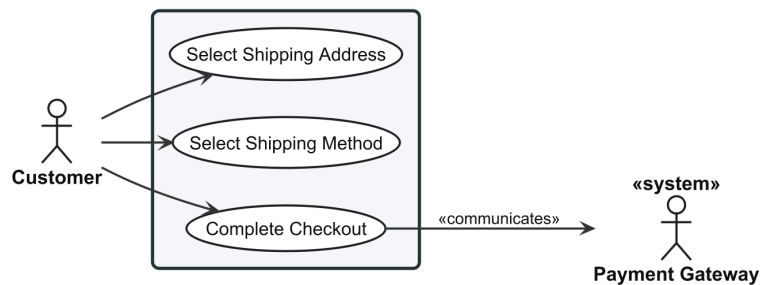


Figure 24: Use case diagram for Checkout Flow (UC-STR-CHK).

2.2.5.8 UC-STR-CHK: Checkout

Table 26: Checkout.

Use Case	UC-STR-CHK: Checkout
Actor	Customer
Support	Payment Gateway
Goal	Complete the multi-step checkout: address selection, shipping method, and order confirmation.
Pre/Post	Pre: customer is authenticated; cart is not empty; stock is available for all items. Post: order created with unique number; inventory reserved; payment linked; cart cleared.
Scenario	Select Shipping Address 1. From cart, clicks Proceed to Checkout. 2. System transitions checkout to Address step.

	3. System displays saved addresses with default pre-selected. 4. Selects existing shipping address or enters new one. 5. System determines shipping zone, proceeds to next step. Select Shipping Method 1. System calculates available shipping methods and rates based on address zone, cart weight, cart value. 2. System displays list of available methods with rates and estimated delivery times. 3. Reviews options, selects preferred shipping method. 4. System applies selected method and rate, updates shipment total in order summary. 5. System presents next checkout step (payment). Complete Checkout 1. System displays order summary with line items, shipping, tax, total. 2. Enters payment details or selects saved payment method. 3. System creates payment intent, reserves inventory for each line item. 4. Reviews final order summary, confirms purchase. 5. System captures payment, generates unique order number. 6. System transitions order to Confirmed state, clears cart, displays order confirmation. 7. System sends order confirmation notification.
Alternatives	1. A1. No saved addresses → system presents empty address step with prompt to create new. 2. A2. No methods available for zone → system displays message, prompts different address. 3. A3. Only one method available → system auto-selects, proceeds. 4. A4. Stock depleted during checkout → system notifies, removes affected items, returns to cart. 5. A5. Payment failure → system notifies with reason, allows retry; inventory not reserved.
Exceptions	1. E1. Address validation fails → system highlights missing fields, prevents progression. 2. E2. Rate calculation fails → system displays error, suggests contacting support. 3. E3. Payment captured but inventory reservation fails → system voids payment, notifies order not completed.
Requirements	ORD-GRP-02, ORD-GRP-03

2.2.5.9 Order History

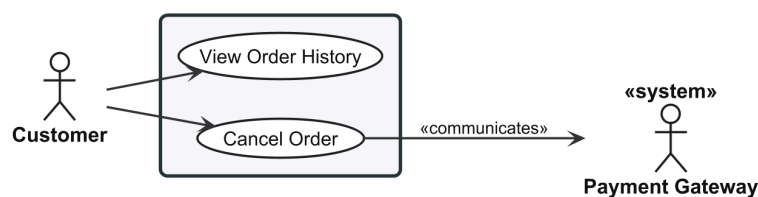


Figure 25: Use case diagram for Order History (UC-STR-OHI).

2.2.5.10 UC-STR-OHI: Order History

Table 27: Order History.

Use Case	UC-STR-OHI: Order History
Actor	Customer
Support	Payment Gateway
Goal	View past orders and cancel pending orders.
Pre/Post	Pre: customer is authenticated. Post: complete order history visible; cancelled orders release inventory and void payment.
Scenario	View Order History 1. Navigates to order history from account menu. 2. System displays past orders in reverse chronological order with pagination, showing order number, date, status, and total. 3. Applies optional date range or status filters. 4. Selects an order to view full detail: line items, shipping address, method, payment state, shipment state, and status timeline. Cancel Order 1. Opens order detail from order history. 2. System displays order detail with current status and cancel action if cancellable. 3. Selects Cancel Order. 4. System displays confirmation explaining inventory release and payment void. 5. Confirms; system releases reserved inventory, voids payment, transitions to cancelled, and sends confirmation.
Alternatives	1. A1. No orders → system displays message with prompt to browse catalog. 2. A2. Order state changed since page loaded → system refreshes and informs cancellation unavailable.
Exceptions	1. E1. Payment gateway unreachable (Cancel) → system cancels order, releases inventory, queues void, notifies customer. 2. E2. Retrieval failure (View) → system displays error and offers retry.
Requirements	ORD-GRP-04

2.2.5.11 Payment Processing

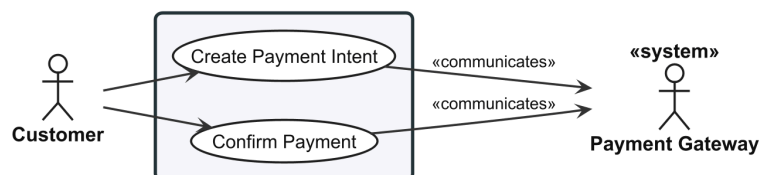


Figure 26: Use case diagram for Payment Processing (UC-STR-PAY).

2.2.5.12 UC-STR-PAY: Payment Processing

Goal: Create and confirm payment for an order. **Trigger:** the customer reaches the payment step of checkout with a calculated order total. **Related requirements:** PAY-GRP-01. The flow creates a payment intent through the gateway, reviews the final order summary, and confirms; alternatives cover invalid or declined payment details and additional authentication, while exceptions handle an unreachable gateway, order-creation failure after payment, and gateway timeouts.

2.2.5.13 Authentication

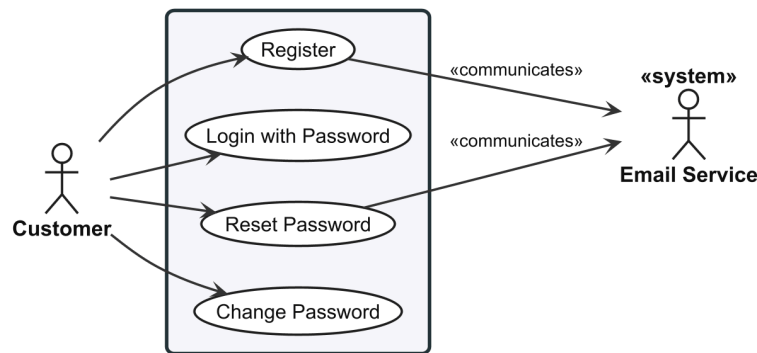


Figure 27: Use case diagram for Authentication (UC-STR-AUT).

2.2.5.14 UC-STR-AUT: Authentication

Table 28: Authentication.

Use Case	UC-STR-AUT: Authentication
Actor	Customer
Support	Email Service
Goal	Register, log in, and manage account credentials.
Pre/Post	Pre: none (public access). Post: customer authenticated or credentials updated.
Scenario	Register 1. Navigates to registration page. 2. Enters email, password, and basic profile information. 3. Submits; system validates email not registered and password strength, creates account, sends verification; confirms. Login with Password 1. Navigates to login page and enters email and password. 2. Submits; system validates credentials, issues access and refresh tokens, associates guest cart if exists; redirects to home page.

	, Reset Password 1. Clicks Forgot Password and submits registered email. 2. System generates time-limited reset token and sends via email. 3. Opens email, clicks reset link, enters new password. 4. Submits; system validates token, updates password, revokes all sessions; confirms. , Change Password 1. Navigates to account security settings and selects Change Password. 2. Enters current password and new password. 3. Submits; system verifies current password, validates new password, updates, revokes all sessions except current; confirms. ,
Alternatives	1. A1. Email already registered → system rejects and suggests login with password reset link. 2. A2. Password does not meet strength requirements → system highlights requirements and prompts retry. 3. A3. Invalid credentials → system rejects with generic message. 4. A4. Account disabled → system rejects with message to contact support. 5. A5. Reset token expired → system displays message and prompts new reset request. 6. A6. Current password incorrect (Change) → system rejects and prompts retry.
Exceptions	1. E1. Verification or reset email fails to send → system creates account/request but logs failure internally. 2. E2. Token issuance fails → system reports failure and suggests retry.
Requirements	IDN-GRP-01, IDN-GRP-02

2.2.5.15 Session Management

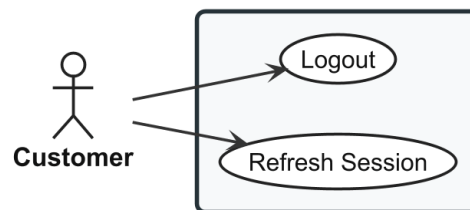


Figure 28: Use case diagram for Session Management (UC-STR-SES).

2.2.5.16 UC-STR-SES: Session Management

Table 29: Session Management.

Use Case	UC-STR-SES: Session Management
Actor	Customer

Goal	Maintain and terminate authenticated sessions.
Pre/Post	Pre: customer has an active session. Post: session extended or terminated.
Scenario	Refresh Session 1. Client detects access token will expire soon. 2. Client sends current refresh token to token endpoint. 3. System validates refresh token (not expired, not consumed). 4. System issues new access token with fresh expiry. 5. System issues new refresh token and invalidates previous (token rotation). 6. Client stores new token pair and continues session. Logout 1. Clicks Logout in the storefront navigation. 2. System sends current refresh token for invalidation. 3. System invalidates the refresh token and removes the session cookie. 4. System redirects to the storefront home page.
Alternatives	1. A1. Refresh token expired → system rejects; client redirects to login. 2. A2. Refresh token consumed (reuse detected) → system revokes all tokens; client redirects to login with security notification. 3. A3. Logout request fails due to network → system clears local tokens and cookies; session expires naturally.
Exceptions	1. E1. Token issuance or invalidation fails → client retains existing pair if access token still valid; clears locally otherwise.
Requirements	IDN-GRP-01

2.2.5.17 Profile and Preferences

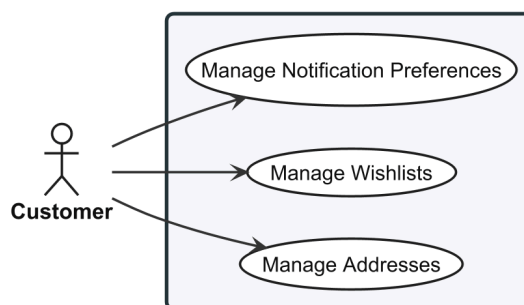


Figure 29: Use case diagram for Profile and Preferences (UC-STR-PRF).

2.2.5.18 UC-STR-PRF: Profile Management

Goal: Manage shipping addresses, wishlists, and notification preferences. **Trigger:** the authenticated customer opens their profile. **Related requirements:** PRF-GRP-01, PRF-GRP-02. The flow manages an address book, named wishlists with variant entries, and per-channel notification opt-in/out; alternatives

cover duplicate wishlist items and addresses referenced by orders, and exceptions handle address validation, archived variants, and persistence failure.

2.2.6 System Use Cases

The System actor represents automated background processes that maintain data consistency, generate embeddings, and perform scheduled operations. Coordination relies on **Hangfire** [24] for job scheduling and **Redis** [22] for persistence.

2.2.6.1 Embedding Operations

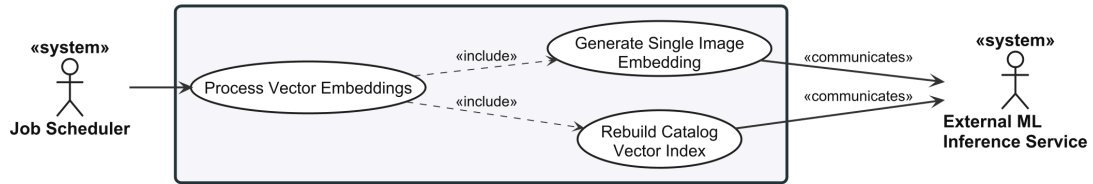


Figure 30: Use case diagram for Embedding Operations (UC-SYS-EMB).

2.2.6.2 UC-SYS-EMB: Embedding Operations

Table 30: Embedding Operations.

Use Case	UC-SYS-EMB: Embedding Operations
Actor	System
Support	ML Service
Goal	Generate and regenerate image embeddings for product search.
Pre/Post	Pre: images uploaded and stored; ML service is operational. Post: embeddings stored in the index for visual search.
Scenario	Generate Image Embeddings 1. System detects a new unprocessed image in the queue. 2. System retrieves the image file from storage. 3. System preprocesses the image (resize, normalise) for model requirements. 4. System sends preprocessed image to ML service with configured model identifier. 5. ML Service computes the visual embedding vector and returns it with metadata. 6. System stores embedding in the index with image reference and metadata. 7. System marks the image as processed. , Regenerate All Embeddings 1. System detects change to embedding model configuration. 2. System validates new model is available via ML service health check. 3. System retrieves list of all product images requiring regeneration. 4. System sends each image to ML service using the new model.

	5. ML Service computes new embedding and returns it. 6. System stores new embedding replacing previous and updates model metadata. 7. System reports completion with success/failure count summary.
Alternatives	1. A1. Image not accessible (deleted/corrupted) → system marks as failed and records reason. 2. A2. Multiple images queued → system processes sequentially, throttling to ML service capacity. 3. A3. Triggered during peak traffic (Regenerate) → system throttles to avoid impacting search performance. 4. A4. No model change detected → system logs event and skips processing.
Exceptions	1. E1. ML service returns error → system retries with exponential backoff; after max retries marks as failed. 2. E2. ML service unreachable → system requeues image and triggers alert. 3. E3. New model not available (Regenerate) → system aborts regeneration; existing embeddings remain in use.
Requirements	CAT-GRP-03

The embedding generation process described in the main success scenario follows the pipeline illustrated in Figure 31: an image is authenticated, pre-processed through standardised transforms, passed through the selected model, and the resulting feature vector is serialised alongside performance metadata.

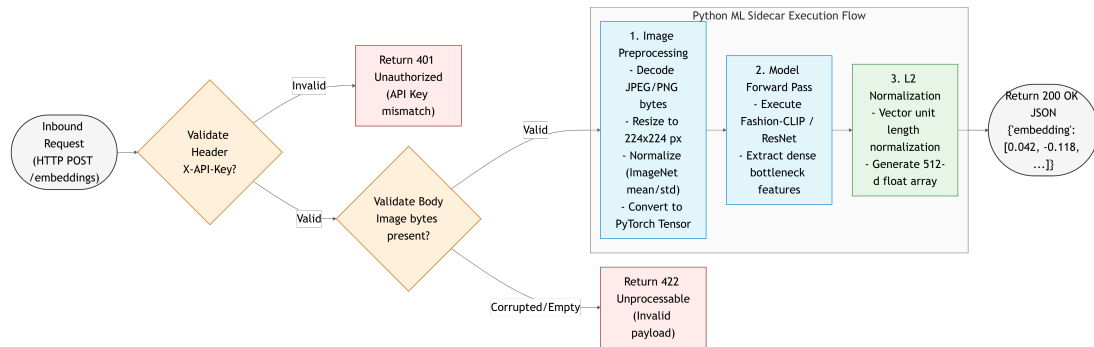


Figure 31: ML embedding pipeline: image upload to normalised vector response.

2.2.6.3 Background Maintenance

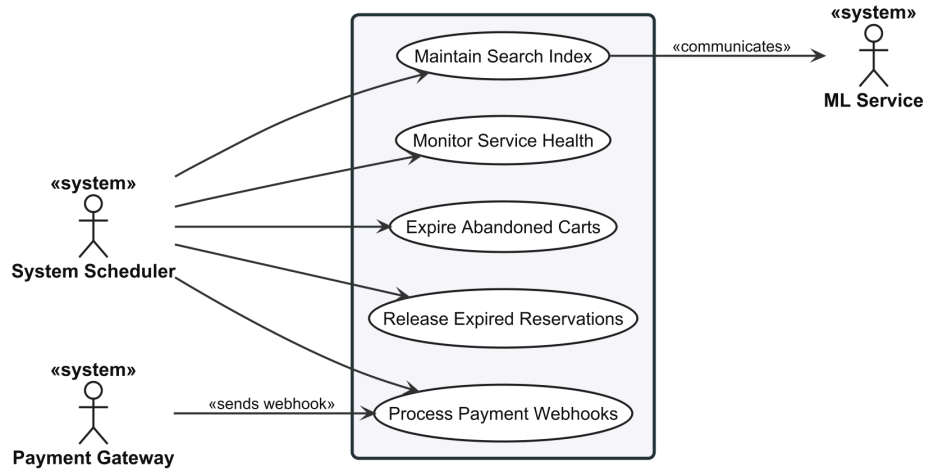


Figure 32: Use case diagram for Background Maintenance (UC-SYS-MNT).

2.2.6.4 UC-SYS-MNT: System Maintenance

Goal: Perform scheduled and event-driven system maintenance tasks. **Trigger:** a scheduled job fires or the payment gateway sends a signed webhook. **Related requirements:** CAT-GRP-04, ORD-GRP-01, INV-GRP-02, PAY-GRP-03. The flow monitors ML service health, expires abandoned carts, releases expired inventory reservations, processes payment webhooks with signature and idempotency validation, and maintains the search index; alternatives and exceptions cover empty batches, duplicate or out-of-order webhooks, and database or index failures.

2.3 SYSTEM ARCHITECTURE & DESIGN

The platform architecture covers six parts: how the services are organized, domain modelling, and the database, API, and security layers. The design follows a service-oriented approach: a Vue 3 frontend, a .NET 10 modular monolith backend, and a Python FastAPI machine learning sidecar, each independently deployable.

- **System Overview.** Three services, eight bounded contexts, technology stack summary.
- **Domain-Driven Design.** Context map, aggregate roots with invariants, state machines.
- **C4 Architecture.** Context, container, and component-level structural views.
- **Database Design.** Per-context schemas, pgvector integration, core design decisions.
- **API Design.** Carter minimal APIs, MediatR CQRS, endpoint conventions.
- **Security Design.** JWT authentication, permission-based authorisation, defensive hardening.

2.3.1 System Overview

ReSys.Shop has three services – a Vue 3 frontend, a .NET 10 backend [19], and a Python **FastAPI** ML sidecar [23] – and eight **bounded contexts** using **Domain-Driven Design** with MediatR dispatch between modules.

Table 31: System services with technology stacks and HTTP contracts.

Service	Technology Stack	Responsibilities
Vue Frontend	Vue 3 + TypeScript + Vite	• Customer storefront and administrator dashboard • Image upload and visual search UI
.NET Backend	.NET 10 + ASP.NET Core + Carter + EF Core	• REST API via Carter minimal APIs and MediatR CQRS • PostgreSQL persistence with pgvector and Redis caching
Python ML	Python 3.12 + FastAPI + PyTorch	• Embedding model inference (Fashion-CLIP and others) • Multi-model support with lazy-loading strategy

Internally, the backend is partitioned into eight bounded contexts, each owning a dedicated database schema. Table 32 lists each context, its aggregate root, and key domain entities.

Table 32: Bounded contexts with aggregate roots, key entities, and MediatR dispatch.

Bounded Context	Aggregate Root	Key Domain Entities
Catalog	Product	Variant, VariantImage, OptionType, Option-Value, Classification, Taxonomy, Taxon
Ordering	Order	LineItem, Adjustment, Shipment
Payment	PaymentIntent	PaymentCapture
Inventory	StockItem	StockLocation, StockMovement, Stock-Reservation
Identity	User	Role, UserRole, RefreshToken, UserLogin
Profile	UserProfile	Address, Wishlist
Shipping	ShippingMethod	ShippingRate, ShippingZone
Location	Country	State

2.3.2 Domain-Driven Design

The backend follows **Domain-Driven Design** (DDD) with eight bounded contexts, each managing explicit aggregate boundaries and domain invariants under a Conformist integration pattern via Published Language contracts.

2.3.2.1 Bounded Context Map

The platform is partitioned into eight **bounded contexts** along business capability boundaries. Each context independently owns its state model, business logic, and vocabulary. Integration follows a **Conformist** pattern with core abstractions from the Shared Kernel (*Result<T>*, *ICommand*, *IQuery*). Context-to-context communication uses only **MediatR ISender** in-process dispatch without direct compile-time project references.

Figure 33 illustrates the eight contexts and their inter-module communication pathways. Table 33 details each context's business capabilities and Published Language boundaries.

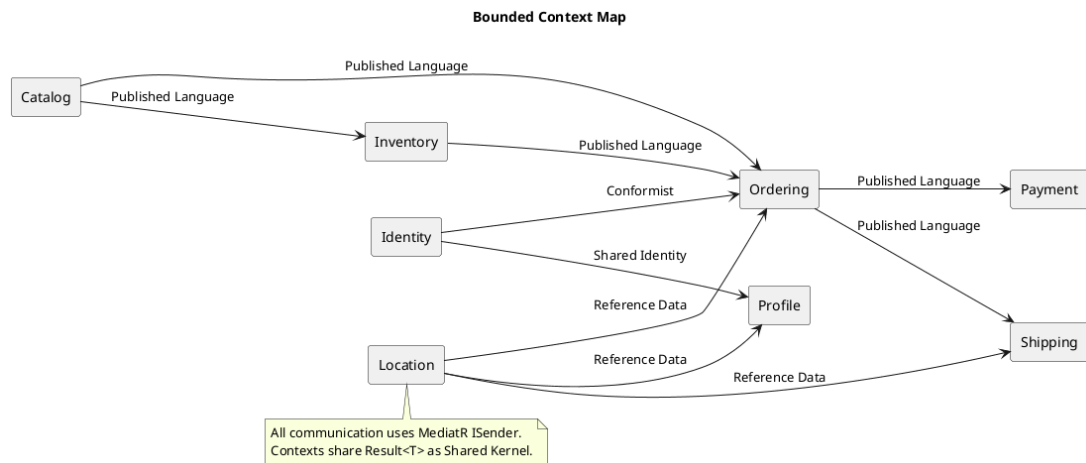


Figure 33: Bounded Context Map: eight contexts with Published Language identifiers.

Table 33: Bounded context responsibilities and Published Language identifiers.

Context	Business Responsibility	Published Language
Catalog	- Product lifecycle, variants with SKU and independent pricing - Image management with automated vector embedding generation - Hierarchical taxonomy classification	<i>ProductId, VariantId, Sku, Price, Slug</i>
Ordering	- Checkout workflow with forward-only state machine - Line item capture with locked price snapshots	<i>OrderId, OrderNumber, Total, Currency, CheckoutState</i>

Context	Business Responsibility	Published Language
Payment	- Payment intent lifecycle (authorize, capture, refund, void) - Local state mirroring for offline operation	<i>PaymentIntentId, PaymentState, Amount</i>
Inventory	- Stock tracking with reservations to prevent overselling - Append-only audit logging for stock adjustments	<i>StockItemId, QuantityOnHand, QuantityReserved</i>
Identity	- JWT authentication with refresh token rotation - RBAC with domain.category.resource.action claims	<i>UserId, Email, PermissionClaim</i>
Profile	Customer address book and wish-lists	<i>ProfileId, AddressId</i>
Shipping	Delivery method definitions, rate calculation, and zone configuration	<i>ShippingMethodId, Rate</i>
Location	ISO 3166 country and state reference data	<i>CountryId, StateId, IsoCode</i>

2.3.2.2 Aggregates and Invariants

An aggregate root defines a transactional consistency boundary, managing child entities and enforcing core business rules using a pragmatic DDD strategy without base class overhead or forced domain event dispatches.

- **Product (Catalog Root):** Manages product families, variants, media, options, and taxonomy mappings. Enforces slug uniqueness and a designated master variant. Variant images store 512-dimensional vector embeddings for visual search.
- **Order (Ordering Root):** Governs the purchasing pipeline with locked price snapshots at checkout initialization. Enforces the structural balance invariant

$$\text{Total} = \text{ItemTotal} + \text{AdjustmentTotal} + \text{ShipmentTotal}$$

. Completed orders are immutable except through formal administrative cancellations.

- **PaymentIntent (Payment Root):** Models transactional authorization states (*Pending, RequiresAction, Processing, Succeeded, Canceled, Failed*). Enforces cumulative capture debit limits and mirrors gateway states locally.
- **StockItem (Inventory Root):** Tracks on-hand and reserved stock per warehouse location. Enforces non-negative balance constraints

$$\text{QuantityOnHand} \geq 0$$

and writes all adjustments to an append-only ledger.

2.3.2.3 Ubiquitous Language Glossary

The ubiquitous language establishes a common domain vocabulary across software design and business rules. Table 34 defines key domain terms.

Table 34: Ubiquitous Language Glossary across domain contexts.

Term	Context	Definition
Product	Catalog	Core entity with shared metadata (description, slug, taxonomy). Prices and SKUs belong exclusively to Variants.
Variant	Catalog	Purchasable unit with SKU, barcode, price, dimensions, and stock flags.
Master Variant	Catalog	Primary variant for product listings. One required per product.
Option Type	Catalog	Reusable attribute categories (e.g., Color, Size) shared across products.
Taxonomy / Taxon	Catalog	Hierarchical classification with nested category nodes.
Cart	Ordering	Temporary purchase container. Purged after 7 days of inactivity.
Line Item	Ordering	Entry linking a variant to a quantity and locked price snapshot.
Checkout State	Ordering	Sequential stages (Address → Delivery → Payment → Confirm → Complete) with forward-only progression.
Payment Intent	Payment	Tracks authorization states across payment gateway interactions.
Payment Capture	Payment	Monetary debit executed against an authorized payment intent.
Stock Item	Inventory	Warehouse record tracking on-hand and reserved stock per location.
Stock Movement	Inventory	Immutable audit entry recording quantity deltas, operator, and reason.
Refresh Token	Identity	Single-use token for JWT access token generation. Reuse triggers session revocation.

2.3.2.4 State Machines

Explicit state machines inside domain entities govern checkout and payment workflows, validating transitions prior to database persistence.

2.3.2.4.1 Order Checkout State Machine

Checkout advances through five forward-only stages: Address, Delivery, Payment, Confirm, and Complete (Figure 34). Users can cancel at any pre-completion stage.

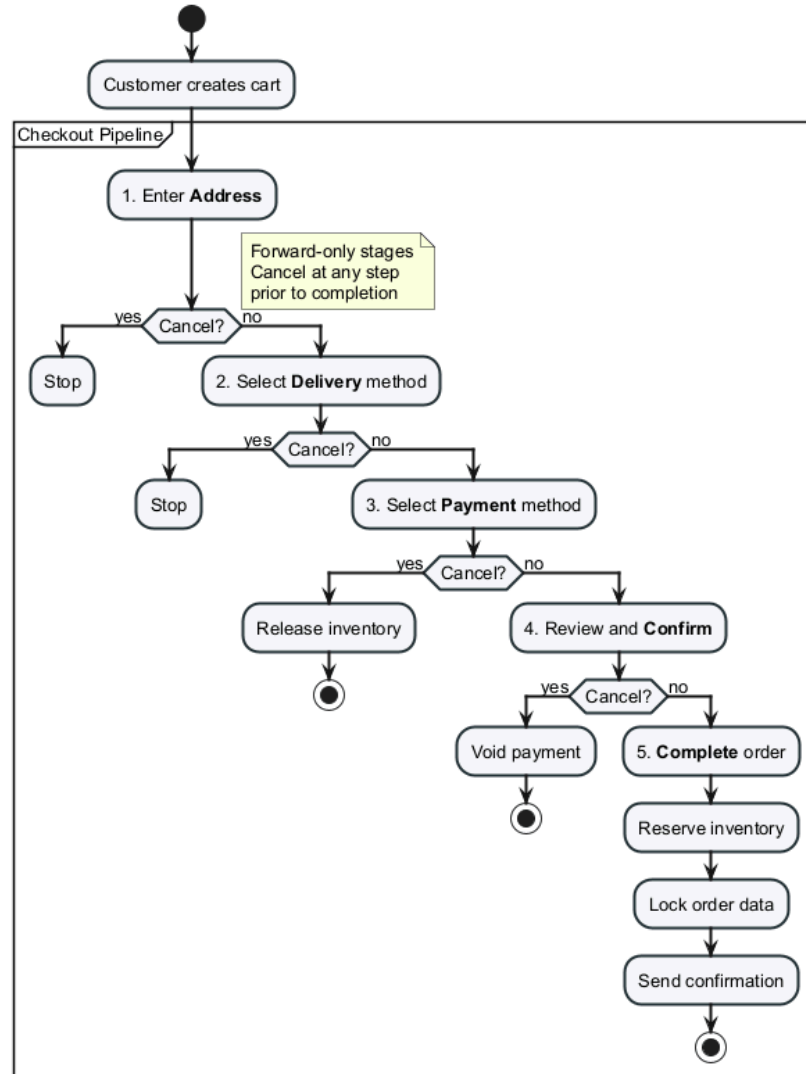


Figure 34: Order checkout state machine showing forward-only stages and cancellation paths.

2.3.2.4.2 Payment Intent State Machine

Payment intents follow a lifecycle from *Pending* through *Processing* to *Succeeded* or *Failed* (Figure 35). Authorized intents support capture and refund operations.

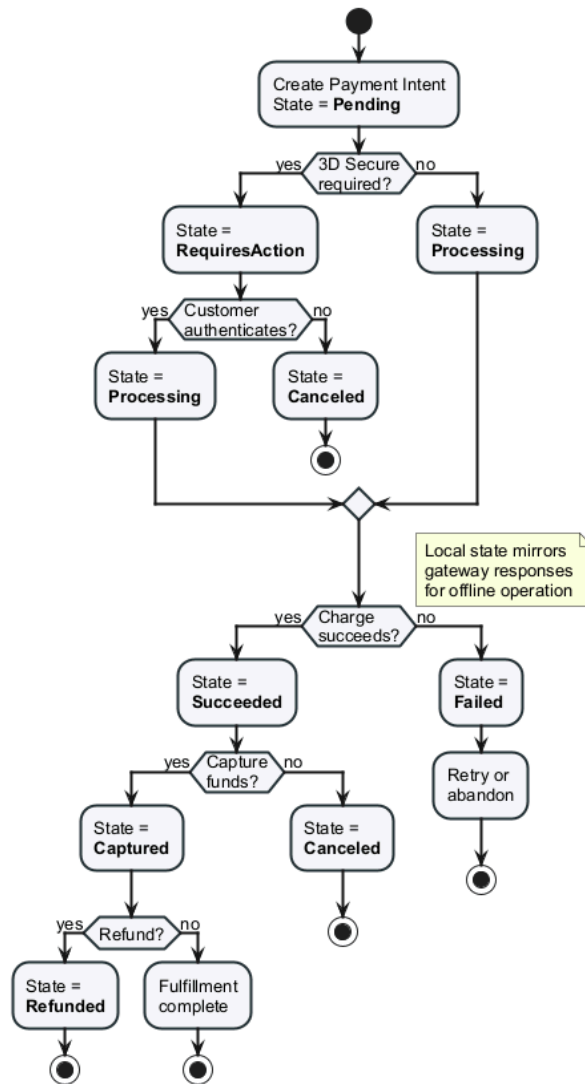


Figure 35: Payment intent lifecycle and terminal states.

2.3.3 C4 Architecture

The C4 model structures software architecture across four abstraction levels: system context, container, component, and code. This section presents the first three C4 levels, omitting code-level structures addressed in later implementation sections.

2.3.3.1 System Context

The system context shows how ReSys.Shop fits into its operating environment, including user roles and external dependencies (Figure 36).

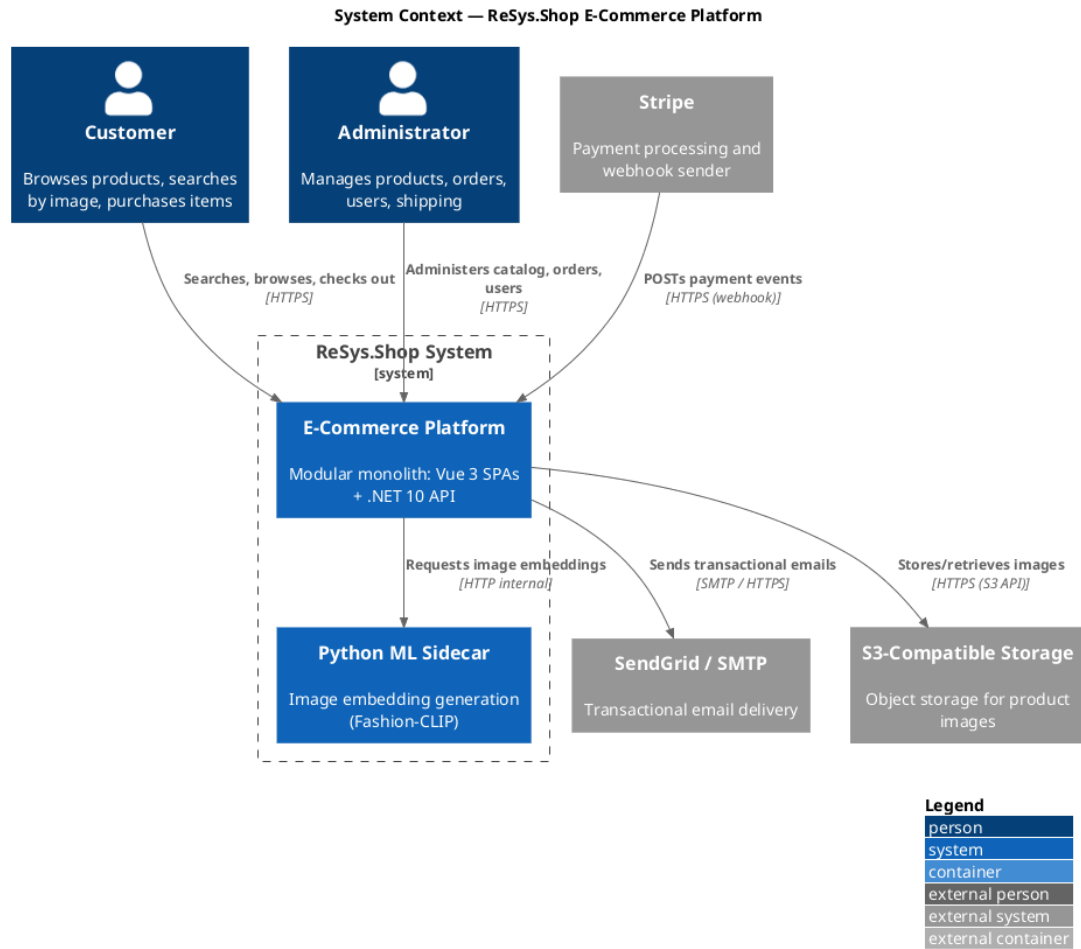


Figure 36: System Context diagram: ReSys.Shop boundary, user roles, external integrations.

The platform interacts with two human user groups:

- **Customers:** Browse the catalog, run visual and keyword searches, manage carts, and complete checkouts.
- **Administrators:** Manage products, process orders, track inventory, and administer user accounts.

Four external integrations:

- **Stripe:** Payment intent lifecycles and webhook notifications via signature verification.
- **SendGrid:** Transactional emails (order confirmations, password resets, shipping updates).
- **S3-Compatible Storage:** Product asset persistence.
- **Python ML Sidecar:** Image embedding generation within the container orchestration boundary.

2.3.3.2 Container

The container view decomposes ReSys.Shop into six standalone deployable processes and data stores (Figure 37).

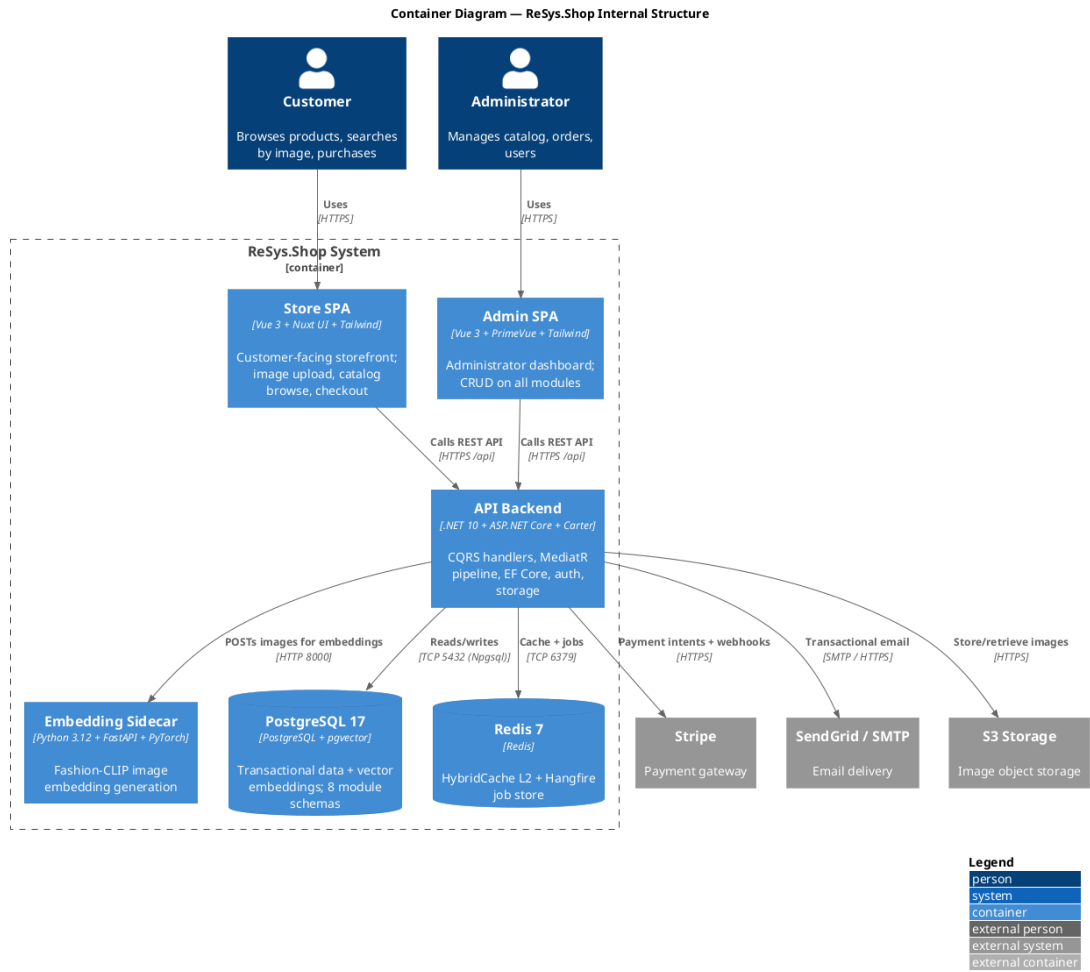


Figure 37: Containers: Vue 3 SPAs, .NET 10 API, ML sidecar, PostgreSQL/pgvector, Redis.

The deployable units:

- **Store & Admin SPAs:** Vue 3 single-page applications interacting with the backend over HTTPS REST endpoints.
- **API Backend:** .NET 10 application executing domain logic via Carter minimal APIs and MediatR CQRS pipelines.
- **Embedding Sidecar:** Python 3.12 FastAPI service loading ML models into GPU/CPU memory for on-demand embedding generation.
- **PostgreSQL 17 (with pgvector):** Relational domain schemas and high-dimensional vector embeddings.
- **Redis 7:** L2 distributed cache for *HybridCache* and persistent job store for Hangfire.

All communication routes through the API Backend; SPAs never query data-bases or external APIs directly.

2.3.3.3 Component

The component view details the internal structure of the API Backend container (Figure 38).

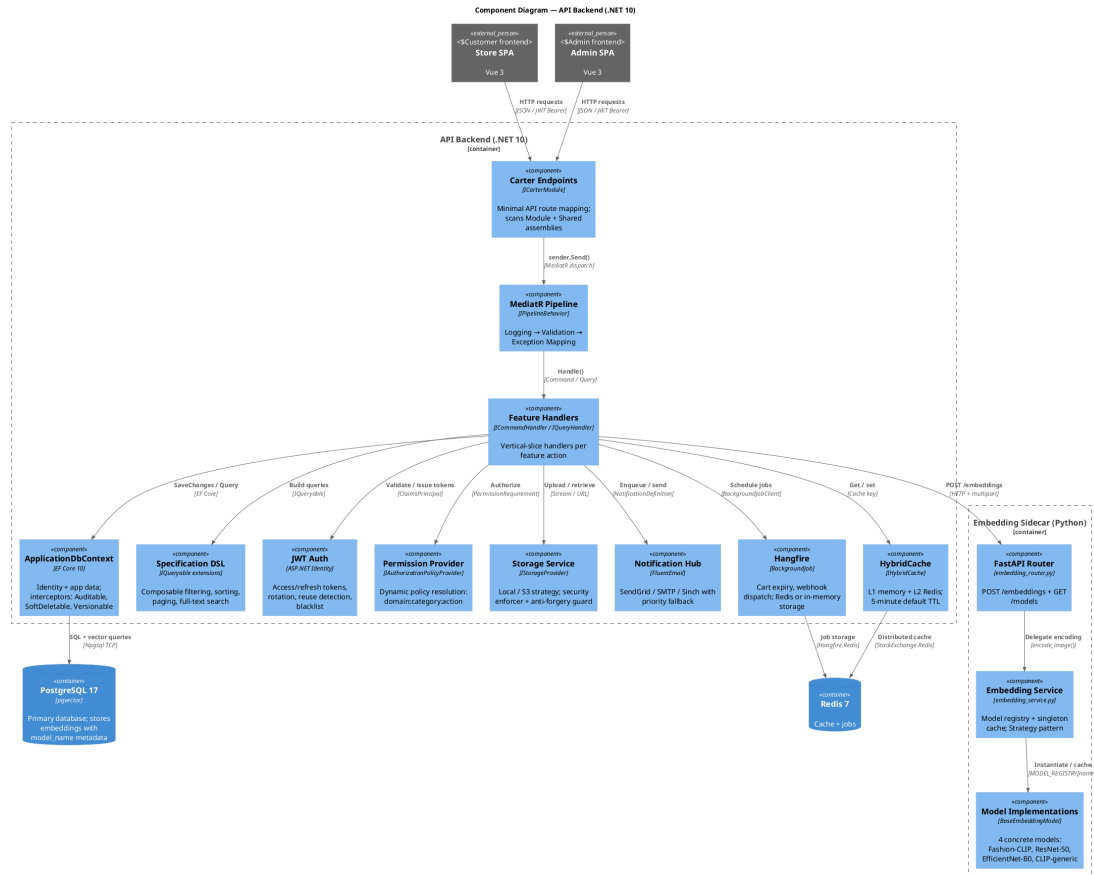


Figure 38: Component diagram: API Backend internals and three-layer Python ML sidecar.

HTTP requests enter through Carter modules, which validate parameters and dispatch commands or queries via MediatR's *ISender* through logging, validation, and exception-handling pipeline behaviors.

Handlers delegate infrastructure tasks to eight internal components:

1. **ApplicationDbContext:** EF Core context with interceptors for auditing, soft-deletes, and concurrency.
2. **Specification DSL:** Composable *IQueryable* extensions for filtering, sorting, paging, and full-text search.
3. **Identity Provider:** ASP.NET Identity with JWT management.
4. **Dynamic Permission Provider:** Runtime resolution of $\{domain\}:\{category\}:\{action\}$ policy claims.

5. **Storage Service:** Interchangeable local or S3-compatible file storage with upload validation.
6. **Notification Hub:** Email (SendGrid/SMTP) and SMS (Sinch) routing with fallback support.
7. **Hangfire Engine:** Background task execution (cart expiration, webhook dispatch, maintenance).
8. **HybridCache:** L1 in-memory and L2 Redis caching combined for cross-instance consistency.

The Python ML Sidecar uses a three-layer layout: a FastAPI router, a singleton model registry for lazy loading, and an interchangeable strategy interface supporting Fashion-CLIP, ResNet-50, EfficientNet-B0, and standard CLIP.

2.3.3.4 Deployment

The deployment diagram illustrates the production infrastructure topology (Figure 39).

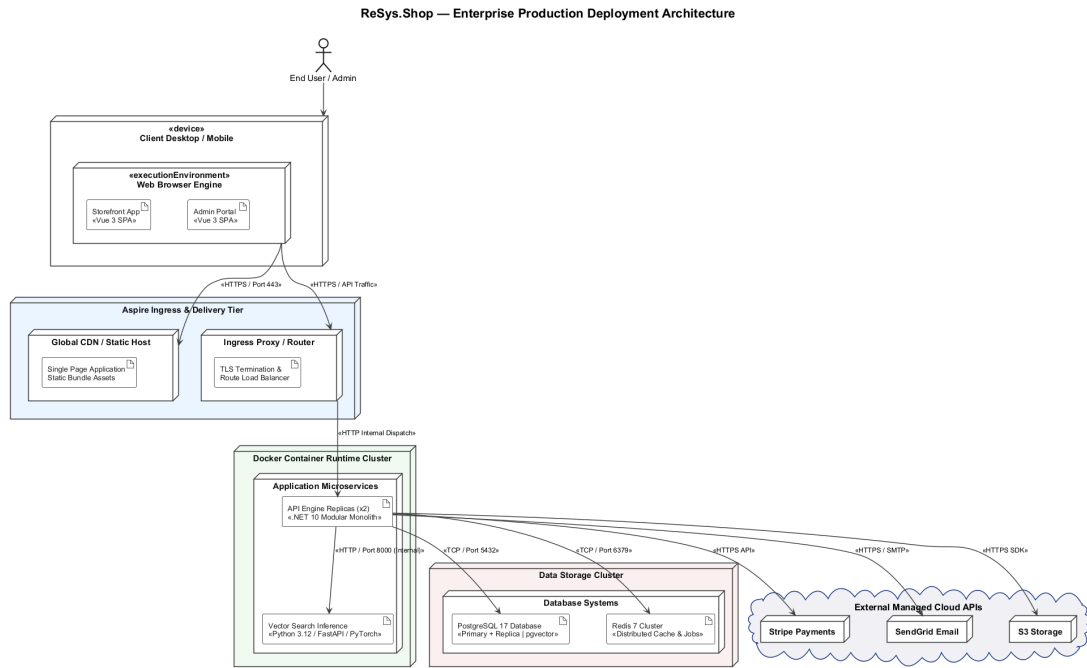


Figure 39: Deployment topology showing containerized orchestration with horizontal scaling.

All services run as Docker containers orchestrated via Docker Compose for service discovery and health monitoring. The API Backend scales horizontally across replicas sharing PostgreSQL and Redis. The stateless ML sidecar scales independently. PostgreSQL runs a primary instance for writes alongside read replicas, with *pgvector* enabled across all nodes. Secrets and API keys are injected via environment configuration.

2.3.4 Database Design

The ReSys.Shop database is a single PostgreSQL 17 instance partitioned into per-context schemas, each owned by one bounded context and managed through Entity Framework Core migrations. Every table uses a UUID primary key and a soft-deletion flag with a global query filter, while contention-sensitive aggregates carry an optimistic concurrency version. This section presents the schema organisation, the core entity-relationship model, and a per-module view of how each schema realises its domain aggregates, followed by the pgvector integration and the cross-cutting design rules.

2.3.4.1 Schema Organisation

Each bounded context owns a dedicated schema that mirrors its aggregate boundaries rather than a generic normalized model. The Catalog schema holds the product aggregate and its variant, media, option, and taxonomy members. The Identity schema extends ASP.NET Identity with users, roles, claims, refresh tokens, and passkeys. The Ordering schema contains the order aggregate with line items and adjustments. The Payment schema tracks payment captures, configured gateways, and inbound webhook events. The Inventory schema records stock locations, items, an append-only movement ledger, reservations, and inter-location transfers. The Shipping schema defines methods, rate tiers, geographic zones, and shipments. The Location schema provides country and state reference data. The Profile schema stores customer profiles, addresses, and wishlists.

Cross-context relationships use identifier references (loose UUID attributes rather than database-level foreign keys), so an order references user, variant, and address identifiers without coupling the schemas. This keeps modules logically independent and avoids coordinating transactions across contexts.

2.3.4.2 Core Entity-Relationship Model

Figure 40 shows the entity-relationship model across all eight bounded contexts. Dotted lines denote cross-context identifier references, while solid lines denote relationships contained within a single schema.

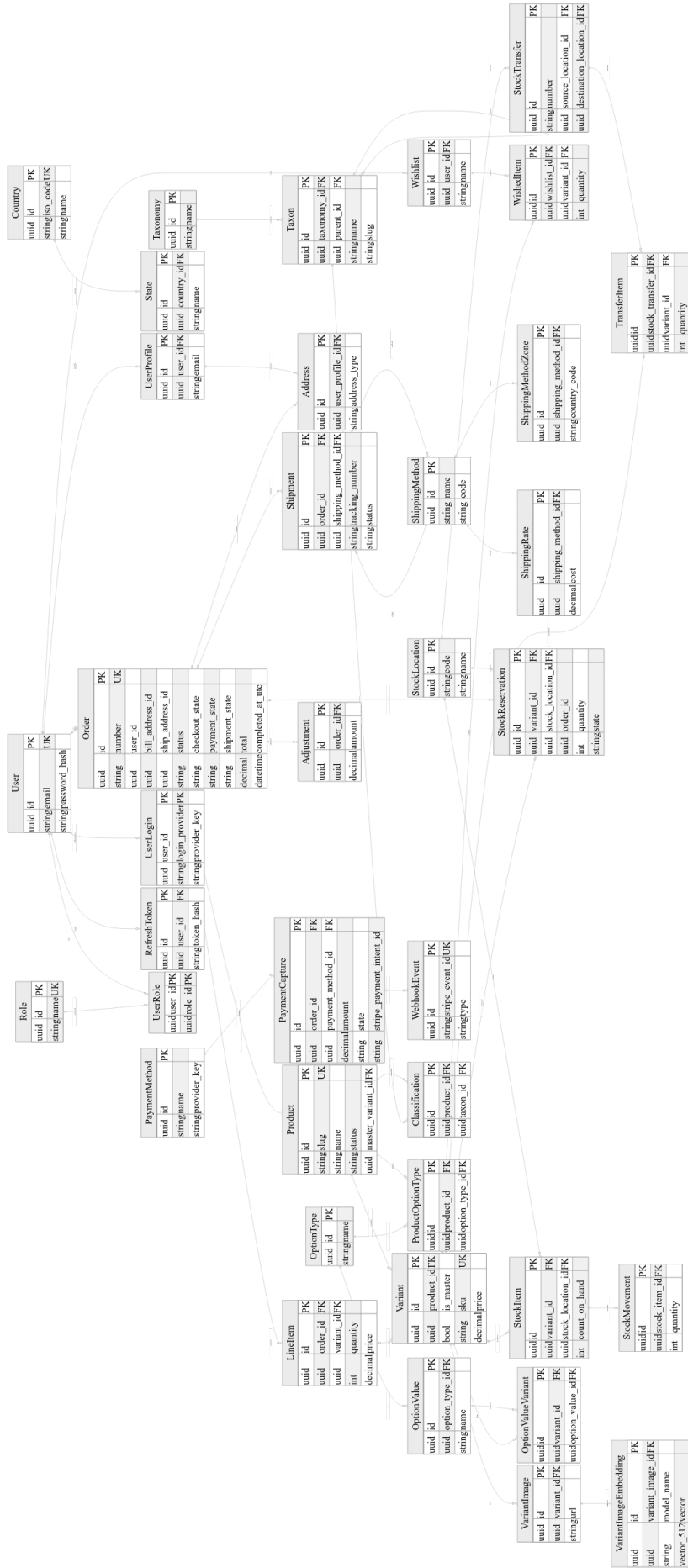


Figure 40: Core entity-relationship model across eight bounded contexts.

The Catalog context centers on Product with one-to-many Variant relationships, each owning image records whose embeddings power visual search. Taxonomy and Taxon trees manage hierarchical classification through self-referencing foreign keys. The Ordering context centers on Order with one-to-many LineItem, which capture price snapshots so historical orders are insulated from later catalog changes. The remaining modules are detailed per schema below.

2.3.4.3 Module Schemas and Aggregates

Each subsection below reviews one schema: its aggregate roots, the members each root owns, and the invariants that preserve consistency, together with the schema-level entity-relationship diagram.

2.3.4.3.1 Catalog Schema

The Catalog schema is product-centric. Product is the root aggregate: it owns variants, variant images and their embeddings, option configuration, and classification assignments, and it enforces a unique slug, a designated master variant, and model-tagged embedding vectors. Taxon is a second root that models the category tree using a nested-set representation; taxonomies group its top-level nodes and taxon rules drive automatic classification.

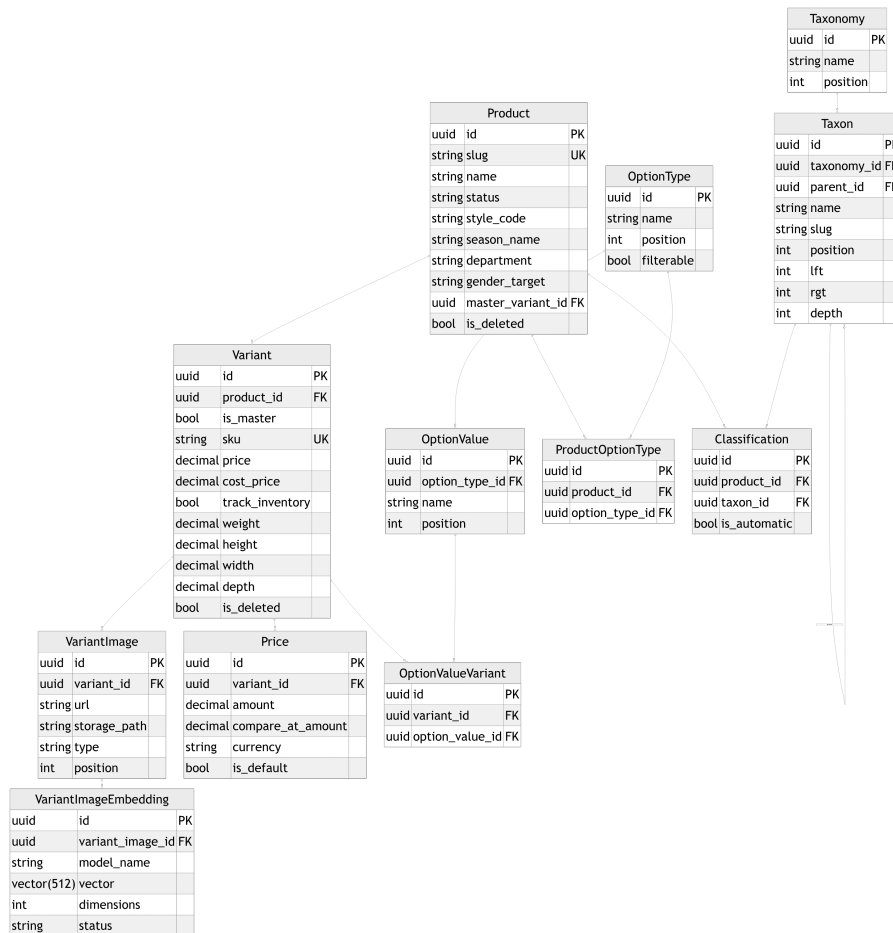


Figure 41: Catalog schema: product aggregate and taxonomy tree.

2.3.4.3.2 Identity Schema

The Identity schema is built on ASP.NET Identity Core. User is the aggregate root; it owns role and claim assignments, external logins, tokens, and rotating refresh tokens, and it will persist WebAuthn passkeys. Credentials are stored as Argon2 hashes, and single-use refresh tokens rotate through a token family so that a reused token revokes the whole session.

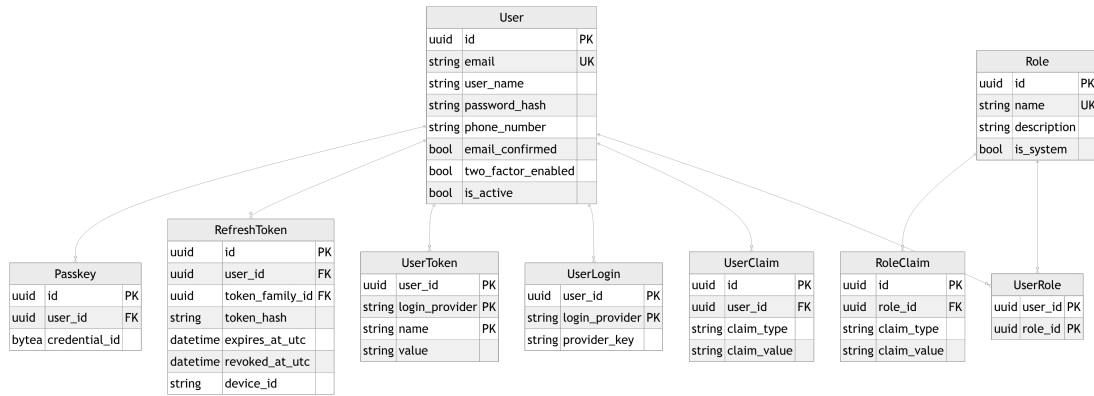


Figure 42: Identity schema: ASP.NET Identity user aggregate.

2.3.4.3.3 Ordering Schema

The Ordering schema is anchored by the Order aggregate, which drives the forward-only checkout state machine. Line items snapshot the ordered variants at purchase time, and adjustments record polymorphic tax, shipping, and discount lines. The order enforces total consistency (total equals item plus adjustment plus shipment totals) and keeps the checkout, payment, and shipment sub-states advancing forward only.

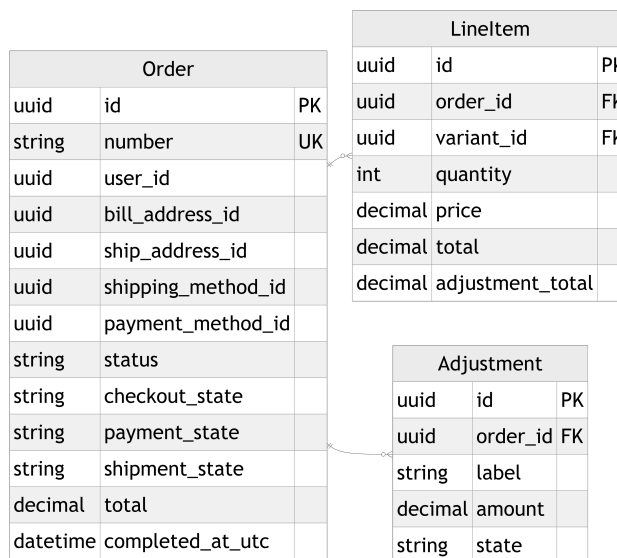


Figure 43: Ordering schema: order aggregate with line items and adjustments.

2.3.4.3.4 Payment Schema

The Payment schema centers on the PaymentCapture aggregate, which tracks a single authorisation through the Stripe lifecycle. Payment methods define the configured gateways, and webhook events log inbound Stripe notifications. The capture enforces that captured and refunded amounts never exceed the authorised amount and remains idempotent across replayed webhooks.

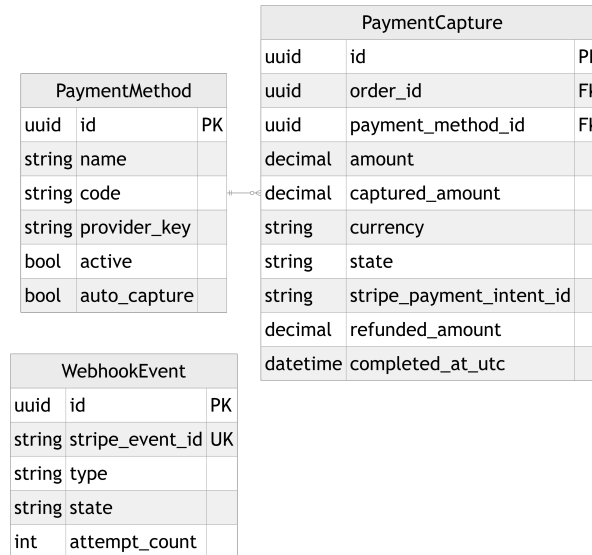


Figure 44: Payment schema: payment capture aggregate and gateway configuration.

2.3.4.3.5 Inventory Schema

The Inventory schema models stock through three roots. StockItem is the authoritative on-hand quantity per variant and location, protected by a uniqueness constraint and an append-only StockMovement ledger. StockReservation holds temporary quantities during checkout with an expiry-based auto-release. StockTransfer moves stock between two locations through a created-to-cancelled lifecycle, enumerated by transfer items.

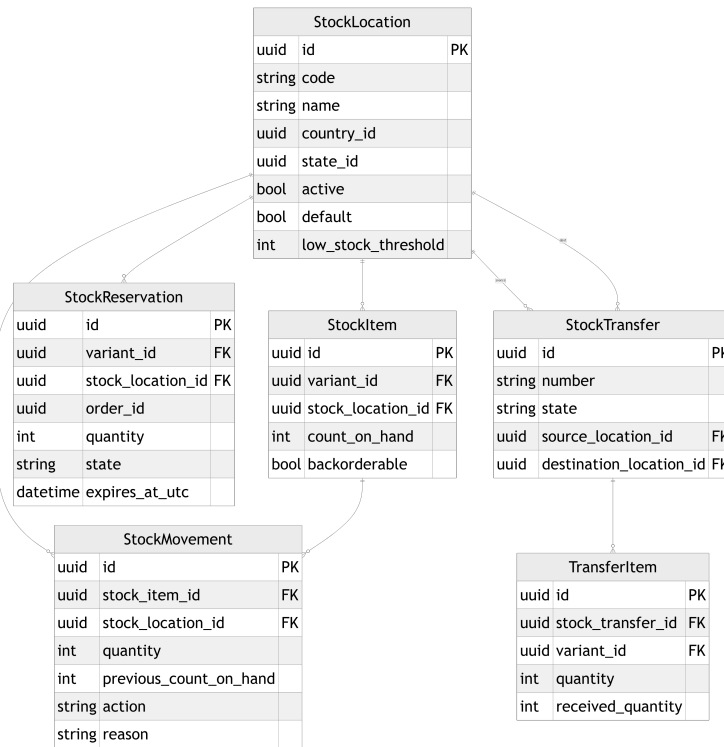


Figure 45: Inventory schema: stock item, reservation, and transfer aggregates.

2.3.4.3.6 Shipping Schema

The Shipping schema centres on the Shipment aggregate, which ties an order to a carrier and a destination address. Shipping methods define the carriers, shipping rates hold tiered cost bands, and shipping method zones restrict a method to specific countries and states.

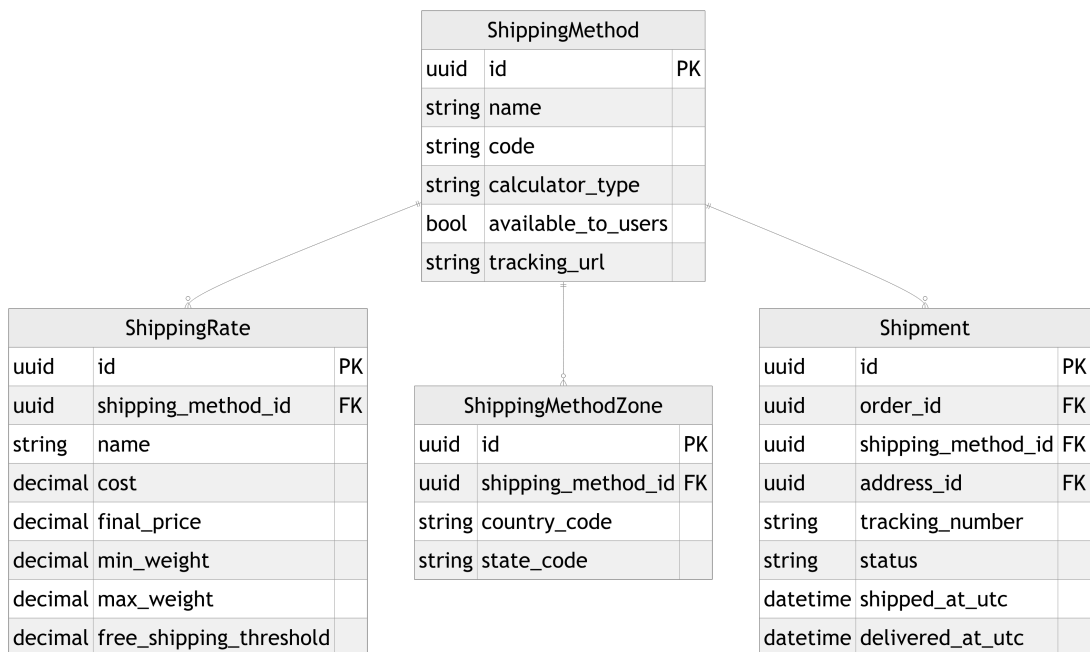


Figure 46: Shipping schema: shipment aggregate with methods, rates, and zones.

2.3.4.3.7 Location Schema

The Location schema provides ISO reference data. Country is the root aggregate and owns its State subdivisions, providing the validation rules and shipping-zone inputs used across the platform.

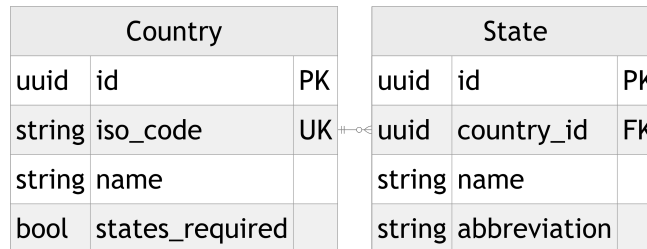


Figure 47: Location schema: country and state reference data.

2.3.4.3.8 Profile Schema

The Profile schema is customer-centric. UserProfile is the 1:1 root complementing the identity user and owns billing and shipping addresses. Wishlist is a second root that groups WishedItem entries, each referencing a variant, and enforces that a variant appears at most once per wishlist.

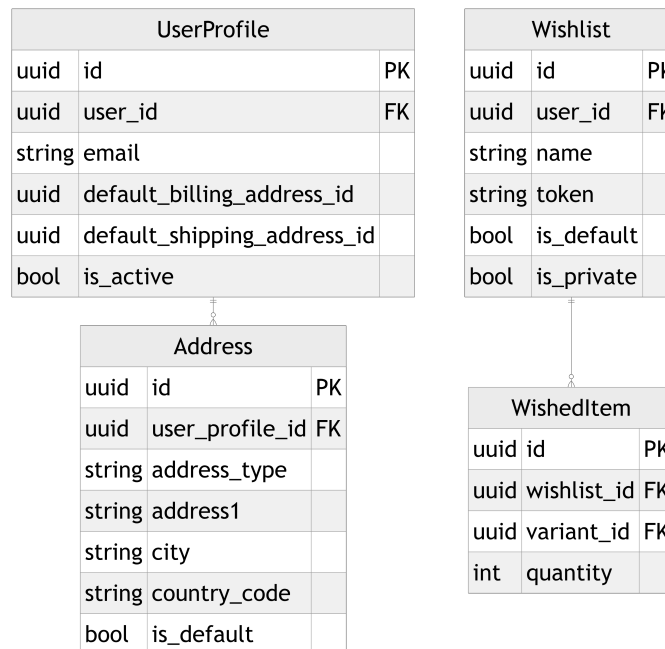


Figure 48: Profile schema: user profile and wishlist aggregates.

2.3.4.4 pgvector Integration

PostgreSQL's pgvector extension runs vector similarity searches inside the relational database. The `variant_image_embeddings` table stores feature vectors in an untyped vector column; per-model HNSW partial indexes cast the column to each model's dimensionality (e.g., 512 for Fashion-CLIP) and filter by `model_name`, using cosine distance to meet the sub-second CBIR latency target (NFR-GRP-01) (see Section 1.4.2 for index detail).

- **Cosine Distance:** The query operator (`<=>`) measures angular distance. Results rank by similarity score $1 - \text{cosine_distance}$, filtered against a configurable threshold (0.70).
- **Model Isolation:** Every record carries a `model_name` (e.g., Fashion-CLIP). Search queries filter by the active model to enforce embedding alignment.

2.3.4.5 Key Design Decisions

The data layer follows five global design rules:

- **UUID Primary Keys:** Safe client-side key generation and no sequential enumeration leakage.
- **Soft Deletion:** The soft-deletion flag is filtered globally by EF Core, preserving referential integrity.
- **Audit Columns:** Creation and modification timestamps are populated by EF Core save interceptors.
- **Composite Indexes:** Targeted indexes serve high-frequency access paths, such as (`user_id`, `status`) and (`session_id`, `status`) on orders.
- **Variable-Dimension Vector Storage:** The vector column is untyped *vector*, and a per-model HNSW partial index casts it to each model's dimensionality (512 for Fashion-CLIP, 384 for DINOv2 ViT-S/14, 768 for CLIP ViT-L/14, 1,280 for EfficientNet-B0, 2,048 for ResNet-50) and filters by *model_name*, letting one column serve models of differing output sizes.

2.3.5 API Design

The API exposes a RESTful interface via **Carter** modules and the **MediatR** CQRS pattern [18], registering approximately 262 endpoints across eight modules.

2.3.5.1 API Architecture

Each request follows a standard MediatR pipeline: Carter endpoint $\rightarrow$ *LoggingBehavior* $\rightarrow$ *ValidationBehavior* $\rightarrow$ *ExceptionMappingBehavior* $\rightarrow$ *Handler.Execute()* $\rightarrow$ *Result<T>.ToResult()* $\rightarrow$ HTTP Response (RFC 7807 Problem Details on error).

A representative endpoint pattern:

```

public void AddRoutes(IEndpointRouteBuilder app)
{
    app.MapPost(CatalogFeature.Admin.Products.Create.Route,
        async ([FromBody] Request r, ISender s, Cancellation token ct) =>
        {
            var result = await s.Send(new Command(r), ct);
            return result.ToResult();
        })
        .HasPermission(CatalogFeature.Admin.Products.Create.Permission);
}

```

Handlers return *Result<T>* success maps to 200/201/204, domain errors to RFC 7807 Problem Details.

2.3.5.2 Endpoint Organisation

Endpoints follow the convention */api/{surface}/{module}/{resource}*, where *surface* is *storefront* or *admin*. All *admin* routes enforce administrator policies via *.HasPermission()*. Eleven inter-module contract DTOs enable cross-module communication: *ReserveCartStock*, *ReleaseCartStockReservations*, *ConsumeCartStockReservations*, *CheckVariantAvailability* (Inventory); *GetCartForCheckout*, *GetCartForShipping*, *AdvanceCheckoutState* (Ordering); *GetPaymentForCheckout*, *MarkPaymentPaid* (Payment); *GetVariantDiscontinuedStatuses*, *GetVariantWeights* (Catalog).

2.3.5.3 API Endpoint Contract

Table 35 summarises the registered Carter endpoints.

Table 35: ReSys.Shop API contract: 262 Carter endpoints across eight modules.

Module	Admin Routes	Storefront Routes	N
Catalog	Products, variants, images, option types/values, taxonomies, pricing, dashboard	Product listing/search/detail, availability, similar products, CBIR search, taxonomy, taxon browsing	80
Identity	Users, roles, permissions catalogue	Login, register, logout, session refresh, email, password reset	37
Ordering	Orders, line items, status transitions, shipping/billing address, dashboard	Cart CRUD, cart items, checkout, shipping rate, customer orders	35
Inventory	Stock locations, stock items, bulk adjust, low stock, reservations, transfers, dashboard	Variant availability, cart reserve/release	32
Profile	Profiles, addresses CRUD	Profiles, addresses, notification preferences, wishlists	27
Location	Countries, states CRUD	Countries, states browse	18
Payment	Payment methods, payments list/detail, capture/void/refund	Payment intent create/confirm, methods, setup intent, Stripe webhook	17

Module	Admin Routes	Storefront Routes	N
Shipping	Shipping methods, shipping rates CRUD	Available methods, calculate cost, rates	15
Dashboard	Aggregated metrics: sales, inventory, catalog, activity	—	1

2.3.5.4 Error Handling

All API endpoints conform to RFC 7807 Problem Details: *400* for FluentValidation failures, *401* for missing/expired JWT, *403* for insufficient permissions, *404* for entity/route resolution failure, *409* for concurrency conflicts (PostgreSQL *xmin*), and *500* via global exception middleware preventing stack trace leakage.

2.3.6 Security Design

The security framework operates across three layers: authentication, authorization, and defense-in-depth infrastructure.

2.3.6.1 Authentication and Session Management

JWT authentication is configured via *JwtSettings* with HS256 algorithm, 15-minute access token expiration, and 30-day maximum token age. Single-use refresh token rotation is enforced: exchanging an expired access token consumes the current refresh token and issues a new pair. If a refresh token that has already been used is submitted again, this is treated as a possible security breach. All active refresh tokens for that user are immediately revoked, and the user must log in again. Unauthenticated shoppers receive an HTTP-only cookie tracking an anonymous session ID; on login, the guest cart automatically merges with the user's persistent cart.

```
// JWT configuration (JwtSettings)
public sealed class JwtSettings
{
    public string Secret { get; init; }
    public string Issuer { get; init; }
    public string Audience { get; init; }
    public int AccessTokenExpirationInMinutes { get; init; } = 15;
    public int RefreshTokenExpirationInDays { get; init; }
    public string Algorithm { get; init; } = "HS256";
    // Token security
    public bool RotationEnabled { get; init; } = true;
    public bool ReuseDetectionEnabled { get; init; } = true;
    public bool SlidingExpirationEnabled { get; init; } = true;
    public int MaxTokenAgeDays { get; init; } = 30;
}
```

2.3.6.2 Dynamic Authorization

Role-Based Access Control separates administrative and storefront surfaces. Unprivileged accounts accessing */api/*/admin/** endpoints receive an immediate *403* prior to command dispatch. The built-in Administrator role bypasses all permission checks.

A three-layer permission architecture resolves claims at runtime:

```
// Permission resolution pipeline
// L1: IPermissionCache      - in-memory cache of resolved permissions per
// user
// L2: IPermissionService    - merges role-derived + direct-grant
// permissions
// L3: IPermissionStore      - EF Core persistence to Identity claims
// tables

// Custom policy provider resolves permission names dynamically
public sealed class PermissionPolicyProvider :
IAuthorizationPolicyProvider
{
    {
        public Task<AuthorizationPolicy> GetPolicyAsync(string policyName)
        {
            if (PermissionContext.IsKnown(policyName))
                return Task.FromResult(new AuthorizationPolicyBuilder()
                    .AddRequirements(new PermissionRequirement(policyName))
                    .Build());
            return FallbackProvider.GetPolicyAsync(policyName);
        }
    }
}
```

Operations enforce granular resource-action claims. Ten *FeatureMetadata* files define the permission registry across modules, each mapping to the *PermissionContext* static catalogue. Permissions use the dot-separated format *domain.category.resource.action*:

```
admin.catalog.products.create
admin.catalog.products.update
admin.catalog.products_variants.delete
admin.identity.roles.manage
admin.ordering.orders.approve
admin.ordering.payments.capture
admin.inventory.stockitems.transfer
```

A custom *IAuthorizationPolicyProvider* resolves claim strings to policies dynamically at runtime, allowing permission modifications in the database without application redeployments.

2.3.6.3 System Hardening

Rate limiting restricts authentication to 5 requests per minute, registration to 3 per hour, and payment processing to 30 per minute. Security middleware injects *Content-Security-Policy*, *Strict-Transport-Security*, *X-Frame-Options*, and *X-Content-Type-Options* headers. Visual search uploads enforce a 10 MB limit and validate magic bytes to verify valid JPEG, PNG, or WebP formats. Stripe payment webhooks validate the *Stripe-Signature* header using HMAC signature verification before executing state transitions.

The storage service includes a pre-upload virus scanning layer via ClamAV (nClam integration) and image format validation through SkiaSharp, preventing malicious payloads from reaching persistent storage.

2.4 IMPLEMENTATION

This section shows how the design from Sections 2.2 and 2.3 was built into working software. It follows the actual structure of the system: the technology

stack used for development, the vertical slice pattern that organizes the .NET codebase, the layer that stores relational and vector data in a single database, the machine learning sidecar (the main research contribution of this project), and the frontend applications that users interact with.

- **Technology Stack.** Framework versions and how the system is containerized.
- **Vertical Slice Core.** Feature co-location, the Carter–MediatR request pipeline, and the functional Result pattern.
- **Data Persistence.** Multi-schema EF Core, pgvector integration with HNSW indexing, and concurrency control.
- **ML Sidecar.** Model management, the embedding generation pipeline, and the full CBIR search flow.
- **Frontend Applications.** Dual-SPA architecture, the visual search interface, and key administration workflows.

2.4.1 Technology Stack

The platform uses pinned versions across three ecosystems: .NET 10 for transactional semantics and API abstractions, Python 3.12 for deep learning (PyTorch, Hugging Face), and Vue 3 for reactive UIs. Centralized package management enforces reproducibility via *Directory.Packages.props* (NuGet), *uv.lock* (Python), and *pnpm-lock.yaml* (JavaScript).

Table 36 details the core technologies grouped by architectural role.

Table 36: Principal technologies grouped by architectural role with pinned versions.

Architectural Role	Technology and Version
Backend Runtime	.NET 10 / ASP.NET Core 10.0.9 (C# 13)
API Framework	Carter 10.0.0, MediatR 14.1.0, FluentValidation 12.1.1
Object Mapping	Mapster 10.0.9
ORM and Database	EF Core 10.0.9, Npgsql 10.0.2, pgvector 0.7.0
Caching Layer	HybridCache 10.6.0, StackExchange.Redis 10.0.9
Background Jobs	Hangfire 1.8.23 (Redis-backed)
Observability	OpenTelemetry 1.16.0
ML Runtime	Python 3.12, PyTorch >= 2.0.0, TorchVision >= 0.15.0
ML Framework	FastAPI >= 0.115.0, Uvicorn, Hugging Face Transformers
ONNX Runtime	onnxruntime >= 1.17.0
Storefront UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Aura)
Admin UI	Vue 3.5, TypeScript 6.0, Vite 8, PrimeVue 5 (Sakai), Chart.js 4

Architectural Role	Technology and Version
State and HTTP	Pinia, Axios 1.x, Zod
Data Storage	PostgreSQL 17 (pgvector/pgvector:pg17-trixie)
Cache Storage	Redis 7 (Alpine)
Test Suites	xUnit v3 3.2.2, pytest >= 8.0, Vitest 4, Playwright

2.4.1.1 Service Containerization

The platform defines six containerized resources with startup dependencies: PostgreSQL and Redis initialize first, followed by the Python ML sidecar with */health* readiness probe, then the .NET API. Vue SPAs run as Vite dev servers reverse-proxying */api/* endpoints. Multi-stage Docker builds isolate runtime dependencies with non-root execution via *tini*.

2.4.2 Vertical Slice Architecture Core

The .NET backend implements Vertical Slice Architecture (VSA) combined with Command Query Responsibility Segregation (CQRS), organizing code by business capability rather than technical layer. Each feature co-locates its request DTOs, domain logic, validation, Carter endpoint, and response models within a single directory using *static partial class* definitions.

2.4.2.1 Feature Co-Location and Execution Mechanics

Every feature is structured across five files: *{Feature}.cs*, *{Feature}.Request.cs*, *{Feature}.Response.cs*, *{Feature}.Endpoint.cs*, and *{Feature}.Validator.cs*. The handler pipeline (Validate → Build → Complete):

Table 37: Sequential execution flow within the CreateProduct command handler.

Step	Action
1. Validate	Checks slug uniqueness; returns <i>DuplicateSlug</i> on collision.
2. Build	Constructs <i>Product</i> via domain factory, persists to database, dispatches cross-module <i>AddVariant</i> via <i>ISender</i> .
3. Complete	Assigns master variant ID, returns <i>Result<Response>.Created(...)</i> .

Communication between modules uses only *ISender.Send()* messages. Bounded contexts never import another context's namespace. This keeps the modules isolated within a single assembly, checked using static analysis and build rules.

```
public sealed record Command(Request Request) : ICommand<Response>;
public sealed class CommandHandler(IApplicationDbContext db, ISender sender)
```

```
        : ICommandHandler<Command, Response>
    {
        public async Task<Result<Response>> Handle(Command c,
            CancellationToken ct)
        {
            if (await db.Set<Product>().AnyAsync(x => x.Slug ==
                c.Request.Slug, ct))
                return ProductResult.Errors.DuplicateSlug;

            var product = c.Request.MapToDomain().Value;
            db.Set<Product>().Add(product);
            await db.SaveChangesAsync(ct);

            var v = await sender.Send(new AddVariant.Command(product.Id, vr),
                ct);
            product.MasterVariantId = v.Value.Id;
            await db.SaveChangesAsync(ct);

            return Result<Response>.Created(product.MapToDetail<Response>(),
                ProductResult.Success.Created(product.Id));
        }
    }
```

2.4.2.2 Request Pipeline Architecture

Each feature implements *ICarterModule* with thin endpoints that parse requests, construct MediatR commands, dispatch via *ISender*, and convert *Result<T>* to HTTP responses. Routes follow */api/{surface}/{module}/{resource}* with Carter auto-discovery at startup.

Three MediatR pipeline behaviors wrap every command: *LoggingBehavior* (outermost, captures request metadata and duration), *ValidationBehavior* (runs FluentValidation, short-circuits on failure), and *ExceptionMappingBehavior* (innermost, wraps unhandled exceptions).

2.4.2.3 Functional Result Pattern

The application uses *Result<T>* (*readonly record struct*) instead of exception-driven control flow. Domain methods return explicitly typed results; *Error* structs carry machine-readable codes, human-readable messages, and optional metadata. The *ToResult()* extension maps domain results to HTTP status codes: 200 OK or 201 Created for success, 400 for validation failures, 404 for missing entities, and 409 for state conflicts.

2.4.3 Data Persistence Architecture

The persistence layer maps eight bounded contexts to dedicated PostgreSQL schemas. Vector embeddings and relational product data are stored together, in a single PostgreSQL 17 instance.

2.4.3.1 Schema Organisation and Vector Storage

Each bounded context owns an isolated schema. Table 38 outlines the distribution:

Table 38: Schema-per-context mapping with UUID references, no cross-schema foreign keys.

Schema	Principal Tables and Aggregate Entities
<i>catalog</i>	<i>products</i> , <i>variants</i> , <i>variant_images</i> , <i>product_image_embeddings</i> , <i>taxonomies</i> , <i>taxons</i>
<i>ordering</i>	<i>orders</i> , <i>line_items</i> , <i>order_adjustments</i> , <i>shipments</i>
<i>payment</i>	<i>payment_intents</i> , <i>payment_captures</i> , <i>payment_methods</i>
<i>inventory</i>	<i>stock_locations</i> , <i>stock_items</i> , <i>stock_movements</i> , <i>stock_reservations</i>
<i>identity</i>	<i>users</i> , <i>roles</i> , <i>user_roles</i> , <i>refresh_tokens</i>
<i>profile</i>	<i>user_profiles</i> , <i>addresses</i> , <i>wishlists</i>
<i>shipping</i>	<i>shipping_methods</i> , <i>shipping_rates</i> , <i>shipping_zones</i>
<i>location</i>	<i>countries</i> , <i>states</i>

Cross-schema references use unconstrained UUIDs, enforcing module independence. Product embeddings reside in *catalog.product_image_embeddings* alongside product metadata, participating in the same ACID transactions.

2.4.3.2 pgvector Indexing Strategy

Embeddings are stored in an untyped *vector* column, discriminated per model through a *model_name* filter column (e.g., *fashion_clip*). Per-model HNSW partial indexes cast the column to each model’s dimensionality and use cosine distance for sub-second CBIR queries (see Section 2.3.4 for index detail and Section 1.4.2 for ANN algorithm comparison).

2.4.3.3 Model-Aware Vector Search

Each embedding carries a *model_name* discriminator and its *dimensions*. Cosine similarity search retrieves candidates via $\leq$ filtered by *model_name* and dimension, transforms distance to similarity, and deduplicates to one result per product; the storefront additionally applies a configurable minimum-similarity threshold and score weighting to the returned scores.

2.4.3.4 Concurrency and Data Integrity

Three complementary strategies maintain integrity under concurrent throughput:

Table 39: Concurrency control mechanisms across system domains.

Pattern	Implementation	Applied To
Optimistic Locking	PostgreSQL <i>xmin</i> column via EF Core <i>IsRowVersion()</i>	General entity updates (HTTP 409 on stale write)
Pessimistic Locking	<i>SELECT FOR UPDATE</i> row locks	Stock allocation during checkout (serializes low-quantity purchases)

Pattern	Implementation	Applied To
Repeatable Read	IsolationLevel.RepeatableRead transaction scope	Sequential order code generation (prevents phantom reads)

Audit tracking uses EF Core interceptors: *AuditInterceptor* auto-populates *CreatedAtUtc*, *CreatedBy*, *UpdatedAtUtc*, and *UpdatedBy* during *SaveChangesAsync*. Stock modifications write append-only entries to *inventory.stock_movements*.

2.4.4 ML Sidecar and CBIR Search

The Python ML sidecar generates vector embeddings from product images over HTTP, managing multiple pre-trained models through a strategy pattern switchable at runtime via *EMBEDDING_MODEL*.

2.4.4.1 Model Management

Six models span four architecture families, selected from a decorator-based registry on first inference:

Table 40: Registered embedding models with output dimensionality and architectural family.

Model ID	Family	Dim	Architecture	Source
CNN	efficientnet-b0	1280	EfficientNet-B0	torchvision ImageNet1K_V1
	resnet-50	2048	ResNet-50	torchvision ResNet50_Weights
ViT	dinov2-vits14	384	ViT-S/14	facebookresearch/ dinov2 (torch.hub)
CLIP	clip-vit-b16	512	ViT-B/16 + CLIP	OpenAI CLIP (torchvision)
	openclip-vit-b-32	512	ViT-B/32 + CLIP	OpenCLIP (HuggingFace)
Fashion	fashion-clip	512	ViT-B/32 + CLIP	patrickjohnncyh/fashion-clip (HuggingFace)

All models conform to *BaseEmbedder* via the Template Method pattern. The base class handles loading, forwarding, and L2-normalisation; subclasses provide only the forward pass. Models load lazily on first request within *torch.no_grad()*. The *device* property resolves CUDA, MPS, or CPU at runtime.

```
class BaseEmbedder(ABC):
    @abstractmethod
    def forward(self, image: torch.Tensor) -> torch.Tensor:
        """Model-specific forward pass."""
```

```

async def extract(self, image_input) -> list[float]:
    image = self._load_image(Image input)
    features = self._forward(image)
    return self._normalize(features)

```

`_load_image` handles URLs, paths, raw bytes, and PIL Images, converting to RGB tensors. `_normalize` applies L2-normalisation and handles CLIP `image_embeds`, ViT `pooler_output`, and CNN feature maps via global average pooling.

2.4.4.2 Embedding Generation Pipeline

Six pipeline stages:

Table 41: Embedding generation pipeline stages.

Stage	Component	Description
1	Authentication	Validates <i>X-API-Key</i> header; rejects with 401.
2	Model Selection	Retrieves active model from registry; initialises from disk if unallocated.
3	Preprocessing	Resizes to 224×224 px, converts to tensor, applies ImageNet normalisation.
4	Forward Pass	Propagates through convolution or self-attention within <code>torch.no_grad()</code> .
5	Pooling	Global average pooling to fixed-length vector (512-dim for CLIP, up to 2048 for ResNet-50).
6	Serialization	L2-normalisation, JSON packaging with model metadata and inference latency.

API endpoints:

Table 42: ML sidecar API endpoints requiring *X-API-Key* authentication.

Method	Path	Purpose
POST	<code>/embeddings/bytes</code>	Multipart image upload with optional <i>model</i> query parameter.
POST	<code>/embeddings</code>	Image URL for batch embedding during catalogue indexing.
GET	<code>/health</code>	Readiness probe: validates CUDA/MPS, active model, and synthetic forward pass.

Response shape:

```

{
  "value": {
    "vector": [0.023, -0.154, ..., 0.042],
    "model": "fashion_clip",

```

```

    "dimensions": 512,
    "inference_ms": 96.8
  },
  "isSuccess": true,
  "statusCode": 200
}

```

2.4.4.3 CBIR Search Flow

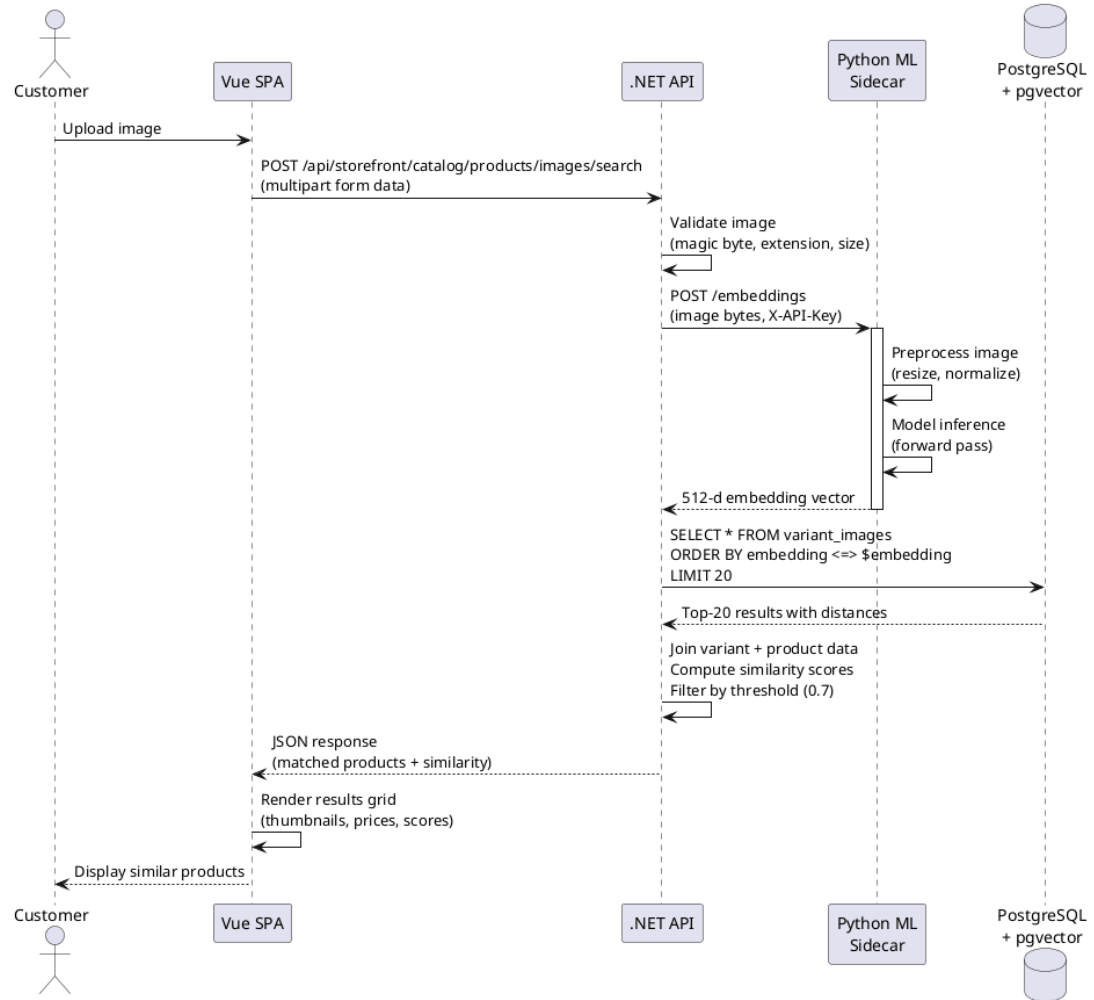


Figure 49: CBIR search sequence: upload, embedding extraction, pgvector search, results.

Six stages across four architectural layers:

1. **Client Validation (Vue 3).** Validates format (JPEG, PNG, WebP) and size (≤ 10 MB). Dispatches multipart form to *POST /api/storefront/catalog/products/images/search*.
2. **Server Validation (.NET API).** Verifies magic bytes, reapplies payload ceiling.
3. **Vector Extraction (ML Sidecar).** Forwards image bytes to */embeddings* sidecar executes preprocessing and inference.

4. **Vector Search (pgvector).** Queries via cosine distance $\leq$ filtered by *model_name* HNSW enables sub-10 ms lookup.
5. **Post-Processing.** Converts distance to similarity ($1 - \text{distance}$), deduplicates by product; the storefront then applies a configurable minimum-similarity threshold to the returned scores.
6. **UI Rendering (Vue 3).** Renders product grid with thumbnails, prices, and similarity scores within the sub-second budget.

The pluggable architecture supports automated benchmarking (update *EMBEDDING_MODEL*, restart, sequential benchmarking of all models without code changes) and production A/B testing via dual sidecar instances with distinct models.

2.4.5 Frontend Applications

The user-facing components consist of two Vue 3 single-page applications: a customer storefront (PrimeVue Aura theme) and an administration dashboard (PrimeVue Sakai theme with Chart.js 4). This section presents the implemented interfaces organized by the 26 use cases defined in Section 2.2.

2.4.5.1 Frontend Architecture

Both applications share Vue 3.5, TypeScript 6.0, Vite 8, Pinia, and Axios. All backend communication follows a typed repository pattern mirroring the .NET *Result<T>* convention with *isSuccess*, *statusCode*, *data*, and *errors[]* fields. The *BaseRepository* wraps Axios with typed CRUD:

```
export interface Result<T> {
  isSuccess: boolean
  isFailure: boolean
  statusCode: number
  data?: T
  errors?: Array<{ code: string; description: string; field?: string }>
}
```

For visual search, the repository extends the base with a multipart upload method: *async searchByImage(file: File): Promise<Result<Product[]>>* dispatching *POST /api/storefront/catalog/products/images/search* with *Content-Type: multipart/form-data*.

2.4.5.2 Storefront Interfaces

The customer storefront implements nine use cases covering product discovery, purchasing, and account management.

2.4.5.2.1 Visual Search: UC-STR-SRC

The visual search interface implements a four-state UI model:

Table 43: Visual search UI state model.

State	Display	Trigger
Empty	Upload prompt with dashed-border drop zone, cloud-upload icon, and format guidance text	Page load or after clearing previous search
Upload	Selected image preview with a “Search Similar Products” button below	User drops or selects an image file
Loading	Skeleton grid of animated placeholder cards (4 columns desktop, 2 mobile)	Search API request in flight
Results	Product card grid with thumbnails, names, prices, and colour-coded similarity badges	API returns non-empty results array

Two input methods are supported: drag-and-drop onto the drop zone and file browser selection. Client-side validation rejects non-image MIME types and files exceeding 10 MB:

```
{
  "results": [{
    "productId": "a1b2c3d4-...",
    "title": "Floral Summer Midi Dress",
    "price": 750000, "currency": "VND",
    "thumbnailUrl": "/images/products/a1b2c3d4_thumb.webp",
    "similarityScore": 0.9328
  }],
  "searchDurationMs": 287,
  "model": "Fashion-CLIP"
}
```

Each product card renders a thumbnail, name, formatted price, and colour-coded similarity badge (green for $\geq 90\%$, amber for $\geq 80\%$, grey below). The query image persists in a sidebar panel while scrolling results.

The four visual search states are illustrated below.

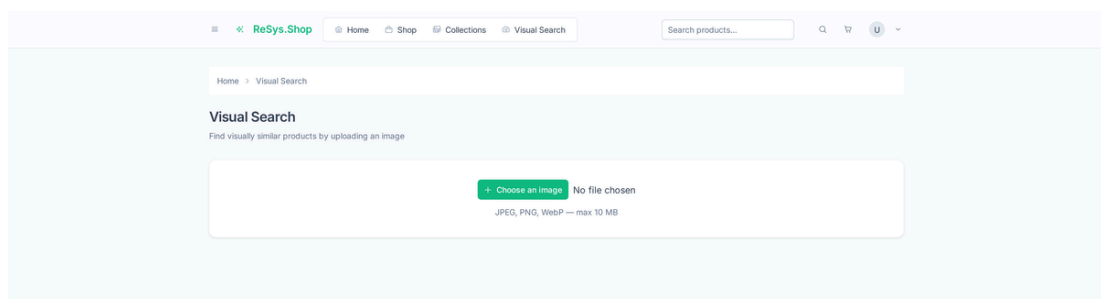


Figure 50: Visual search empty state: drop zone with format note (max 10 MB).

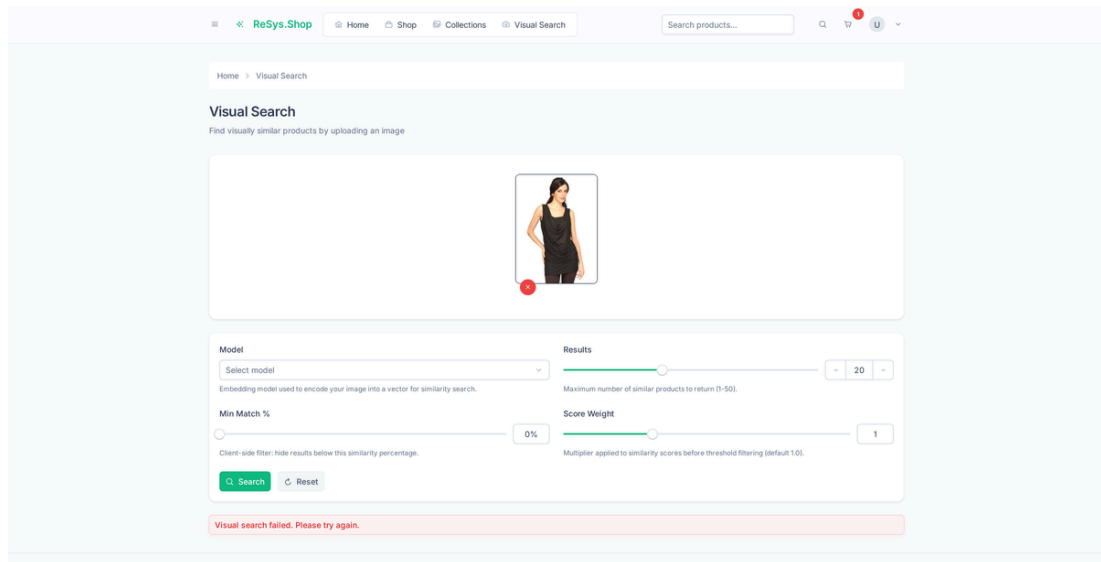


Figure 51: Upload state: selected-image preview with Search Similar Products button.

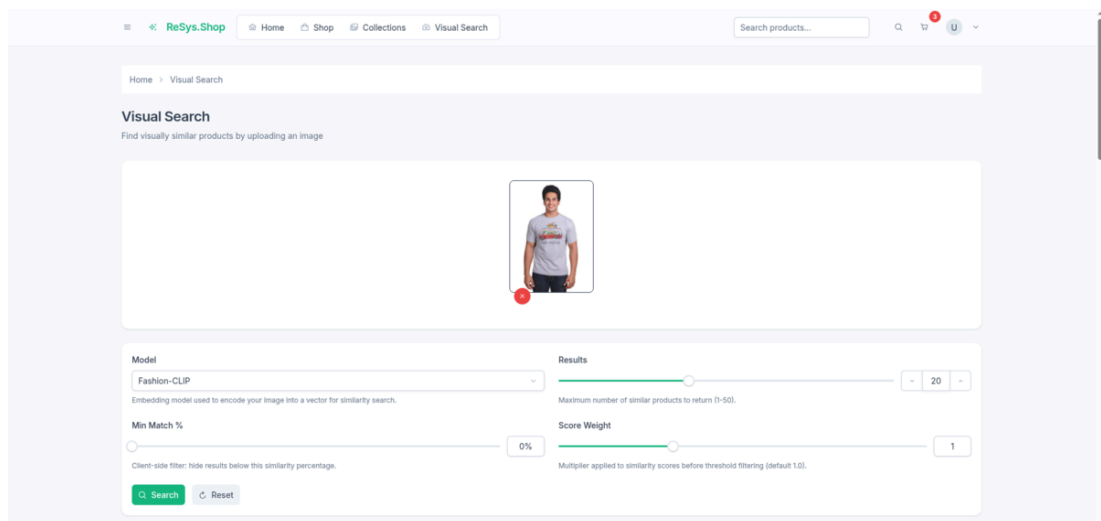


Figure 52: Search parameters panel: Fashion-CLIP model selector and three sliders.

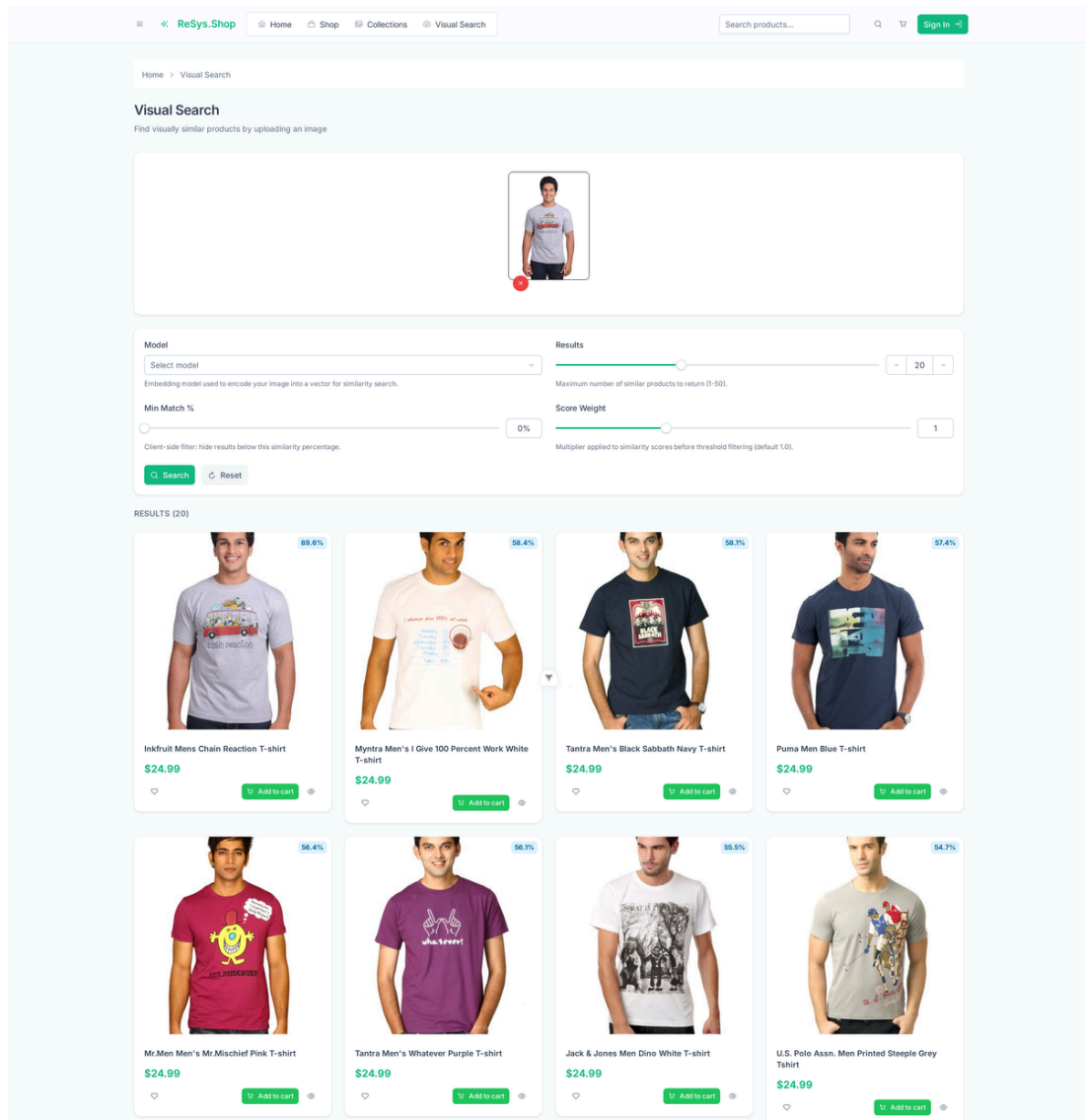


Figure 53: Results state: product grid with similarity badges; query image in sidebar.

2.4.5.2.2 Catalogue Browsing: UC-STR-BRW

The storefront home page presents the brand hero, curated collections, and featured product carousel. The catalogue page presents products in a paginated, faceted grid with left-sidebar category tree navigation built from hierarchical taxonomy data. Selecting a category filters the grid; a keyword search bar sends full-text queries. Product cards show thumbnails, names, prices, and a hover overlay with quick-add-to-cart (see screenshots below).

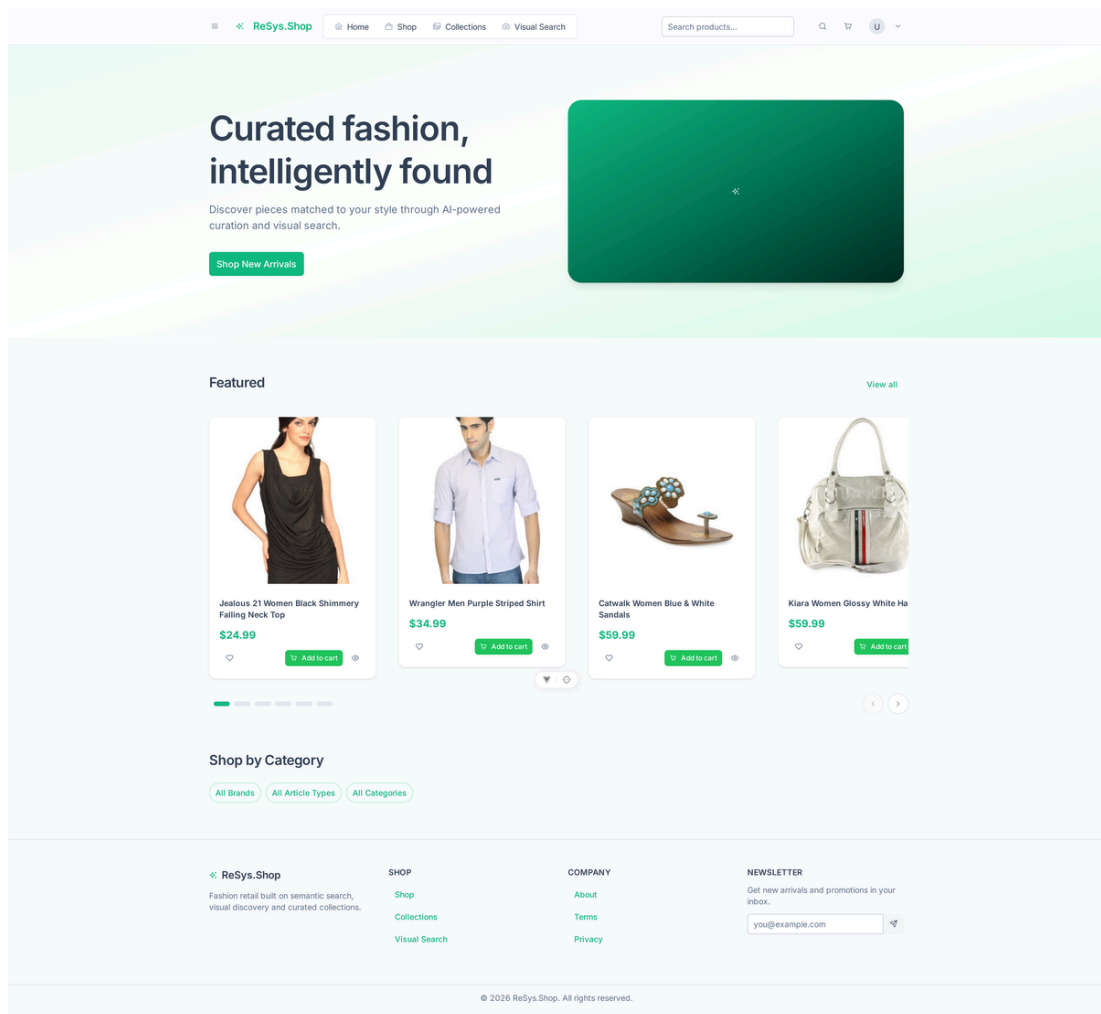


Figure 54: Storefront home: hero banner, collection cards, featured carousel.

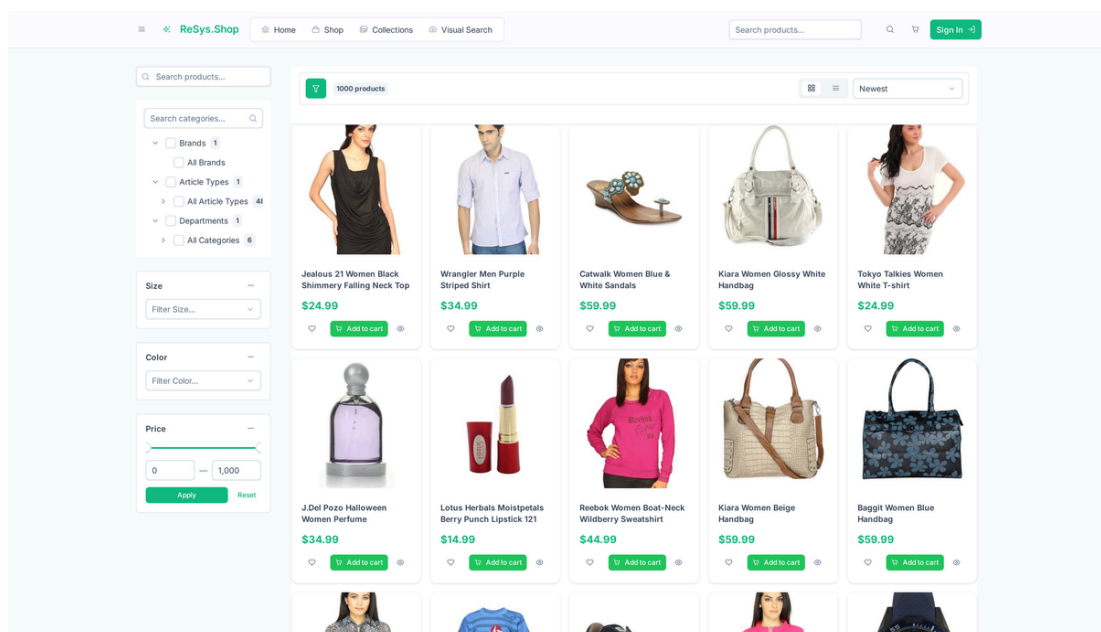


Figure 55: Catalogue browsing: sidebar category tree, paginated product grid, search bar.

The product detail page displays complete product information with a variant image gallery, size and colour pickers with real-time stock indicators, current price with strikethrough original price, quantity selector, and Add to Cart button. Expandable sections provide description, material, care instructions, and size guide. A Similar Products carousel appears at bottom (see screenshot below).

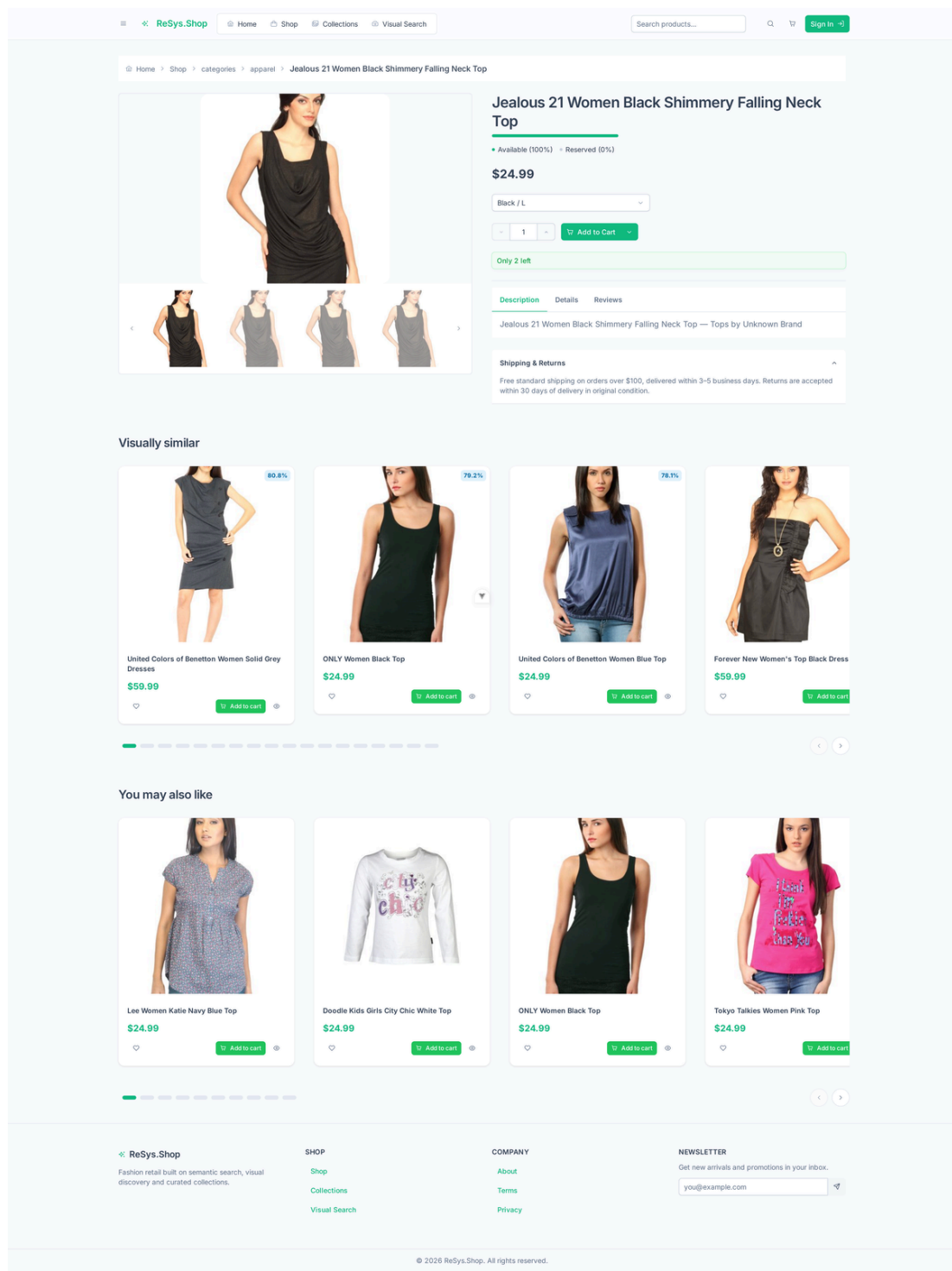


Figure 56: Product detail: gallery, size/colour pickers with stock badges, Add to Cart.

2.4.5.2.3 Shopping Cart: UC-STR-CRT

The cart page lists line items with thumbnail, title, variant details, unit price, quantity controls, per-line subtotal, and remove button. A sticky summary panel shows item count, subtotal, and Proceed to Checkout button. Cart data persists via backend API and synchronises across tabs through Pinia state. Guest carts merge into authenticated accounts on login (see screenshots below).

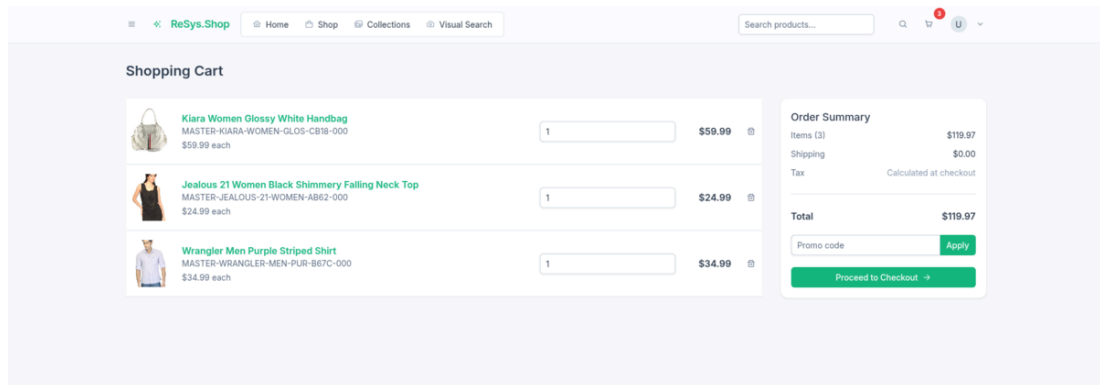


Figure 57: Cart: line items with quantity controls and sticky Order Summary panel.

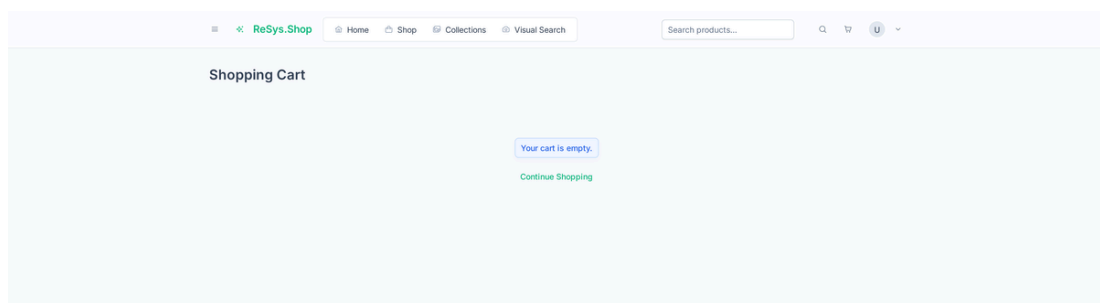


Figure 58: Empty cart: bag icon with Continue Shopping link back to catalogue.

2.4.5.2.4 Checkout: UC-STR-CHK

Checkout progresses through the five-state pipeline (Address, Delivery, Payment, Confirm, Complete) with a Stepper wizard. Each step renders as a single-page form; each transition is validated server-side.

- **Address:** Saved addresses with picker selection and inline “Add new address” form (screenshot below).
- **Delivery:** Available shipping methods with carrier names, delivery estimates, and calculated rates (screenshot below).
- **Payment:** Payment methods with provider icons, selected method confirmed via payment-intent creation (screenshot below).

- **Confirm:** Read-only order summary with line items, addresses, method, and totals; “Place Order” button finalises (screenshot below).
- **Complete:** Order confirmation with generated order number, summary, and “Continue Shopping” link (screenshot below).

ReSys.Shop Home Shop Collections Visual Search Search products...

Checkout

1 Shipping 2 Delivery 3 Payment 4 Review 5 Confirmation

Saved Address
Choose a saved address

First Name
Last Name
Street Address
City
ZIP / Postal Code
Phone
Email
user@resys.shop
Country / State
Country / State

Back to Cart Continue to Delivery

Figure 59: Address step: stepper wizard, saved-address picker with inline add form.

ReSys.Shop Home Shop Collections Visual Search Search products...

Checkout

1 Shipping 2 Delivery 3 Payment 4 Review 5 Confirmation

Express Shipping \$14.99
2-3 business days
EXPRESS

Free Shipping \$9.99
7-14 business days
FREE

Standard Shipping \$5.99
5-7 business days
STANDARD

Back Continue to Payment

Figure 60: Delivery step: shipping methods with carriers, delivery dates, and rates.

ReSys.Shop Home Shop Collections Visual Search Search products...

Checkout

1 Shipping 2 Delivery 3 Payment 4 Review 5 Confirmation

Credit Card
CASH_ON_DELIVERY
Test Card (Bogus)
Bogus Card

Back Continue

Figure 61: Payment step: stepper wizard with Credit Card and Cash on Delivery options.

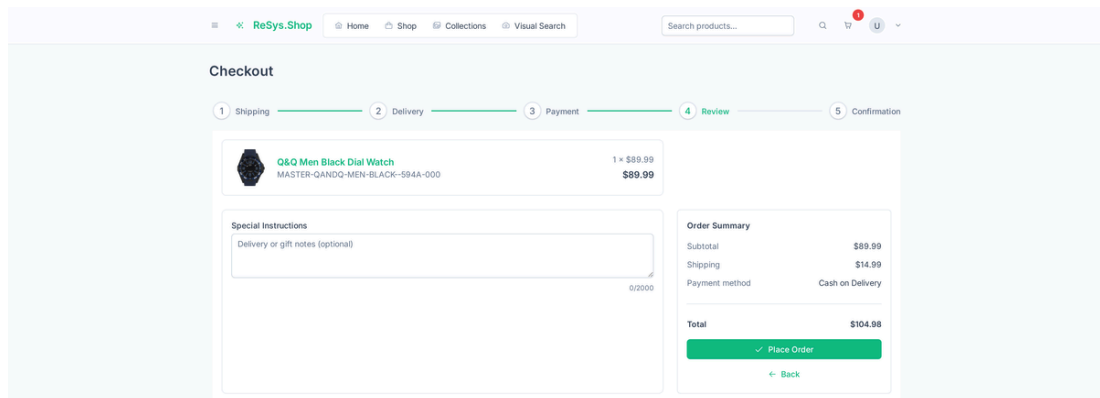


Figure 62: Confirm step: read-only summary with totals and Place Order action.

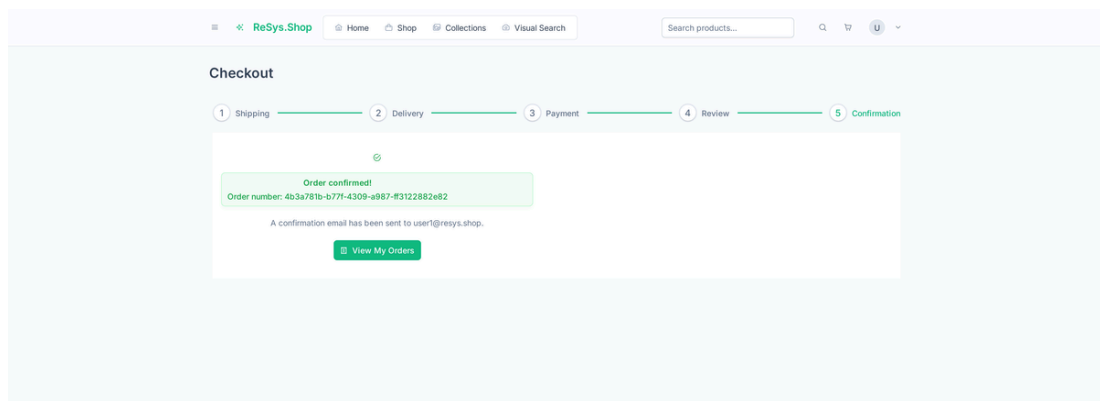


Figure 63: Complete step: confirmation with generated order number and summary.

2.4.5.2.5 Order History: UC-STR-OHI

Past orders display in vertically stacked cards with order number, date, colour-coded status badge, item count, and total. Expanding a card reveals line items with thumbnails, quantities, and prices. Active orders show a Cancel button when cancellable. A detail page displays the full state transition timeline (see screenshots below).

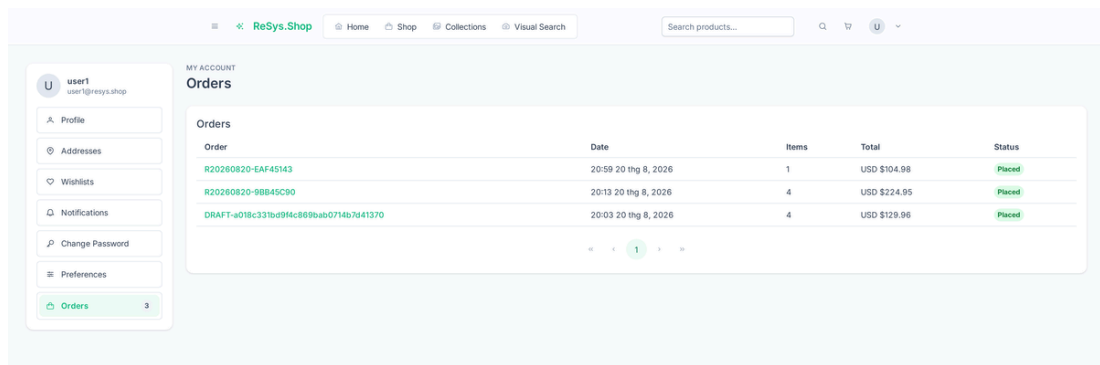


Figure 64: Order history: stacked order cards with status badges, expandable line items.

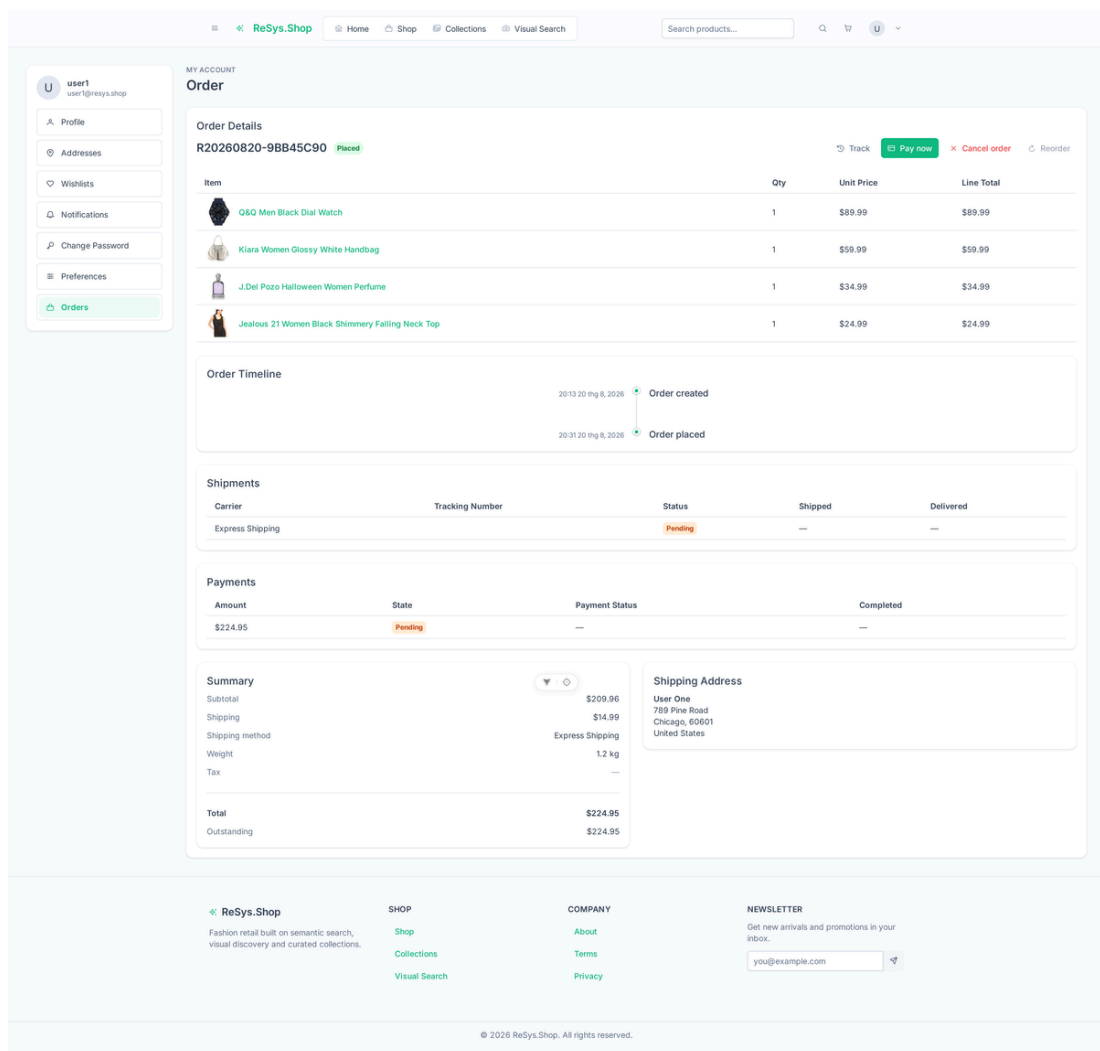


Figure 65: Order detail: status header, state timeline, line items, addresses, totals.

2.4.5.2.6 Authentication: UC-STR-AUT, UC-STR-SES

Two authentication pathways: email/password login and guest sessions. Registration includes email, password with strength indicator, and full name.

Password reset uses a two-step email-token flow. The session page lists active sessions with device, IP, and last-activity; “Logout All Devices” terminates all sessions (see screenshots below).

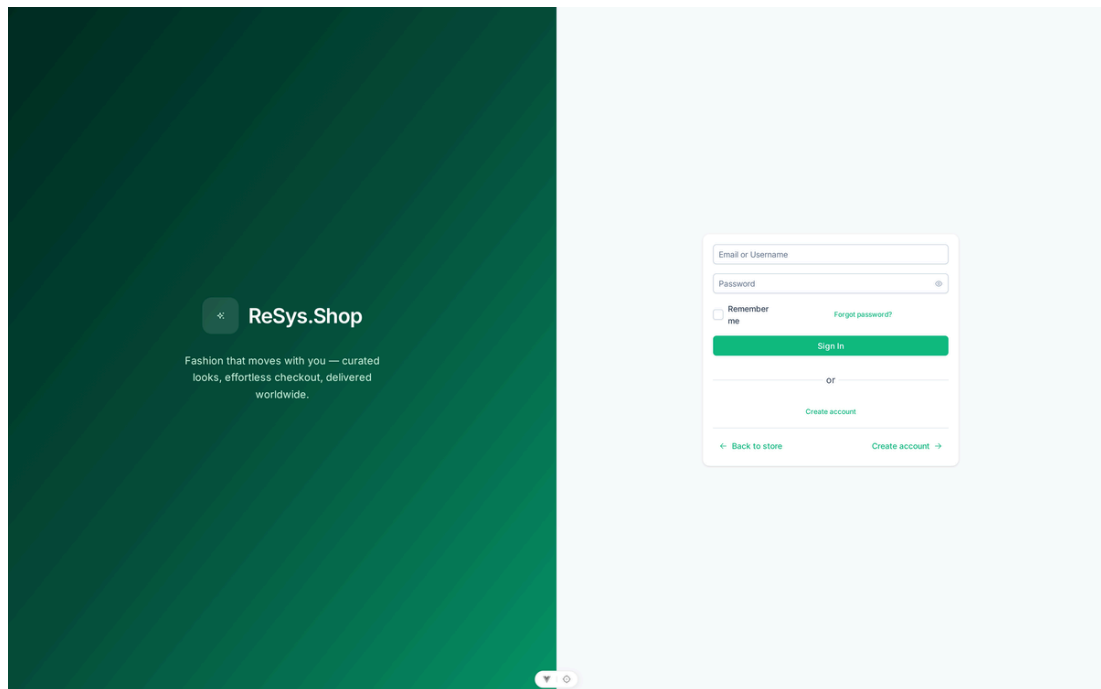


Figure 66: Login page: email/password card with register link.

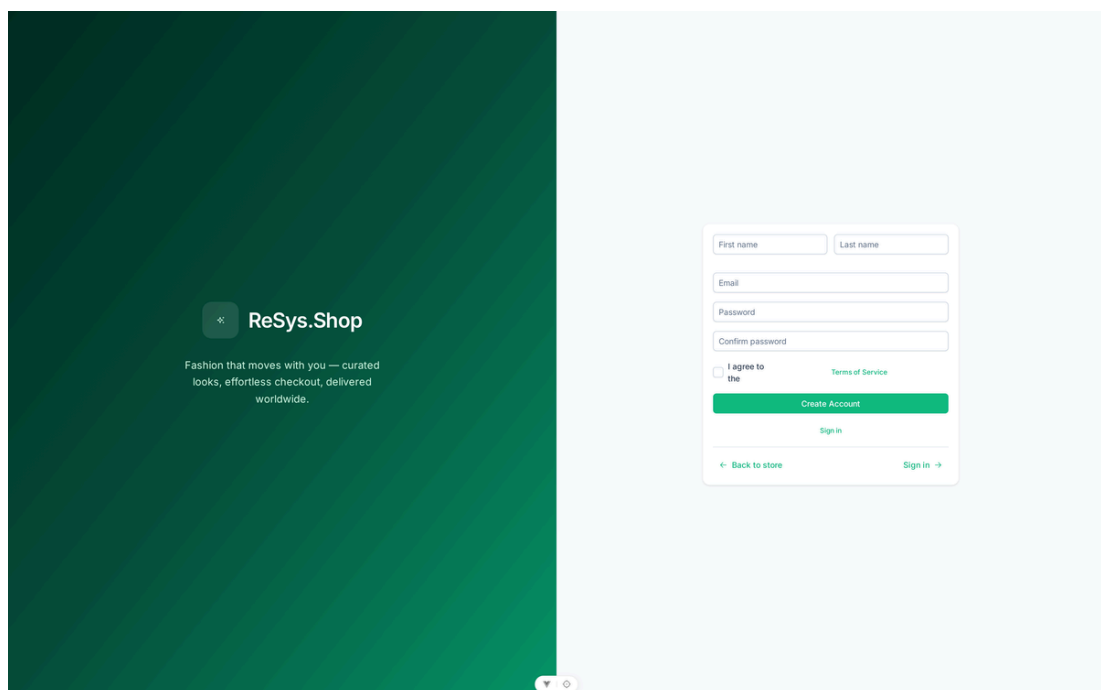


Figure 67: Registration page: account form with password strength indicator.

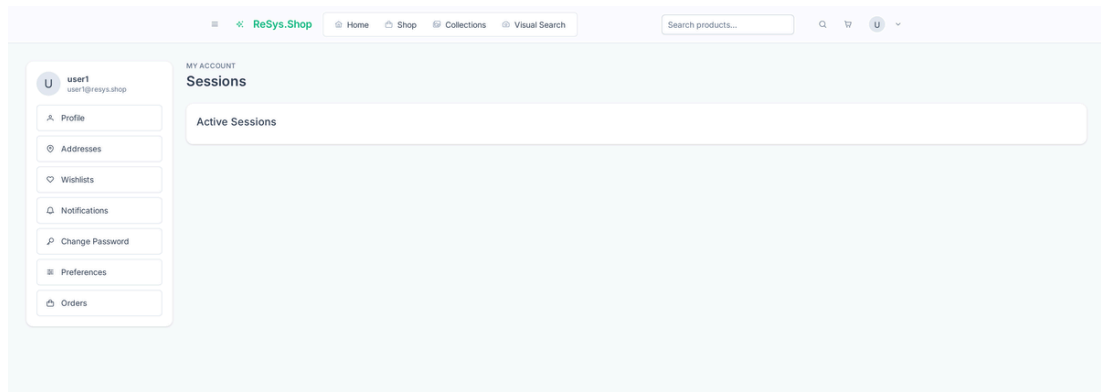


Figure 68: Session management: active sessions with device, IP, last activity; logout-all.

2.4.5.2.7 Payment Processing: UC-STR-PAY

During checkout, the payment step presents methods from the payment-methods API. Selecting a method triggers payment intent creation. For Stripe, the Stripe Elements embedded UI collects card details within an iframe, isolating sensitive data from the storefront's JavaScript context.

2.4.5.2.8 Profile Management: UC-STR-PRF

Three management areas: address book with type labels and default-address toggle, named wishlists with product thumbnails and privacy settings, and notification preferences with per-channel toggles for order updates, promotions, and stock alerts (see screenshots below).

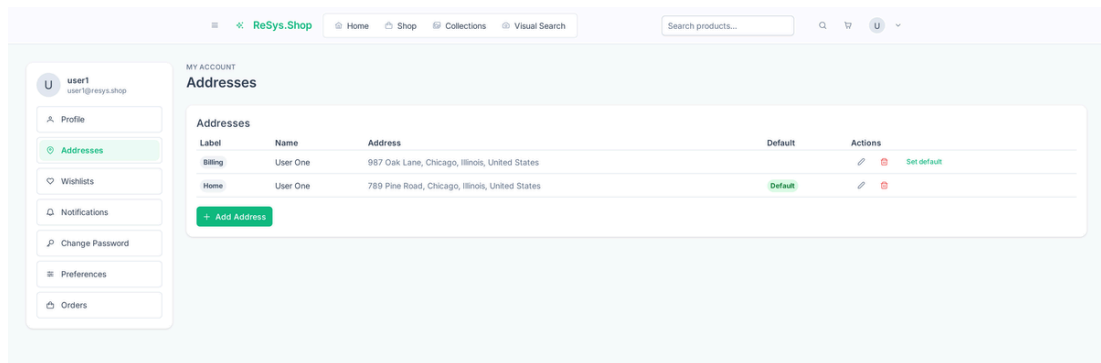


Figure 69: Address book: address cards with edit/delete and inline add-new form.

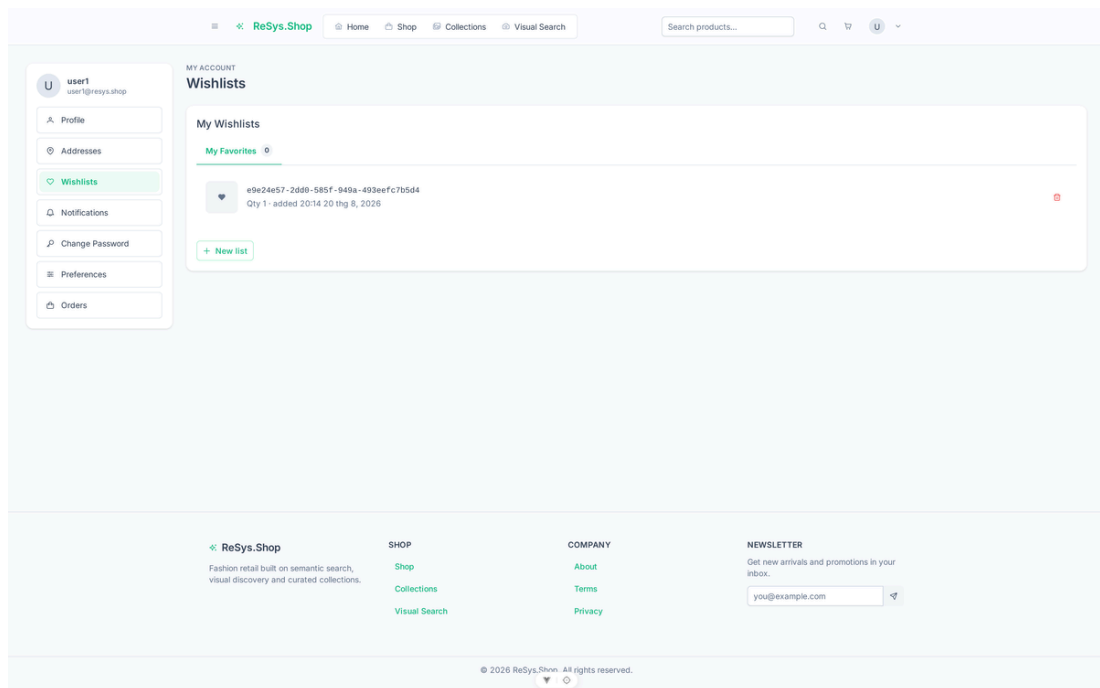


Figure 70: Wishlists: cards with item counts, privacy toggles, product thumbnails.

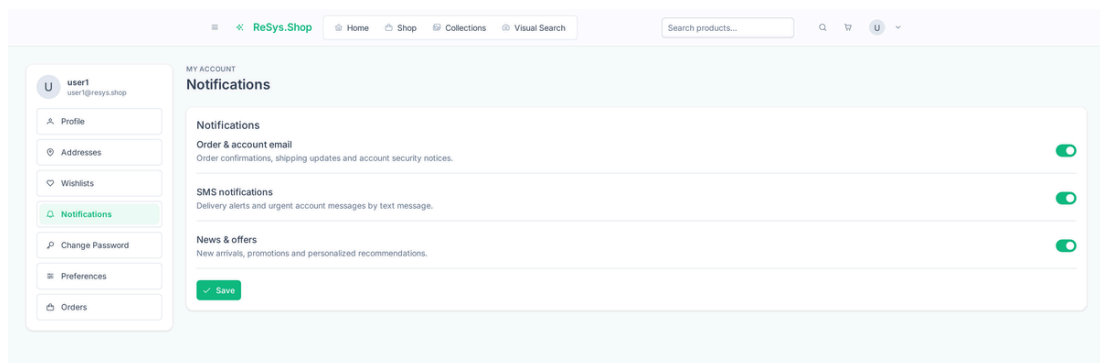


Figure 71: Notification preferences: Email/SMS toggles per notification category.

2.4.5.3 Administration Interfaces

The administration dashboard implements fifteen administrative use cases using PrimeVue data-table components with server-side pagination, sorting, filtering, and form dialogs with inline validation.

2.4.5.3.1 Product Management: UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG, UC-ADM-TAX, UC-ADM-OPT

The product management module is the most data-intensive admin area, organized into five interlinked management surfaces.

Product list (UC-ADM-PROD). Paginated data table with thumbnail, name, SKU, category, status badge (Draft/Active/Archived), and price. Toolbar:

keyword search, category/status filters, New Product button. Bulk actions support status and category changes (see screenshot below).

	Name	Slug	Status	Department	Season	Variants	Created
☐	109F Blue A-Line Dress	109f-blue-a-line-dress	Active	Apparel	Summer	3	Aug 20, 2025, 08:02 PM
☐	109F Women Multi Coloured Dress	109f-women-multi-coloured-dress	Active	Apparel	Summer	4	Aug 20, 2025, 08:02 PM
☐	109F Women Red & White Striped Tunic	109f-women-red-and-white-striped-tunic	Active	Apparel	Summer	4	Aug 20, 2025, 08:02 PM
☐	4711 Kölnisch Wasser Men Perfume	4711-kolnisch-wasser-men-perfume	Active	Personal Care	Spring	1	Aug 20, 2025, 08:02 PM
☐	ADIDAS Kids Adispeed Black Sports Shoes	adidas-kids-adispeed-black-sports-shoes	Active	Footwear	Fall	6	Aug 20, 2025, 08:02 PM
☐	ADIDAS Kids Boys Duramo Synthetic Black Sports Shoes	adidas-kids-boys-duramo-synthetic-black-sports-shoes	Active	Footwear	Fall	9	Aug 20, 2025, 08:02 PM
☐	ADIDAS Men Adi Cells Black Flip Flops	adidas-men-adi-cells-black-flip-flops	Active	Footwear	Summer	5	Aug 20, 2025, 08:02 PM
☐	ADIDAS Men Bayern Munchen Red Jersey	adidas-men-bayern-munchen-red-jersey	Active	Apparel	Summer	5	Aug 20, 2025, 08:02 PM
☐	ADIDAS Men Black Anzo Breeze Casual Shoes	adidas-men-black-anzo-breeze-casual-shoes	Active	Footwear	Fall	5	Aug 20, 2025, 08:02 PM

Figure 72: Admin product listing: paginated table with SKU, status, VND price, toolbar.

Product form (UC-ADM-PROD). Multi-section page: Basic Info (name, auto-generated slug, description, status), Fashion Metadata (style code, season, material, department, gender), SEO (meta title, description). Option Type assignment uses a source/target PickList dual-transfer control (see screenshot below).

Figure 73: Product creation form: basic info, fashion metadata, SEO, option assignment.

Variant management (UC-ADM-VAR). Embedded table: SKU, barcode, master-variant radio, option combination, inventory toggle, price, dimensions. Add Variant modal with option-value selectors, SKU auto-generation, and time-bound pricing (see screenshots below).

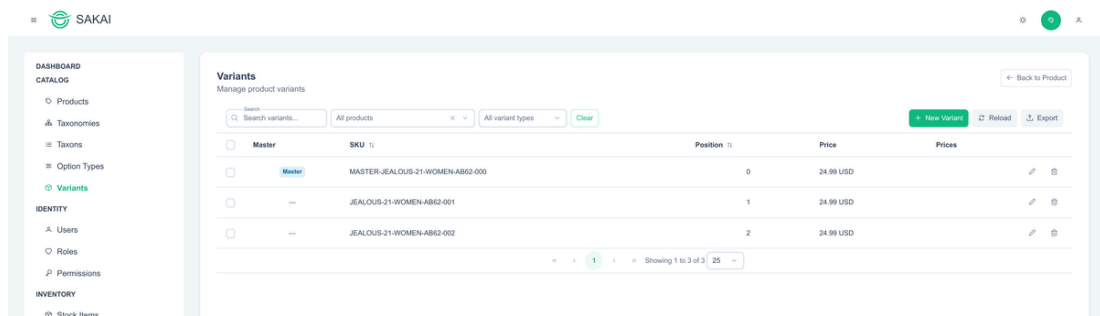


Figure 74: Variant management: Master radio, SKU, options, inventory toggle, pricing.

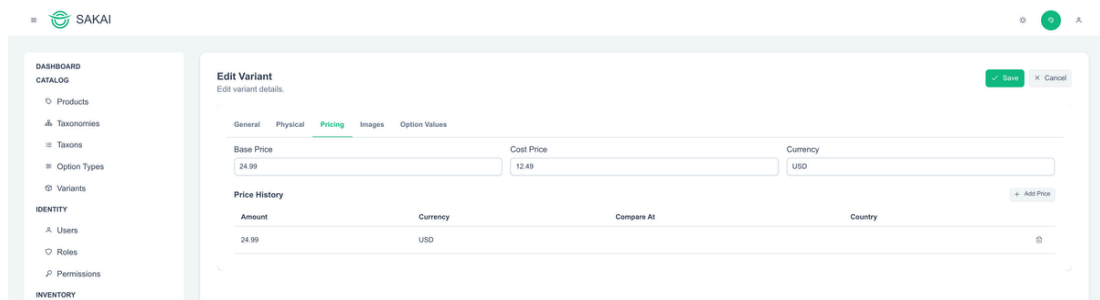


Figure 75: Variant pricing tab: Current/Sale/Scheduled history with inline add form.

Image and embedding management (UC-ADM-IMG). Sortable image grid with drag handles, thumbnails, alt-text input, and delete action. Each image shows embedding status: green checkmark with model name for processed, yellow spinner for pending, red exclamation with Regenerate button for failed. “Regenerate All Embeddings” triggers re-embedding with active model (see screenshot below).

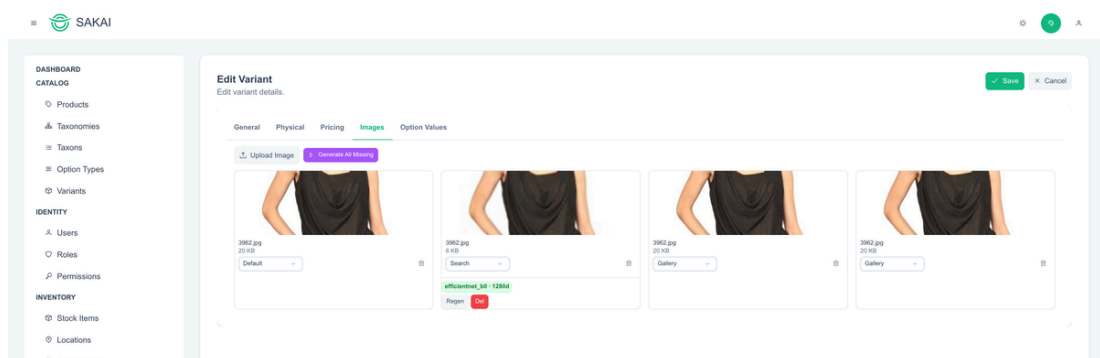


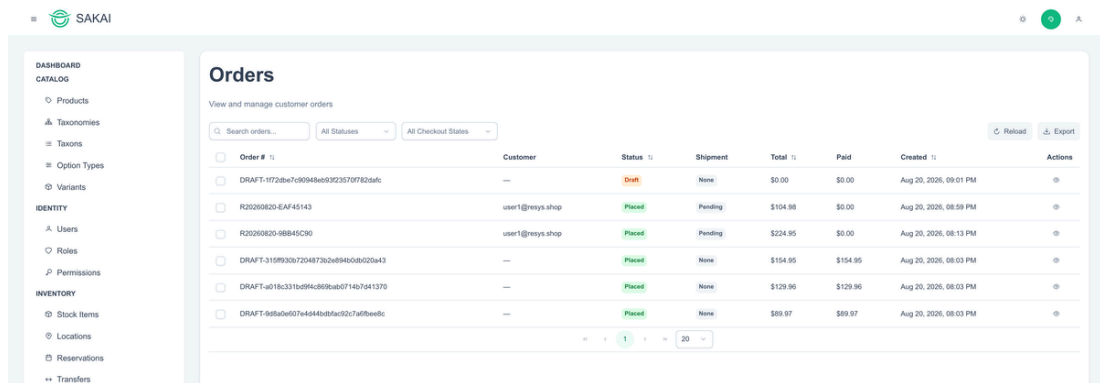
Figure 76: Images tab: sortable grid with Fashion-CLIP embedding status and regeneration.

Taxonomy and option types (UC-ADM-TAX, UC-ADM-OPT). Tree editor with left panel showing expandable taxonomy, drag-and-drop reordering, con-

text-menu actions. Right panel shows selected taxon details. Option types table lists Size/Colour/Material with ordered values and drag handles.

2.4.5.3.2 Order Management: UC-ADM-ORD, UC-ADM-ORD-ITEMS

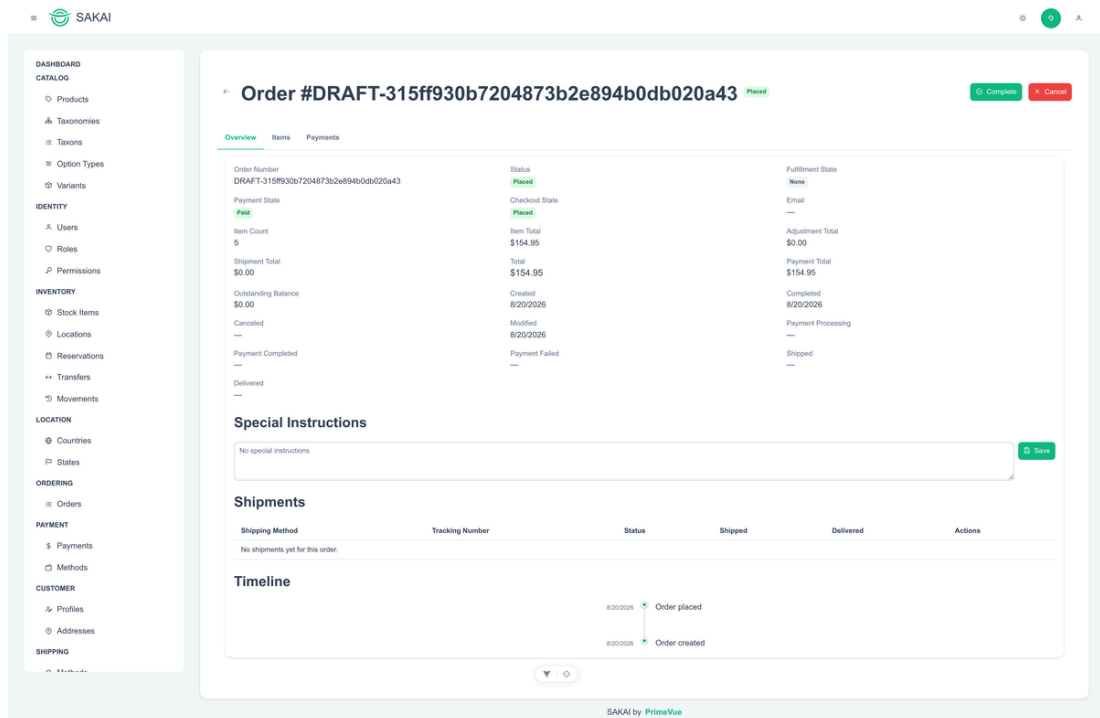
Filterable, paginated order grid with order number, customer name (linked), checkout state (colour-coded badge), payment status, shipment status, total, creation date. Filters: status multi-select, date range, payment method, keyword search (see screenshots below). Order detail page shows state transition timeline, line items table with thumbnails and variant details, address blocks, payment section with transaction ID and capture/refund/void buttons, shipment tracking. State transition buttons appear conditionally per checkout state.



The screenshot shows the 'Orders' management page in the Sakai application. It features a sidebar with navigation links for Dashboard, Catalog, Identity, Inventory, Location, Ordering, and Payment. The main content area displays a table of orders with columns for Order #, Customer, Status, Shipment, Total, Paid, Created, and Actions. The table is filtered by 'All Statuses' and 'All Checkout States'. The orders are listed in a grid, with each row representing an order and its details.

Order #	Customer	Status	Shipment	Total	Paid	Created	Actions
DRAFT-1728ba7c0948a6923570782a6c	---	Draft	None	\$0.00	\$0.00	Aug 20, 2026, 09:01 PM	ⓘ
R2020620-EAF45143	user1@reysa.shop	Placed	Pending	\$104.95	\$0.00	Aug 20, 2026, 08:59 PM	ⓘ
R2020620-88B45C90	user1@reysa.shop	Placed	Pending	\$224.95	\$0.00	Aug 20, 2026, 08:13 PM	ⓘ
DRAFT-315f930b7204873b2e894b0db020a43	---	Placed	None	\$154.95	\$154.95	Aug 20, 2026, 08:03 PM	ⓘ
DRAFT-a613c331d994c889a6071467441379	---	Placed	None	\$129.95	\$129.95	Aug 20, 2026, 08:03 PM	ⓘ
DRAFT-8d8a6b67e4844dbda932c7afbf6e6c	---	Placed	None	\$89.97	\$89.97	Aug 20, 2026, 08:03 PM	ⓘ

Figure 77: Order management: table with checkout/payment/shipment statuses and filters.



The screenshot shows the 'Order #DRAFT-315f930b7204873b2e894b0db020a43' detail page. It features a sidebar with navigation links for Dashboard, Catalog, Identity, Inventory, Location, Ordering, and Payment. The main content area displays the order details, including the order number, status (Placed), and fulfillment state (None). It also shows a timeline of events, a table of line items, and a payment section with buttons for 'Complete' and 'Cancel'.

Order Number	Status	Fulfillment State
DRAFT-315f930b7204873b2e894b0db020a43	Placed	None

Payment State	Checkout State	Email
Placed	Placed	---

Item Count	Item Total	Adjustment Total
5	\$154.95	\$0.00

Shipment Total	Total	Payment Total
\$0.00	\$154.95	\$154.95

Outstanding Balance	Created	Completed
\$0.00	8/20/2026	8/20/2026

Cancelled	Modified	Payment Processing
---	8/20/2026	---

Payment Completed	Payment Failed	Shipped
---	---	---

Delivered

Special Instructions

No special instructions

Shipments

Shipping Method	Tracking Number	Status	Shipped	Delivered	Actions
No shipments yet for this order.					

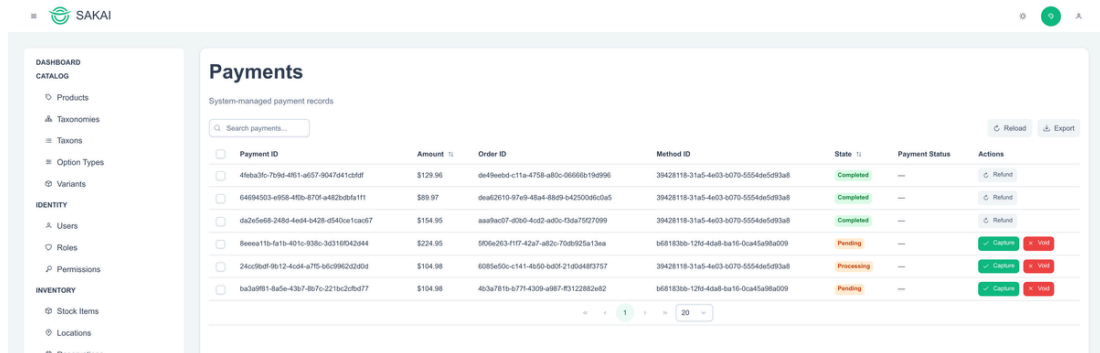
Timeline

- 8/20/2026 Order placed
- 8/20/2026 Order created

Figure 78: Order detail: status header, state timeline, line items, payment actions.

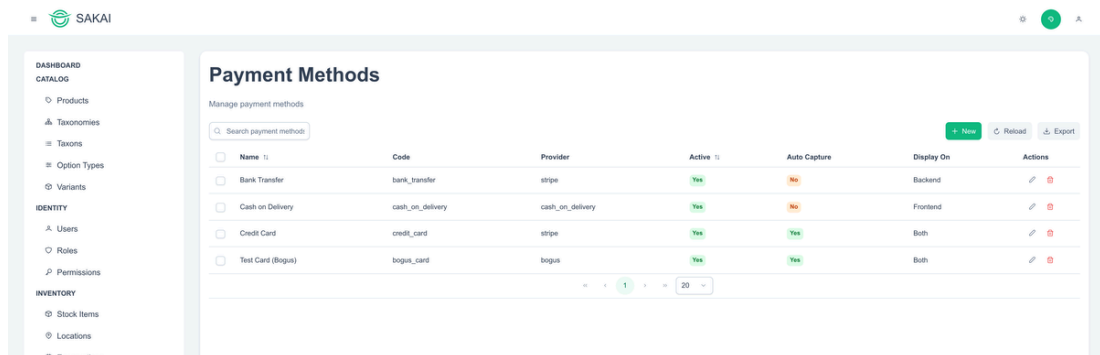
2.4.5.3.3 Payment Management: UC-ADM-PAY, UC-ADM-PAY-METHOD

Payment detail panel shows payment intent ID, gateway provider, transaction ID, amount, currency, state badge with transition timeline, and capture/refund/void action bar (conditionally enabled). Payment log records gateway interactions; webhook event log records Stripe-triggered changes (see screenshots below). Payment method management table lists configured gateways with active toggle and supported currencies.



Payment ID	Amount	Order ID	Method ID	State	Payment Status	Actions
48ba3fc-7b64-4851-a057-9047041c2d5f	\$129.96	de4f9ebcd-c11a-4758-a83c-96666219d996	39428118-31a5-4a03-b070-5554a6d93ba8	Completed	---	Refund
64694503-a956-4f0b-670f-a482b0bf111	\$89.97	dea22010-97e9-48a4-88d9-b4220069c0a5	39428118-31a5-4a03-b070-5554a6d93ba8	Completed	---	Refund
da2b5e68-2486-4e64-b428-d540e1cad7	\$154.95	aaa9ac07-d0a0-4c02-ad0c-03da7927099	39428118-31a5-4a03-b070-5554a6d93ba8	Completed	---	Refund
8eeea11b-fa10-401c-930c-3d318042944	\$224.95	5f06a203-f177-42a7-a82c-7b0b021a13ea	b691830b-125d-4a0b-ba10-dca43a8ba009	Pending	---	Capture, Void
24c0c0df-9b12-4c04-a7f5-b6c992d2d0a	\$104.98	6065a50c-c141-4b50-b05f-21d04893797	39428118-31a5-4a03-b070-5554a6d93ba8	Processing	---	Capture, Void
ba3a981-6a5e-43e7-bb7c-221bc2b0d77	\$104.98	403a781b-b771-4309-a987-8312282ba82	b691830b-125d-4a0b-ba10-dca43a8ba009	Pending	---	Capture, Void

Figure 79: Payment detail: Payment Intent ID, gateway, timeline, refund/void actions.



Name	Code	Provider	Active	Auto Capture	Display On	Actions
Bank Transfer	bank_transfer	stripe	Yes	No	Backend	✎ ✖
Cash on Delivery	cash_on_delivery	cash_on_delivery	Yes	No	Frontend	✎ ✖
Credit Card	credit_card	stripe	Yes	Yes	Both	✎ ✖
Test Card (Bogus)	bogus_card	bogus	Yes	Yes	Both	✎ ✖

Figure 80: Payment methods: gateways with active toggles and supported currencies.

2.4.5.3.4 Dashboard

The admin dashboard summarises key operational metrics for orders, revenue, inventory, and recent activity, with navigation into each management surface (see screenshot below).

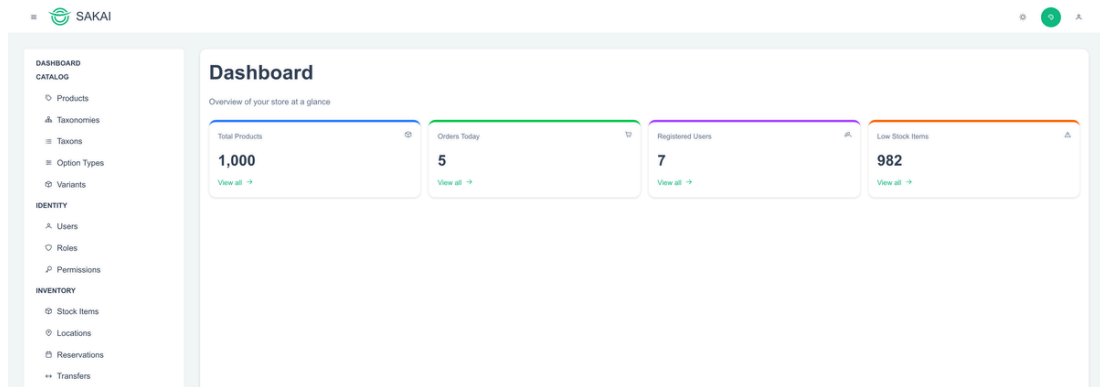


Figure 81: Admin dashboard: order/revenue/inventory summary cards.

2.4.5.3.5 Inventory Management: UC-ADM-STK, UC-ADM-LOC

Stock list. Table grouped by product showing each variant with SKU, size/colour, on-hand, reserved, available (computed), and low-stock indicator (red badge when below threshold). Filterable by location, category, and low-stock-only (see screenshot below).

The screenshot shows the 'Stock Items' table with the subtitle 'Manage inventory stock items'. It includes a search bar and a dropdown for 'All locations'. The table has columns for 'Stock Location ID', 'Variant ID', 'Count On Hand', 'Backorderable', and 'Actions'. The 'Actions' column contains icons for edit, delete, and a red badge indicating low stock.

Stock Location ID	Variant ID	Count On Hand	Backorderable	Actions
903b34a5-6580-5748-9803-91a91a053b86	75c2e6d2-cd29-5a51-9094-49021ae3ea28	48	Yes	
904e0def-c494-5d84-9cb5-5144fbc45216	aa1058a6-f846-565a-a46c-039b5ea42b9e	42	Yes	
903b34a5-6580-5748-9803-91a91a053b86	cd4188c-4a02-5885-80a5-c8ba89440f	31	Yes	
904e0def-c494-5d84-9cb5-5144fbc45216	dbef7805-2a80-5bcb-84bf-6737010632d0	38	Yes	
903b34a5-6580-5748-9803-91a91a053b86	a9611b2c-4206-53a5-a255-a53e73e15cab	4	Yes	
904e0def-c494-5d84-9cb5-5144fbc45216	965d1c10-33a6-52c2-ba48-378a91c30784	21	Yes	
903b34a5-6580-5748-9803-91a91a053b86	2969e90-8233-5185-992a-aa188539098	31	Yes	
903b34a5-6580-5748-9803-91a91a053b86	5719d243-87a8-5e4a-9901-dbc1a91ada30	1	Yes	
904e0def-c494-5d84-9cb5-5144fbc45216	c106d12b-5139-518f-a843-ab0892a7f19d	45	Yes	
903b34a5-6580-5748-9803-91a91a053b86	029d3c1-4990-5d29-e8455c2a8c11	1	Yes	
903b34a5-6580-5748-9803-91a91a053b86	d54b882-07d8-539e-adaf-365277aee04	19	Yes	
904e0def-c494-5d84-9cb5-5144fbc45216	44aee07d8-44a5-549c-a15c-ad075a6d1a0078	7	Yes	

Figure 82: Inventory stock: On Hand/Reserved/Available per variant, low-stock badge.

Stock movements. Append-only audit log: timestamp, product/variant, movement type (Receiving, Selling, Returning, Transferring), quantity delta, before/after quantities, reason, operator. Paginated and filterable (see screenshot below).

Stock Movements

Audit log of stock quantity changes

Search by reason...

Reload Export

Stock Item ID	Action	Quantity	Previous Count	Originator	Reason	Created
e8a10583-030e-4a24-a8eb-6120302990c8	restock	+109	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
7bda0d5f-9172-42df-b4a9-6853b1636313	restock	+23	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
a945d587-6103-49a5-6696-105c055a8f	restock	+27	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
94e293c3-7b8a-42d5-6593-618ae739c38b	restock	+152	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
b597785f-e66a-4136-a00a-47a7af548894	restock	+46	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
5598b6e6-b424-4521-a29d-8f8d8b434e8	restock	+125	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
f21147f1-5a0e-473e-bc04-9a105170033	restock	+56	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
672a208f-d68b-49d0-89b6-84317f56e9f	restock	+6	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
2e470106-d666-4d85-602d-6598b8d2ee5	restock	+34	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
4c9930af-4805-4224-8998-c06221519001	restock	+47	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
12272db2-2351-4de5-8737-43a1a320604b	restock	+104	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
c0ff86c5-c89b-492b-b3ef-ebc0f9fae8b9	restock	+43	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
3aadb86b-8592-44e9-94b9-c0c0718a03c	restock	+1	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
f020b29-c703-4fee-b1a9-a8205c3e4c3	restock	+10	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
9ed9533c-42d0-4d34-8f82-08169ba4651	restock	+38	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
e25afac6-7616-49f4-8593-47f626077763	restock	+50	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
d087b246-cb6b-402d-b083-c89d1610b28	restock	+134	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
4e1a2771-7409-4821-bc92-43a11475711	restock	+1	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
8149606a-980e-480b-b0c8-79b8b078851d	restock	+1	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM
e10580ea-7146-401f-b30b-c747a3b3bab	restock	+184	0	Adjustment	Initial stock seeding	Aug 20, 2026, 08:03 PM

Figure 83: Stock movements: audit log with +/- quantity deltas, reasons, and filters.

Stock locations. Management table with location name, address, active toggle, item count. Restock form: product/variant selection, quantity, unit cost, reason. Transfer form: source/destination locations, variant, quantity; lifecycle progresses Created to In-Transit to Received (see screenshots below).

Stock Locations

Manage stock locations

Search stock locations...

New Location Reload Export

Name	Code	City	Active	Default	Actions
Express Hub	EXPRESS	Los Angeles	Yes	No	
East Distribution	EAST	Atlanta	Yes	No	
Main Warehouse	MAIN	New York	Yes	Yes	

Figure 84: Stock locations: table with name, address, active badge, item count.

Edit Stock Location

Edit the details of the stock location.

Save Cancel

Name *

Express Hub

Code

EXPRESS

City

Los Angeles

Postal Code

90001

Phone

+13105550100

Position *

2

Active

☒

Figure 85: Restock dialog: variant, quantity, unit cost, reason, and notes.

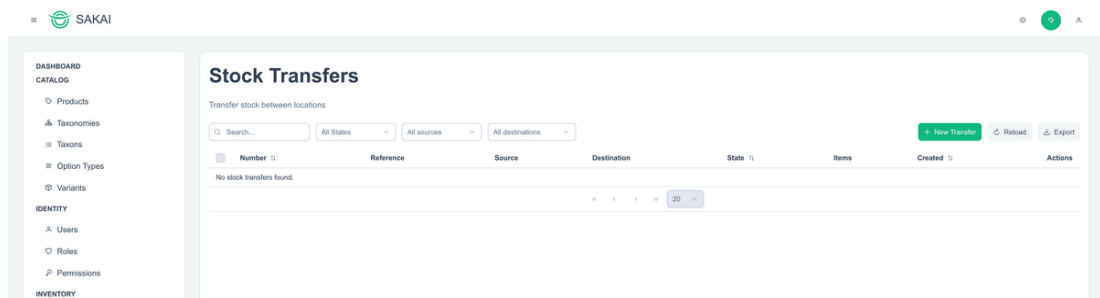


Figure 86: Transfer dialog: source/destination locations, variant, quantity.

2.4.5.3.6 User and Role Administration: UC-ADM-USR, UC-ADM-ROL

User management. Paginated table: avatar, name, email, registration date, enabled toggle, roles (compact badges), last login. Filters: status, role, keyword search. Create/edit form: full name, email, password (create only), enabled toggle, role multi-select (see screenshots below).

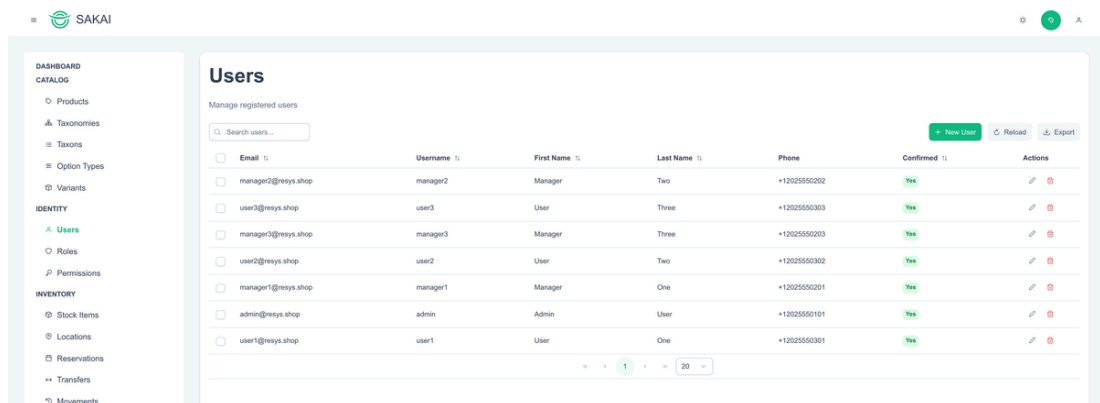


Figure 87: User list with status toggle and roles; search and filter toolbar.

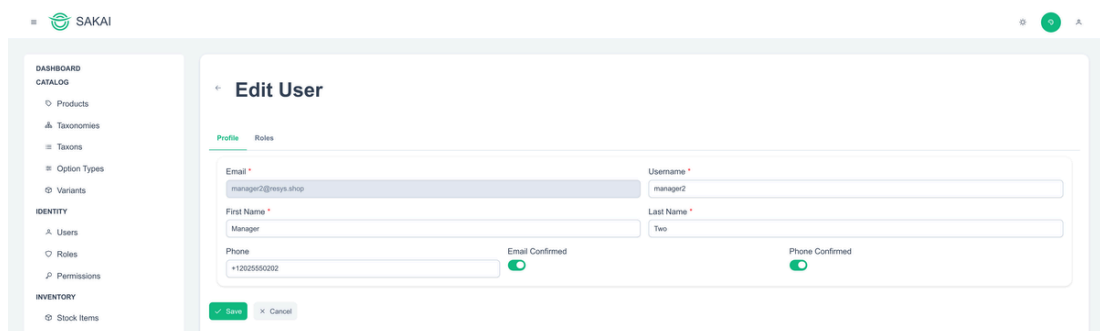


Figure 88: User edit form: name, email, enabled toggle, role checkboxes.

Role management. Role table: name, description, user count, creation date. Expanding a role shows permission assignments in an expandable tree grouped

by domain, each with *domain.category.resource.action* checkbox toggles (see screenshots below).

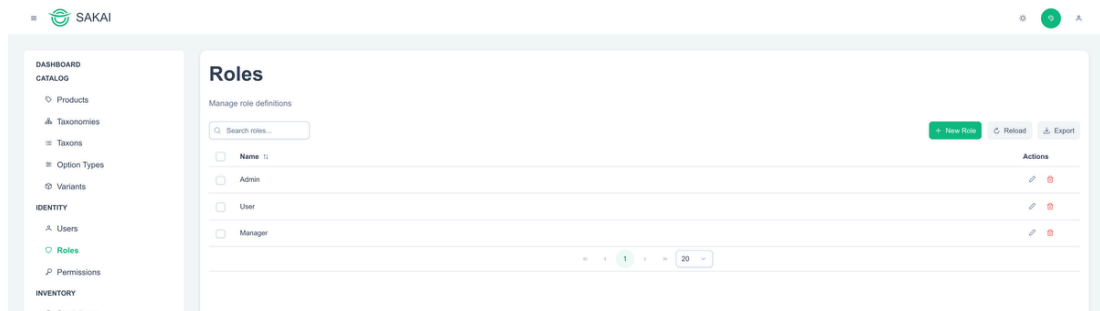


Figure 89: Roles: table with name, description, user count badge, Edit/Delete.

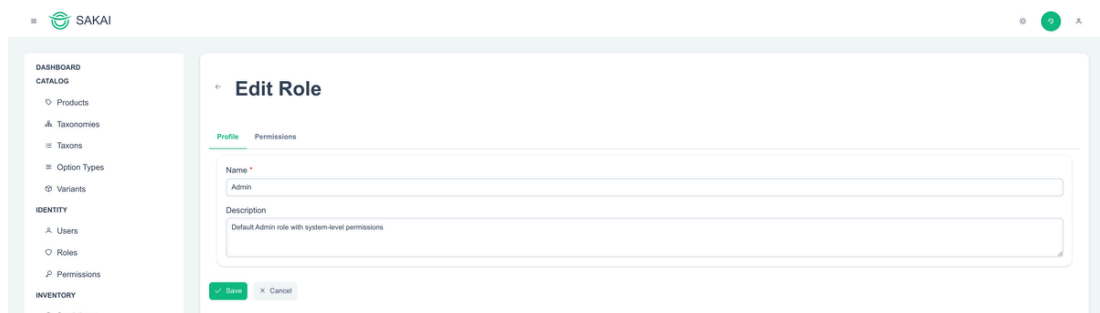


Figure 90: Role editor: permissions tree with domain.category.resource.action toggles.

2.4.5.3.7 Shipping Configuration: UC-ADM-SHP

Shipping methods table: carrier name, delivery estimate, active toggle, rates count. Each method has a rates table per geographic zone and weight/value tier. Add-rate dialog: zone dropdown, weight range, value range, rate amount (see screenshots below).

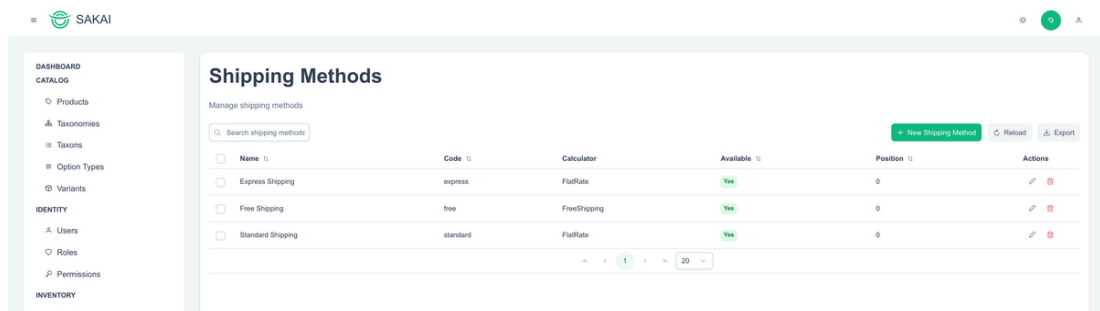


Figure 91: Shipping methods: carrier, delivery estimate, active toggle, rates count.

Name	Shipping Method ID	Cost	Final Price	Selected	Delivery Range	Actions
Express	eeed5a69-8554-4686-bbd1-3437242b141d	\$14.99	\$14.99	No	2-3 business days	Edit Delete
Free Shipping	8037c2e6-444e-4366-99fb-3d1eafcaad8f	\$9.99	\$9.99	No	7-14 business days	Edit Delete
Standard	681e5eb3-e6df-4a2b-8af5-d589f518cd2	\$5.99	\$5.99	No	5-7 business days	Edit Delete

Figure 92: Rates: table with zone, weight range, value range, rate, active toggle.

2.4.5.3.8 System Processes: UC-SYS-EMB, UC-SYS-MNT

Background automation is monitored through the Hangfire dashboard accessible at `/jobs`. The overview displays total succeeded/failed jobs, recurring schedules, and queue depths. Recurring jobs: cart expiry (hourly, releases inventory for carts inactive 7 days), reservation expiry (every minute: releases holds exceeding the 15-minute timeout); failed embedding generations retry up to 3 times; payment webhook processing is triggered by Stripe events (validates HMAC, checks idempotency, updates state). The monitoring interface is illustrated below.

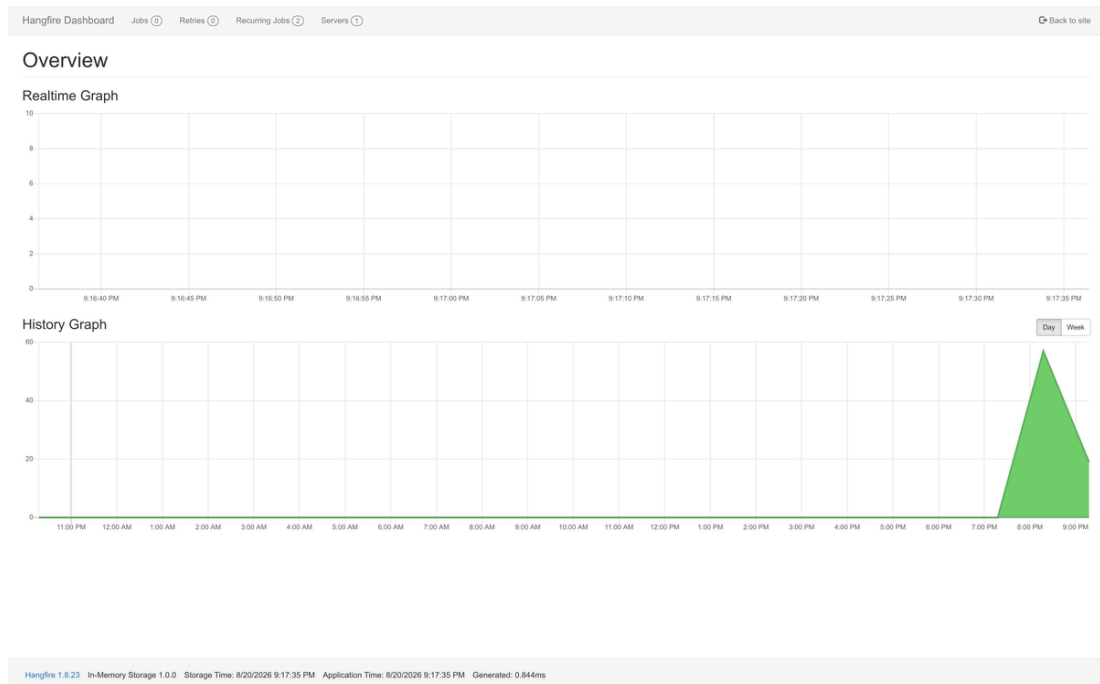


Figure 93: Hangfire dashboard overview: job metrics with realtime and history graphs.

Hangfire Dashboard

Jobs

Retries

Recurring Jobs

Servers

Back to site

Enqueued

0/1

Scheduled

0

Processing

0

Succeeded

76

Failed

0

Deleted

0

Awaiting

0

Queues

Queue	Length	Fetches	Next jobs
default	0	N/A	No jobs queued.

Figure 94: Queues: queue depths, next jobs, and per-state job counts.

Hangfire DashboardJobs ①Retries ①Recurring Jobs ②Servers ①⌕ Back to site

Enqueued ⑥/11Scheduled ①Processing ①Succeeded 76Failed ①Deleted ①Awaiting ①

ReservationExpiryJob.RunAsync

The job is finished. It will be removed automatically in 2 hours.

```
// Id: #56
using Module.Inventory.Backgrounds;

var reservationExpiryJob = ActivateReservationExpiryJob();
await reservationExpiryJob.RunAsync(null);
```

Parameters

RecurringJobId	*reservation-expiry*
Time	1787234282
CurrentCulture	*en-US*

State

Succeeded20 minutes ago (+1ms)

Latency: 1ms
Duration: 1ms
Result: 0

Processing<1ms
Server: GENERAL-WORKER-SERVER:301576
Worker: 82ad8304

Enqueued<1ms
Triggered by recurring job scheduler

Created20 minutes ago

Hangfire 1.8.23 In-Memory Storage 1.0.0 Storage Time: 8/20/2026 9:17:41 PM Application Time: 8/20/2026 9:17:41 PM Generated: 0.847ms

Figure 95: Job detail: method, parameters table, state history timeline.

3 TESTING AND ASSESSMENT

This chapter verifies the platform against the requirements of Section 2.1 and assesses the six selected embedding models, answering the three research questions of Chapter 1. Sections 3.1–3.4 cover functional software testing, and Sections 3.5–3.8 present the benchmark analysis on the 5,000-image Fashion Product Images Dataset [7] with 3-fold cross-validation.

- **Goal of Testing.** Objectives for verifying the .NET backend and Python ML sidecar against system requirements.
- **Testing Strategy.** Unit, integration, and end-to-end testing layers.
- **Scenario of Testing.** Testing environment and hardware specifications.
- **Result of Testing.** Functional test cases across visual search, ML embedding, cart/checkout, and admin management.
- **Benchmark Protocol and Experimental Setup.** Dataset, models, metrics, cross-validation methodology, hardware environment.
- **Retrieval Performance and Accuracy.** Aggregate accuracy metrics (mAP, P@K, R@K) and RQ1 answer.
- **Computational Efficiency and Resource Trade-offs.** Latency, throughput, storage, RAM trade-offs and RQ2 answer.
- **Synthesis, Deployment Strategy, and Limitations.** Accuracy-efficiency comparison, deployment recommendations, limitations, and RQ3 answer.

3.1 GOAL OF TESTING

The goal of testing is to verify that the platform functions correctly against its system requirements, with emphasis on the core research components: visual search (CBIR), the ML embedding pipeline, and multi-model benchmarking.

Table 44: Testing objectives focused on core research features.

Testing Type	Objectives for Core Research Features
Functional	• Visual search returns ranked product results with similarity scores. • ML sidecar generates embeddings of correct dimensionality per model. • Shopping cart supports add, update, remove, and guest-to-user merge. • Checkout enforces forward-only state transitions. • Admin CRUD for products, variants, images, and taxonomies.
API Integration	• .NET backend dispatches image bytes to Python sidecar and receives vectors. • pgvector queries return results filtered by model_name. • Error responses conform to RFC 7807 Problem Details. • Sidecar health endpoint reports correct status for container orchestration.

Testing Type	Objectives for Core Research Features
Database	• Product, variant, and embedding records are persisted atomically. • Inventory reservations prevent overselling under concurrent checkout. • Soft-deletion preserves referential integrity.
Security	• JWT tokens expire after 15 minutes. • Endpoints reject requests with insufficient permission claims. • Upload validation rejects mismatched magic bytes.

3.2 TESTING STRATEGY

The ReSys.Shop platform was subjected to a multi-level testing strategy covering three distinct layers: unit tests for isolated logic, integration tests for component interactions, and end-to-end tests for critical user workflows. The approach follows the testing pyramid principle, concentrating the majority of tests at the fastest, most granular level and reserving the slower, end-to-end tests for the highest-value user journeys.

3.2.1 Unit Testing

Unit tests form the foundation of the verification strategy:

1. The .NET backend uses xUnit v3 to test handler logic, domain invariants, and validation rules in isolation.
2. Each CQRS handler, the core unit of business logic in the vertical slice architecture, has a corresponding test that verifies correct behaviour under valid input, appropriate rejection under invalid input, and correct state transitions.
3. Domain invariants such as the order state machine transitions and the inventory non-negative stock constraint are enforced through tests that execute without any external dependencies.
4. The Python machine learning sidecar uses pytest to validate the embedding generation pipeline, ensuring each supported model produces vectors of the expected dimensionality and that the preprocessing pipeline normalises input images to model-expected formats.

3.2.2 Integration Testing

Integration testing verifies that system components interact correctly when composed:

1. Testcontainers provisions ephemeral PostgreSQL and Redis instances for each test run, ensuring database queries, including vector similarity search with pgvector, are tested against real infrastructure.

2. Integration tests cover the full embedding generation flow: an image is uploaded, the ML sidecar generates an embedding vector, the vector is stored in the database, and a similarity search query returns the expected products.
3. Cross-service communication between the .NET backend and the Python sidecar is validated, establishing that the HTTP contract between the two services is honoured.

3.2.3 End-to-End Testing

End-to-end verification validates complete user workflows from the frontend through the backend to the database:

1. The key user flows (visual search, checkout, and admin product management) were verified manually using documented HTTP test files that simulate the sequence of API calls a frontend client makes during a real user session.
2. Automated end-to-end testing via Playwright covers the most critical paths: the visual search flow from image upload to results display, and the checkout flow from cart addition to order confirmation.
3. These tests run against a fully deployed containerized environment, giving confidence that the system operates correctly when all services are composed.

3.3 SCENARIO OF TESTING

The testing scenarios cover the four functional areas identified in Section 3.1: visual search (7 scenarios), ML embedding pipeline (6 scenarios), shopping cart and checkout (8 scenarios), and admin product management (7 scenarios). Each scenario is validated with step-by-step test cases defined in Section 3.4.

3.3.1 Testing Environment

- **Hardware.** Intel Core i7-1165G7 (4 cores, 2.80 GHz), 16 GB DDR4 RAM, 512 GB NVMe SSD.
- **Database.** PostgreSQL 17 with pgvector 0.7.0, provisioned via Testcontainers for integration tests.
- **Cache.** Redis 7 (Alpine) for HybridCache L2 storage and Hangfire job persistence.
- **ML Sidecar.** Python 3.12 FastAPI service with PyTorch 2.13.0, CPU-only inference.
- **Web Browsers.** Google Chrome 131, Firefox 135.

3.4 RESULT OF TESTING

3.4.1 Visual Search (CBIR) – UC-STR-SRC

Table 45: Visual search test cases: normal flow, edge cases, and fault recovery.

No	Description	Step	Expected Result	Result
1	Upload query image	Drop JPEG on drop zone; click “Search Similar Products”.	Grid with thumbnails and colour badges (green $\geq 90\%$, amber $\geq 80\%$).	Pass
2	Loading state	Upload image; observe during API request.	Skeleton grid of animated placeholder cards displayed.	Pass
3	No results	Upload image with no visually similar products.	“No similar products found” with “Try Again” button.	Pass
4	Invalid file type	Attempt to upload a PDF on the search page.	Rejected client-side with unsupported format message.	Pass
5	File exceeds 10 MB	Attempt to upload an image over 10 MB.	Rejected with size-limit warning before network request.	Pass
6	Sidecar unavailable	Stop ML sidecar; upload image and search.	<i>HTTP 503</i> catalogue and cart endpoints remain functional.	Pass
7	Sidecar recovery	Restart sidecar; wait for health check; search.	Health probe OK; search returns correct results.	Pass

3.4.2 ML Embedding Pipeline

Table 46: ML embedding pipeline test cases across four model architectures.

No	Description	Step	Expected Result	Result
8	Fashion-CLIP	<i>POST</i> /embeddings/bytes with fashion image.	512-dim vector; model “fashion_clip”; inference_ms present.	Pass
9	EfficientNet-B0	Set model to EfficientNet-B0; <i>POST</i> request.	1280-dim vector; model “efficientnet_b0”.	Pass
10	ResNet-50	Set model to ResNet-50; <i>POST</i> request.	2048-dim vector; model “resnet50”.	Pass
11	CLIP-generic	Set model to CLIP ViT-B/16; <i>POST</i> request.	512-dim vector; model “clip_vit_b16”.	Pass
12	Model-name filtering	Generate with two models; search and verify.	Results filtered by model name; match active model only.	Pass
13	Invalid API key	<i>POST</i> /embeddings/bytes without API key.	<i>HTTP 401 Unauthorized</i> .	Pass

3.4.3 Shopping Cart and Checkout – UC-STR-CRT, UC-STR-CHK

Table 47: Shopping cart and checkout test cases.

No	Description	Step	Expected Result	Result
14	Add item to cart	Select variant (size, colour); click “Add to Cart”.	Cart count updates in header; toast displayed.	Pass
15	Update quantity	Open cart; change quantity from 1 to 3.	Line and cart subtotals recalculate correctly.	Pass
16	Remove item	Open cart; click remove on a line item.	Item removed; subtotal updated; items displayed.	Pass
17	Guest cart merge	Add items as guest; register and log in.	Guest cart merges; cookie invalidated; no items lost.	Pass
18	Exceed stock	Add item; increase quantity beyond available stock.	Rejected with max-available notification; capped.	Pass
19	Checkout pipeline	Complete Address, Delivery, Payment, Confirm, Complete.	Progress bar advances; order generated; inventory reserved.	Pass
20	Empty cart checkout	Navigate to checkout with empty cart.	Redirected to cart; “Your cart is empty” displayed.	Pass
21	Cancel order	Open order in pre-completion state; cancel.	Order Cancelled; inventory released; payment voided.	Pass

3.4.4 Admin Product Management – UC-ADM-PROD, UC-ADM-VAR, UC-ADM-IMG

Table 48: Admin product, variant, image, and taxonomy management test cases.

No	Description	Step	Expected Result	Result
22	Create product	Fill name, slug, fashion meta-data; click “Save”.	Product created with generated slug; notification.	Pass
23	Duplicate slug	Create product with existing slug.	Error: slug exists; product not created.	Pass
24	Add variant	Select Size M and Colour Navy; enter SKU, price.	Variant added with option combination in table.	Pass

No	Description	Step	Expected Result	Result
25	Upload images	Upload 3 images; set display order; check status.	Green checkmark with model name per image; order kept.	Pass
26	Regenerate embeddings	Switch model; click “Regenerate All Embeddings”.	Status reset to pending; then green with new model.	Pass
27	Archive product	Change status to Archived; verify storefront.	Hidden from storefront; admin shows Archived badge.	Pass
28	Taxonomy management	Create taxon under “Dresses”; drag to reorder.	New taxon with auto-slug; tree reordered; count updates.	Pass

3.4.5 Summary

All 28 test cases passed across four functional areas: visual search (7), ML embedding pipeline (6), cart and checkout (8), and admin product management (7). Error states, edge cases (empty carts, duplicate slugs, stock exhaustion), and recovery scenarios (sidecar restart) were handled correctly. These test cases validate the implemented features against their expected behaviour; they do not constitute an adversarial or negative test suite. Development-phase defects (e.g., embedding dimension mismatches during model switching, CORS configuration errors for the ML sidecar, and cart-merge race conditions) were identified and resolved during iterative testing and are not formally tracked in this table.

3.5 BENCHMARK PROTOCOL AND EXPERIMENTAL SETUP

The benchmark uses the Fashion Product Images Dataset: 5,000 catalogue images across five categories (Apparel 2,500, Accessories 1,250, Footwear 750, Personal Care 350, Sporting Goods 150). Each image carries a category label as ground truth for binary relevance. All images are preprocessed to 224×224 pixels with ImageNet normalisation (mean 0.485/0.456/0.406, std 0.229/0.224/0.225).

3.5.1 Models Benchmarked

Six models spanning four architectural families were benchmarked by the framework (selected from the eleven candidates it supports):

Table 49: Six benchmarked models representing CNN, ViT, CLIP, and domain-tuned architectures.

Architecture	Benchmarked Models (dimension)
Convolutional Neural Network	ResNet-50 (2,048-dim), EfficientNet-B0 (1,280-dim)
Vision Transformer	DINOv2 ViT-S/14 (384-dim): self-supervised ViT
CLIP-based	CLIP ViT-B/16 (512-dim): generic CLIP wrapper, CLIP ViT-B/32 (512-dim): OpenAI CLIP
Fashion-specific	Fashion-CLIP (512-dim), fine-tuned on 700,000+ fashion images

3.5.2 Performance Metrics

Three accuracy metric families (mAP, P@K, R@K), measured at three depths (K=5, 10, 20), and five efficiency metrics were measured per model.

Table 50: Performance metrics and definitions.

Metric	Definition
mAP	Query-averaged Mean Average Precision over the top-20 ranked results (average precision per query, then mean across queries); primary accuracy metric.
P@K	Fraction of top-K results sharing the query’s category.
R@K	Fraction of all relevant catalogue items appearing in top-K results.
Inference Time	Mean milliseconds per image embedding (preprocessing + forward pass).
Throughput	Images embedded per second under sustained load.
Load Time	One-time cost of loading model weights from disk into memory.
Storage	Disk space of the embedding index for the 5,000-image catalogue.
RAM	Peak main memory consumption. Reported as approximate ranges derived from each model’s parameter count plus PyTorch runtime overhead, because direct psutil measurement proved unreliable on this Linux kernel. Cost scales from 100 MB (EfficientNet-B0) to 600 MB (CLIP-based).

3.5.3 Methodology

The protocol used 3-fold stratified cross-validation preserving category distribution. Accuracy metrics (mAP, P@K, R@K) used exact cosine search over all gallery embeddings to isolate model quality from index effects. The pgvector production benchmark used IVFFlat with 100 lists (sub-10 ms query latency at 65–72% recall@10, near-instant build) for the 512-dim and 1,280-dim models; ResNet-50’s 2,048-dim embeddings were not indexed with IVFFlat in this benchmark. HNSW is designated for larger catalogue scales.

3.5.4 Statistical Analysis

This benchmark reports descriptive statistics: mean and standard deviation of mAP, P@K, and R@K across three stratified folds. With only three independent mAP values per model, formal significance testing has limited power. The non-overlapping-bounds heuristic (mean plus or minus two standard deviations) is used as a descriptive visual aid throughout Chapter 3, not as a significance test: non-overlapping bounds suggest the models may be visually separable, but do not imply a p-value or confidence interval. A paired test across queries (e.g., Wilcoxon signed-rank test on per-query Average Precision values) would provide substantially more statistical power by exploiting within-fold query-level variance, and is recommended for future work using the raw per-query AP values recorded during benchmarking.

3.5.5 Hardware Environment

Table 51: Hardware environment. All benchmarks executed on CPU without GPU acceleration.

Component	Specification
CPU	Intel Core i7-1165G7 (4 cores / 8 threads, 2.80 GHz)
RAM	16 GB DDR4
Database	PostgreSQL 17, pgvector 0.7.0

3.6 RETRIEVAL PERFORMANCE AND ACCURACY

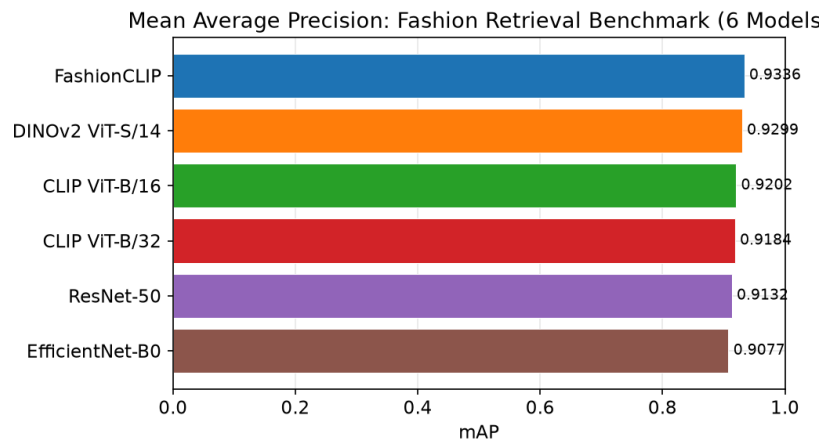


Figure 96: mAP comparison across six benchmarked models, with Fashion-CLIP leading at 0.9336.

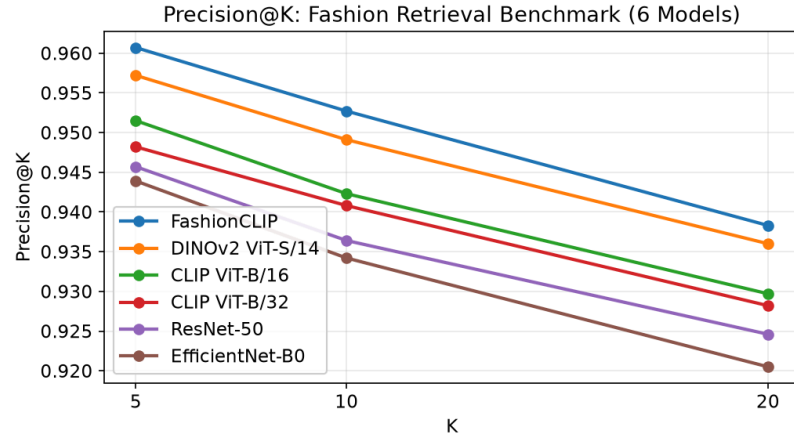


Figure 97: Precision at K = 5, 10, and 20, with Fashion-CLIP highest across all six models.

Table 52: Aggregate Retrieval Metrics, 3-Fold Cross-Validation (Category-Only Ground Truth)

Model	mAP (mean ± SD)	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	0.9336 ± 0.0060	0.9607	0.9527	0.9383	0.0282	0.0488	0.0816
DINOv2 ViT-S/14	0.9299 ± 0.0058	0.9572	0.9491	0.9360	0.0275	0.0484	0.0813
CLIP ViT-B/16	0.9202 ± 0.0043	0.9515	0.9423	0.9297	0.0275	0.0474	0.0790
CLIP ViT-B/32	0.9184 ± 0.0060	0.9482	0.9408	0.9282	0.0272	0.0474	0.0785
ResNet-50	0.9132 ± 0.0057	0.9457	0.9364	0.9246	0.0262	0.0452	0.0766
EfficientNet-B0	0.9077 ± 0.0076	0.9439	0.9342	0.9205	0.0257	0.0443	0.0748

1

¹† Recall values appear low because the category-only ground truth defines hundreds to thousands of relevant items per query (Appendix B); even a perfect model would show R@20 < 0.10 under this scheme.

Table 52 reports the full aggregate metrics.

F3.6.1 Fashion-CLIP achieved the highest retrieval accuracy across every metric (mAP 0.9336), but the six-model field is tightly clustered: the full spread from Fashion-CLIP to EfficientNet-B0 (0.9077) is only 2.86%. Fashion-CLIP’s lead over the nearest competitor, DINOv2 ViT-S/14 (0.9299), is just 0.40%, and its advantage over the generic CLIP ViT-B/16 (0.9202) is 1.46%.

F3.6.2 The three transformer-based models (Fashion-CLIP, DINOv2 ViT-S/14, CLIP ViT-B/16, CLIP ViT-B/32) form the top tier, separated from the two CNN models (ResNet-50, EfficientNet-B0) by roughly 0.5–1.5 percentage points in mAP.

With only 3 folds, formal significance testing has limited power. As a descriptive visual check (not a formal significance test), the non-overlapping-bounds heuristic (mean $\pm$ two standard deviations) shows that Fashion-CLIP’s lower mAP bound (0.9216) overlaps the upper bounds of every other model: DINOv2 (0.9415), CLIP ViT-B/16 (0.9288), CLIP ViT-B/32 (0.9304), ResNet-50 (0.9246), and EfficientNet-B0 (0.9229). No model can therefore be separated from any other at this heuristic level.

Table 53: Key Caveats for Retrieval Performance

ID	Caveat	Why it matters
C1	Top-2 model gap (0.40%) within measurement uncertainty at 3-fold CV	Do not claim the top model is definitively best
C2	Sporting Goods stratum thin (150 images, about 50 per test fold)	mAP unstable at finer label granularity
C3	Personal Care stratum thin (350 images total)	Same risk as C2 at category+colour+pattern scheme

3.6.1 Ground-Truth Sensitivity

The category-only scheme above is the broadest relevance definition and yields the highest absolute scores. To validate that the model ranking is not an artefact of this single label scheme, all six models were re-measured under two progressively stricter ground-truth definitions: **category + colour** (requiring master category and base colour agreement) and **category + colour + pattern** (additionally requiring pattern-attribute agreement). The three configurations are summarised below and reported in full in Appendix A.

Table 54: mAP Under Three Ground-Truth Definitions (6 Models, 3-Fold CV)

Model	Category-Only	Category + Colour	Cat. + Colour + Pattern
Fashion-CLIP	0.9336	0.2439	0.2071
DINOv2 ViT-S/14	0.9299	0.1899	0.1651
CLIP ViT-B/16	0.9202	0.2253	0.1861
CLIP ViT-B/32	0.9184	0.2250	0.1859
ResNet-50	0.9132	0.2028	0.1629
EfficientNet-B0	0.9077	0.2248	0.1842

Two patterns emerge. First, absolute mAP degrades monotonically as the relevance criterion tightens (category-only ≈ 0.91 – 0.93 , category + colour ≈ 0.19 – 0.25 , category + colour + pattern ≈ 0.16 – 0.21), because finer-grained relevance admits fewer correct matches per query. The model ranking is **not** invariant: under the coarse category-only scheme DINOv2 ViT-S/14 is the runner-up to Fashion-CLIP, but under category + colour it collapses to last place (0.1899), while the CLIP family (trained with textual colour and pattern tokens) and EfficientNet-B0 remain comparatively stable. This indicates that DINOv2 captures category-level semantics strongly yet encodes fashion-specific fine attributes (colour, pattern) less precisely than the CLIP-family models. Fashion-CLIP, which combines fashion-domain fine-tuning with the CLIP architecture, retains the lead under every scheme, indicating that its advantage is not an artefact of the label definition. Per-stratum reliability degrades for Sporting Goods and Personal Care, where the combination of small class size and fine-grained relevance labels produces very few relevant matches per query.

Answer to RQ1. Fashion-CLIP achieved the highest mean mAP across every accuracy metric and under all three ground-truth definitions, although its advantage over the nearest competitor, DINOv2 ViT-S/14, is small (0.40%) and, with only three folds, within measurement uncertainty. The 1.46% mAP advantage over the generic CLIP ViT-B/16 shows that domain-specific fine-tuning gives measurable retrieval quality improvements beyond general-purpose contrastive pre-training. However, the six-model comparison reveals a tightly packed top tier: DINOv2 ViT-S/14 (self-supervised) closes to within 0.40% of Fashion-CLIP on category-only retrieval, and the CLIP family appears markedly more stable than DINOv2 as the relevance criterion tightens to colour and pattern. The recommendation depends on the task: Fashion-CLIP leads on visual similarity, while DINOv2 is a strong, lighter-weight alternative for coarse category retrieval.

3.7 COMPUTATIONAL EFFICIENCY AND RESOURCE TRADE-OFFS

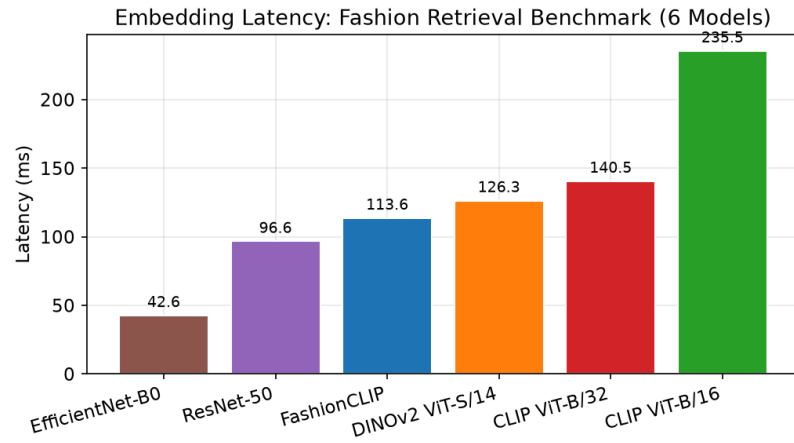


Figure 98: Inference latency across the six benchmarked models.

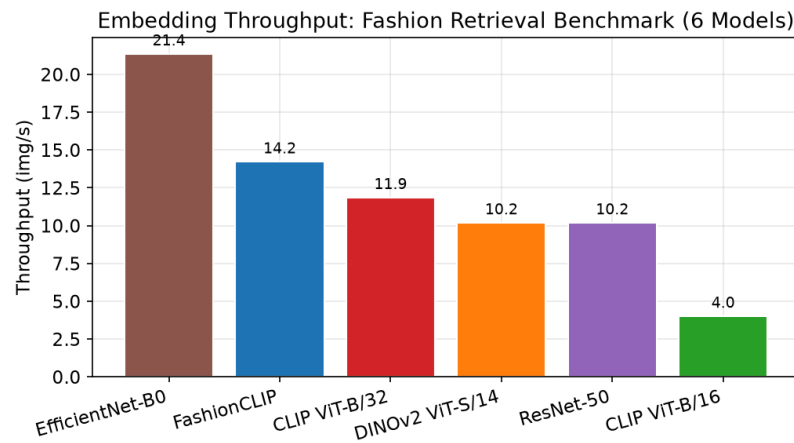


Figure 99: Throughput across the six benchmarked models.

Table 55: Efficiency Metrics, 3-Fold Cross-Validation

Model	Latency (ms)	Throughput (img/s)	Load (ms)	Storage (MB)	RAM (MB)*
EfficientNet-B0	42.6 ± 5.6	21.4 ± 1.0	118.3	8.1	100
ResNet-50	96.6 ± 7.4	10.2 ± 0.0	385.6	13.0	150
Fashion-CLIP	113.6 ± 3.7	14.2 ± 0.5	5,109.9	3.3	600
DINOv2 ViT-S/14	126.3 ± 5.1	10.2 ± 0.2	1,223.4	2.4	250
CLIP ViT-B/32	140.5 ± 7.8	11.9 ± 0.2	1,868.8	3.3	600

Model	Latency (ms)	Throughput (img/s)	Load (ms)	Storage (MB)	RAM (MB)*
CLIP ViT-B/16	235.5 $\pm$ 7.3	4.0 $\pm$ 0.1	6,518.5	3.3	600

2

Table 55 presents the full efficiency results. EfficientNet-B0 leads every efficiency metric. Its inference time of 42.6 ms is $3.0\times$ faster than DINOv2 ViT-S/14 (126.3 ms) and $5.5\times$ faster than CLIP ViT-B/16 (235.5 ms). Its throughput of 21.4 img/s is $1.5\times$ higher than Fashion-CLIP (14.2) and $2.1\times$ higher than DINOv2 and ResNet-50 (10.2). Load time of 118.3 ms is less than a third of ResNet-50 (385.6 ms) and 16–55x lower than the multi-second CLIP load times. The EfficientNet-B0 latency standard deviation (5.6 ms, CV 13%) is comparable to the other models; any higher variance observed in earlier runs reflected CPU frequency scaling on the single-test hardware and was resolved by pinning the CPU governor before the final benchmark. This profile makes EfficientNet-B0 the strongest candidate for CPU-only deployment at interactive latencies without cold-start penalties.

The CLIP-family and DINOv2 models exhibit different inference latencies driven by self-attention scaling with image patches. Fashion-CLIP (113.6 ms) and CLIP ViT-B/32 (140.5 ms) are markedly faster than CLIP ViT-B/16 (235.5 ms), a consequence of the larger ViT-B/16 patch configuration on this CPU; CLIP ViT-B/16 also has the lowest throughput (4.0 img/s). DINOv2 ViT-S/14 (126.3 ms) sits between the CLIP models. Load times of 1.2-6.5 seconds for the transformer models reflect weight initialisation cost, a one-time startup penalty, not per-request.

ResNet-50 holds an intermediate position (96.6 ms latency, 10.2 img/s throughput) and has the largest storage footprint: 13.0 MB, $1.6\times$ larger than EfficientNet-B0 (8.1 MB) and $4.0\text{--}5.4\times$ larger than the transformer models (2.4-3.3 MB). Storage scales linearly with embedding dimensionality: 384-dim DINOv2 (2.4 MB), 512-dim CLIP family (3.3 MB), 1,280-dim EfficientNet-B0 (8.1 MB), and 2,048-dim ResNet-50 (13.0 MB) for 5,000 images. At production scale with millions of items, this becomes a meaningful differentiator.

RAM values are reported as approximate ranges derived from each model’s documented parameter count plus PyTorch runtime overhead, because direct process-level measurement via psutil proved unreliable on this Linux kernel, producing negative and zero artefacts. Memory consumption scales with model size: the lightweight EfficientNet-B0 (5.3 M parameters) sits at roughly 100 MB,

²* Estimated from model parameter count plus PyTorch runtime overhead; direct measurement unreliable on the test kernel.

ResNet-50 (25.6 M) and DINOv2 ViT-S/14 (21 M) at roughly 150-250 MB, and the CLIP-family models (150 M parameters) at roughly 600 MB. These figures are indicative rather than instrumented; actual values vary with batch size and PyTorch overhead.

Answer to RQ2. The accuracy-speed trade-off is large and non-linear. EfficientNet-B0 (42.6 ms) achieves 97.2% of Fashion-CLIP’s mAP (0.9077 vs 0.9336) at 37.5% of the latency. ResNet-50 combines a middle-range mAP (0.9132) with middle-range latency (96.6 ms) and the largest storage footprint (13.0 MB). The relationship is not simply “slower equals more accurate”: CLIP ViT-B/16 has the highest latency (235.5 ms) yet ranks below the faster Fashion-CLIP, DINOv2, and CLIP ViT-B/32, which shows that architecture and patch configuration, not raw inference time, govern accuracy. Practitioners choosing between EfficientNet-B0 and Fashion-CLIP weigh a 2.86% mAP improvement against a $2.67\times$ latency increase.

3.8 SYNTHESIS, DEPLOYMENT STRATEGY, AND LIMITATIONS

3.8.1 Accuracy-Efficiency Trade-off

Table 56 presents the combined accuracy and efficiency view. Looking at accuracy and efficiency together, two distinct clusters appear: a transformer-based top tier and a CNN-based lower tier.

Table 56: Combined Accuracy and Efficiency Comparison

Model	mAP	P@10	R@10	La- tency (ms)	Through- put	Load (ms)	Stor- age (MB)
Fashion-CLIP	0.9336	0.9527	0.0488	113.6	14.2	5,109.9	3.3
DINOv2 ViT-S/14	0.9299	0.9491	0.0484	126.3	10.2	1,223.4	2.4
CLIP ViT-B/16	0.9202	0.9423	0.0474	235.5	4.0	6,518.5	3.3
CLIP ViT-B/32	0.9184	0.9408	0.0474	140.5	11.9	1,868.8	3.3
ResNet-50	0.9132	0.9364	0.0452	96.6	10.2	385.6	13.0
EfficientNet-B0	0.9077	0.9342	0.0443	42.6	21.4	118.3	8.1

Fashion-CLIP stands out in the high-accuracy range: mAP 0.9336 leads every other model, while its 113.6 ms inference time remains within the sub-second interactive budget. The four transformer-based models (Fashion-CLIP, DINOv2 ViT-S/14, CLIP ViT-B/16, CLIP ViT-B/32) form a tightly packed top tier spanning 0.9184–0.9336, separated from the two CNN models (ResNet-50 0.9132, EfficientNet-B0 0.9077) by roughly half a percentage point. EfficientNet-B0 delivers the fastest inference (42.6 ms) and highest throughput (21.4 img/s) but

the lowest mAP (0.9077). RAM values are estimated from parameter counts and should be treated as approximate (see Table 55 footnote).

3.8.2 Deployment Recommendations

Table 57: Deployment Scenario Recommendations

Scenario	Model	Reasoning
Retrieval quality, interactive search	Fashion-CLIP	Highest mAP (0.9336); 113.6 ms within sub-second budget; lazy-loading and embedding caching via pluggable architecture
CPU-only, latency-sensitive	EfficientNet-B0	42.6 ms inference; 97.2% of Fashion-CLIP's mAP at 37.5% of latency; 118.3 ms load time for fast cold-start
Coarse category retrieval, lighter-weight	DINOv2 ViT-S/14	Within 0.40% of Fashion-CLIP on category-only mAP; roughly half the load time of CLIP models

The pluggable configuration mechanism (Section 2.4) allows switching between recommendations by changing a single environment variable; embeddings tagged by model name allow multiple models to coexist.

3.8.3 Limitations

1. **Single-source dataset.** The Fashion Product Images Dataset originates from a single e-commerce platform; results may not generalise to other markets or photography conventions.
2. **Class imbalance.** Retrieval scores are dominated by the Apparel class (2,500 images, 50% of dataset); rankings on a more balanced catalogue may differ.
3. **Thin strata.** Sporting Goods (150 images, 50 per test fold) and Personal Care (350 images) produce unreliable per-stratum rankings, especially under fine-grained ground truth.
4. **Imperfect ground truth.** The binary category-label criterion is an imperfect stand-in for visual similarity: same-category products may be visually dissimilar, and different-category products may share strong visual features.
5. **Ground-truth sensitivity.** As shown in Table 54, absolute mAP falls from 0.93 under category-only labels to 0.20-0.25 under category+colour and 0.16-0.21 under category+colour+pattern, with the model ranking reordering as fine attributes dominate.
6. **Single hardware profile.** All inference figures are tied to one CPU configuration (Intel i7-1165G7) without GPU acceleration.
7. **Pilot scale.** The 5,000-image benchmark establishes retrieval quality and relative model ranking at pilot scale; it does not characterise behaviour at

production catalogue scale (tens of thousands of SKUs or more). Production-scale validation is addressed in Future Work (Part 3, item 4).

8. **Approximate RAM.** RAM figures are estimated from model parameter counts because direct psutil measurement proved unreliable on the benchmark’s Linux host.
9. **Limited statistical power.** With six models over three folds, the benchmark detects large effects but may miss smaller differences (see Table 53).
10. **Confounded fine-tuning.** Fashion-CLIP’s retrieval advantage over general-purpose CLIP may partly reflect differences in its 700K-image fashion pre-training corpus rather than architecture or fine-tuning alone; isolating each factor’s contribution is outside this thesis’s scope. Using the non-overlapping-bounds heuristic (mean $\pm$ two standard deviations), Fashion-CLIP’s lower mAP bound (0.9216) overlaps the upper bounds of every other model, including both CNNs, so no model is statistically separable from the field on category-only retrieval; Fashion-CLIP should be described as having the highest observed mean mAP rather than a demonstrated superiority.

3.8.4 Summary

Six findings emerge from the six-model benchmark:

1. **Domain-specific fine-tuning matters.** Fashion-CLIP’s 1.46% relative mAP improvement over generic CLIP ViT-B/16 shows that domain adaptation yields measurable gains, though the gap to the self-supervised DINOv2 ViT-S/14 is only 0.40%.
2. **Architecture choice dominates the trade-off.** CNN and transformer-based models sit in distinct accuracy-efficiency ranges; practitioners should choose family by operational constraints, then select the best model within that family.
3. **Ground-truth definition reshapes the ranking.** DINOv2 ViT-S/14 is competitive under coarse category labels but collapses under fine colour/pattern labels, whereas the CLIP family stays consistent, a reminder that benchmark conclusions depend on the relevance criterion.
4. **The pluggable model architecture is genuinely useful in practice.** Switching models via one environment variable transforms model assessment into systematic comparison, enabling production A/B testing and graceful fallback.
5. **Commodity CPU hardware is enough.** Even CLIP models complete inference within 240 ms; combined with pgvector IVFFlat indexes (2.7-6.5 ms), total end-to-end latency stays under one second.
6. **Open-source tools are enough.** Pre-trained open-source models and pgvector provide production-oriented visual search without proprietary APIs or specialised hardware.

Answer to RQ3. The sidecar architecture isolates ML inference from web application logic. The *EMBEDDING_MODEL* environment variable enables model switching without backend code changes. On CPU, end-to-end search latency stays under one second across all six benchmarked models. Independent scaling and fault isolation are enabled by the sidecar boundary without distributed infrastructure overhead; this study demonstrates the latency and model-separation behaviour, while treating the scaling and isolation guarantees as architectural design properties rather than experimentally stress-tested results. The design suggests that a polyglot architecture with a dedicated AI sidecar is viable for real-time interactive search on commodity hardware. This viability assumes a team with at least one engineer comfortable operating a Python service alongside the .NET deployment; for a very small team without that capability, the overhead of maintaining a second language runtime may outweigh the architectural benefit.

PART 3: CONCLUSION AND FUTURE WORK

This chapter summarises the thesis, answers the research questions, states the contributions, discusses limitations and future work, and traces requirements to findings.

I. SUMMARY OF WORK

This thesis built a fashion e-commerce platform integrating a Vue 3 storefront, .NET 10 modular monolith, and Python ML sidecar. The visual search pipeline was assessed through systematic benchmarking of six models under 3-fold cross-validation on 5,000 fashion images. Three principal findings emerged. Domain-specific pre-training provides measurable retrieval advantages. The accuracy-efficiency trade-off can be managed through architecture choice. The polyglot sidecar architecture is viable for .NET enterprise stacks, achieving interactive response times on commodity hardware.

Answering the Research Questions

Table 58: Research Question Answers

RQ	Answer	Evidence
RQ1	Fashion-specific vs general-purpose models: Fashion-CLIP achieved highest mAP (0.9336); advantage over DINOv2 ViT-S/14 (0.40%) within measurement uncertainty at 3-fold CV; CLIP family more stable than DINOv2 under fine-grained ground truth	Table 52, Table 54, F3.6.1, F3.6.2
RQ2	Accuracy-speed trade-off spans 2.86% mAP across 2.7x latency range (42.6–113.6 ms); EfficientNet-B0 achieves 97.2% of Fashion-CLIP's accuracy at 37.5% of latency	Table 55, Table 56
RQ3	Sidecar architecture separates ML inference from web logic; end-to-end search latency under one second on CPU; model switching via environment variable	Section 2.3, Table 56

Achievement of Technical Objectives. All four technical objectives were addressed within the scope of the 5,000-image benchmark and commodity hardware. Model integration was demonstrated through the operational search pipeline. Polyglot architecture achieved clean separation using the sidecar pattern. pgvector proved feasible: IVFFlat queries execute under 10 ms (2.7-6.5

ms). Benchmark analysis yielded accuracy and efficiency metrics across six benchmarked models spanning four architectural families.

II. CONTRIBUTIONS

This thesis makes five concrete contributions:

- **A six-model benchmark for fashion image retrieval.** Systematic benchmarking with seven reported accuracy columns (three metric families at three depths) and five efficiency metrics across four architecture families, six models supported, 3-fold cross-validation protocol, plus a three-way ground-truth sensitivity analysis.
- **A reference CBIR implementation integrated into a production-style e-commerce platform.** The pipeline shows that open-source tools (PyTorch, FastAPI, pgvector, .NET 10) can support competitive visual search.
- **A pluggable model architecture enabling runtime model switching.** Strategy-pattern Model Manager controlled via environment variable decouples model selection from application code.
- **Demonstration of pgvector’s ACID-compliant vector storage.** Embeddings in the same PostgreSQL database as product data eliminate stale-index bugs.
- **A validated polyglot architecture pattern for .NET and Python AI.** The sidecar integration gives other teams a working example for incorporating Python ML into .NET applications.

As referenced by the title’s **Recommendation**, product recommendation in this work is delivered through the **Similar Products** visual-similarity feature (Section 2.3): given a target product, its primary image embedding is queried against the catalogue to surface visually similar items. This is a query-driven, content-based recommendation mechanism; collaborative-filtering or session-based personalisation is not implemented and is left to future work.

III. LIMITATIONS

Several limitations affect how well these findings apply more broadly:

1. The benchmark uses 5,000 product images from a single dataset; results may not generalise to other markets.
2. All figures were measured on a single laptop with CPU-only inference.
3. The binary category-label ground truth is an imperfect stand-in for visual similarity.
4. No formal user study was conducted.
5. All models were used as published without fine-tuning.
6. CLIP-based models’ text-to-image capability was not assessed.

7. The enriched-label scheme reduces $P@20$ substantially (from ~ 0.90 under category-only labels to ~ 0.30 under category+colour+pattern labels, see Appendix A.2 and A.3) due to the finer-grained relevance criterion.
8. RAM figures are approximate, estimated from each model’s parameter count plus PyTorch runtime overhead (ranging from 100 MB for EfficientNet-B0 to 600 MB for the CLIP-family models) because direct process-level measurement proved unreliable.
9. The 5,000-image benchmark establishes retrieval quality and relative model ranking, but does not characterise behaviour at production catalogue scale (millions of items), where index build time, query throughput under concurrent load, and embedding-storage growth become the dominant operational concerns.
10. Fashion-CLIP’s retrieval advantage over general-purpose CLIP may partly reflect differences in its 700K-image fashion pre-training corpus rather than architecture or fine-tuning alone; isolating each factor’s contribution is outside this thesis’s scope.

Ethical Considerations

Visual search also raises ethical considerations that this engineering thesis does not resolve:

1. Image-based recommendation may reinforce homogeneity, repeatedly surfacing visually similar items and narrowing catalogue diversity rather than broadening discovery.
2. Where a deployment learns from user behaviour, the underlying embeddings can inherit and amplify bias present in the training data (e.g., skewed representation across body types, skin tones, or styles).
3. Uploading personal images for search raises privacy expectations around storage, retention, and deletion, which a production deployment must address through explicit consent and retention policies.
4. Transformer inference carries an energy cost that scales with catalogue and request volume.

These concerns are relevant to any responsible deployment of the reference implementation and are acknowledged here as open issues rather than resolved claims.

IV. FUTURE WORK

These directions address the limitations identified above. They are grouped roughly by expected effort: near-term, low-risk extensions (1-3) build directly on the existing pipeline; medium-term enhancements (4-6) require additional data or components; and the longest-lead item (7) targets a separate deployment target.

1. Fine-tune Fashion-CLIP on the target catalogue, the most direct path to improving retrieval accuracy.
2. Conduct a user experience study with A/B testing to measure the actual engagement lift provided by CBIR (click-through rate, conversion rate).
3. Implement multi-modal search combining text and image queries, exploiting CLIP-family models' shared latent space.
4. Scale the benchmark to production-size catalogues (100,000 to 1,000,000 images) to validate pgvector HNSW scalability and model ranking stability.
5. Investigate ONNX Runtime optimisation to reduce transformer inference latency by 30 to 50 percent through operator fusion and hardware-specific kernels.
6. Add personalised re-ranking using user-level signals (past purchases, browsing history, wishlists).
7. Develop a mobile application with on-device inference using quantised EfficientNet-B0 for offline visual search.

These directions outline next steps from research demonstration toward a production-grade visual commerce engine, each based on empirical findings and architectural decisions documented in preceding chapters.

V. REQUIREMENTS TRACEABILITY

Table 59 shows that every objective and research question from Chapter 1 is addressed in a specific chapter section and produces a verifiable finding.

Table 59: Requirements traceability mapping objectives and research questions to chapters.

Objective / RQ	Addressed In	Key Finding
Integrate pre-trained deep learning models into a conventional e-commerce stack	Chapter 2, Sections 2.2.3–2.2.4, 2.3.2–2.3.3	Functional visual search pipeline: Vue storefront, .NET backend, Python ML sidecar, PostgreSQL pgvector, sub-second latency.
Architect a polyglot system connecting .NET and Python ML	Chapter 2, Sections 2.2.3–2.2.4, 2.3.1–2.3.3	Sidecar pattern isolates ML inference from transactional logic; integration tests validate cross-service HTTP contract.
Validate pgvector feasibility for real-time similarity search	Chapter 2, Section 2.3.4, Section 2.4.3	IVFFlat cosine similarity queries under 10 ms (2.7–6.5 ms); vectors share PostgreSQL database with relational data.
Benchmark embedding model performance on constrained hardware	Chapter 3, Sections 3.3–3.8	Six models benchmarked on 5,000 images. Fashion-CLIP: mAP 0.9336; EfficientNet-B0: 42.6 ms. F3.6.1, F3.6.2

Objective / RQ	Addressed In	Key Finding
RQ1: Fashion-specific vs general-purpose model comparison	Chapter 3, Section 3.4; Chapter 3, Section 3.6	Fashion-CLIP highest mAP (0.9336); DINOv2 ViT-S/14 within 0.40% on category-only retrieval. F3.6.1, F3.6.2, C1
RQ2: Accuracy vs speed trade-offs	Chapter 3, Sections 3.4–3.7	Fashion-CLIP: mAP 0.9336 at 113.6 ms; EfficientNet-B0: 97.2% of accuracy at 37.5% of latency (42.6 ms). Table 55
RQ3: Sidecar architecture viability for real-time search	Chapter 2, Sections 2.3.2–2.3.3; Chapter 3, Section 3.8.4	End-to-end latency under one second; model switching via environment variable. Table 56
Build AI service	Chapter 2, Section 2.3.2	Python FastAPI service with three-layer architecture, lazy-loading, containerised via Docker.
Set up vector search	Chapter 2, Section 2.3.4, Section 2.4.4	pgvector with cosine similarity; IVFFlat queries under 10 ms; vector storage co-exists with relational data.
Connect the services	Chapter 2, Sections 2.3.2–2.3.3	.NET CBIR handler: client validation, magic-byte verification, HTTP to ML sidecar, pgvector query, result deduplication.
Create the user interface	Chapter 2, Sections 2.3.3 and 2.3.5	Vue 3 storefront: drag-and-drop upload, similarity badges, product-card display, client-side validation.
Assess the results	Chapter 3, Sections 3.3–3.8	Six-model benchmark, 3-fold cross-validation on 5,000 images, seven reported accuracy columns (three metric families at three depths) and five efficiency metrics, plus three-way ground-truth sensitivity.

All objectives and research questions are addressed; no gaps remain.

This thesis demonstrated that integrating deep learning-based visual search into a conventional e-commerce platform is technically feasible with open-source tools on modest hardware. The contributions offer reference patterns for practitioners pursuing visual search in e-commerce.

REFERENCES

- [1] Statista Research Department, “Fashion e-Commerce Market Worldwide.” [Online]. Available: <https://www.statista.com/outlook/dmo/ecommerce/fashion/worldwide>
- [2] Pinterest Engineering, “Pinterest Visual Search: 600M+ Monthly Searches.” [Online]. Available: <https://newsroom.pinterest.com/>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778.
- [4] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [5] A. Radford *et al.*, “Learning Transferable Visual Models From Natural Language Supervision,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [6] P. J. Chia *et al.*, “Contrastive Language and Vision Learning of General Fashion Concepts,” *Scientific Reports*, vol. 12, p. 18958, 2022, doi: 10.1038/s41598-022-23052-9.
- [7] P. Aggarwal, “Fashion Product Images Dataset.” [Online]. Available: <https://www.kaggle.com/datasets/paramaggarwal/fashion-product-images>
- [8] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.
- [9] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2008.
- [10] A. Dosovitskiy *et al.*, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021.
- [11] M. Oquab *et al.*, “DINOv2: Learning Robust Visual Features without Supervision,” *arXiv preprint arXiv:2304.07193*, 2023.
- [12] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2018.

- [13] A. Kane, “pgvector: Open-source vector similarity search for Postgres.” [Online]. Available: <https://github.com/pgvector/pgvector>
- [14] M. Shaw and D. Garlan, *Software Architecture: Perspectives on an Emerging Practice*. Prentice Hall, 2012.
- [15] S. Newman, *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media, 2019.
- [16] J. Lewis and M. Fowler, “Microservices: A Definition of This New Architectural Term,” *martinfowler.com*, 2014, [Online]. Available: <https://martinfowler.com/articles/microservices.html>
- [17] J. Bogard, “Vertical Slice Architecture.” [Online]. Available: <https://jimmybogard.com/vertical-slice-architecture/>
- [18] G. Young, “CQRS and Event Sourcing.” [Online]. Available: https://cQRS.files.wordpress.com/2010/11/cQRS_documents.pdf
- [19] Microsoft Corporation, “ASP.NET Core Documentation.” [Online]. Available: <https://learn.microsoft.com/en-us/aspnet/core/>
- [20] Microsoft Corporation, “Entity Framework Core Documentation.” [Online]. Available: <https://learn.microsoft.com/en-us/ef/core/>
- [21] Evan You and the Vue.js Core Team, “Vue.js Documentation.” [Online]. Available: <https://vuejs.org/>
- [22] Redis Ltd., “Redis Documentation.” [Online]. Available: <https://redis.io/docs/>
- [23] A. Paszke *et al.*, “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [24] S. Karmazin, “Hangfire Documentation.” [Online]. Available: <https://docs.hangfire.io/>
- [25] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” technical report RFC 7519, 2015. doi: 10.17487/RFC7519.
- [26] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, “DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 1096–1104.
- [27] H. Wu *et al.*, “Fashion IQ: A New Dataset Towards Retrieving Images by Natural Language Feedback,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2021, pp. 11307–11317.

- [28] R. Girdhar *et al.*, “ImageBind: One Embedding Space to Bind Them All,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 15180–15190.
- [29] J. Li, D. Li, S. Savarese, and S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 19730–19742.
- [30] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, UK: Cambridge University Press, 2008.

APPENDICES

This appendix provides supplementary material for the benchmark analysis presented in Chapter 3. It includes the complete retrieval results for all six benchmarked models across three ground-truth label schemes (A), the dataset composition and category distribution (B), and the full hardware specifications of the benchmark environment (C). The ReSys.Shop database schema is documented in Section 2.3.4.

A FULL BENCHMARK RESULTS

This appendix presents the complete retrieval accuracy and operational efficiency results from the 3-fold cross-validation benchmark described in Chapter 3. All six benchmarked models are reported under three ground-truth label schemes.

A.1 CATEGORY-ONLY GROUND TRUTH (5 CATEGORIES)

Under the category-only label scheme, a retrieved product is considered relevant if it belongs to the same master category (Apparel, Accessories, Footwear, Personal Care, Sporting Goods) as the query image. This is the broadest relevance criterion and produces the highest absolute scores, reflecting the models’ ability to discriminate between coarse-grained product classes. It is the headline scheme used for the Chapter 3 primary results.

Table 60: Retrieval Accuracy, Category-Only Ground Truth (3-Fold CV, Mean $\pm$ SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	0.9336 $\pm$ 0.0060	0.9607 $\pm$ 0.0041	0.9527 $\pm$ 0.0043	0.9383 $\pm$ 0.0036	0.0282 $\pm$ 0.0014	0.0488 $\pm$ 0.0021	0.0816 $\pm$ 0.0031
DINOv2 ViT-S/14	0.9299 $\pm$ 0.0058	0.9572 $\pm$ 0.0029	0.9491 $\pm$ 0.0026	0.9360 $\pm$ 0.0028	0.0275 $\pm$ 0.0021	0.0484 $\pm$ 0.0030	0.0813 $\pm$ 0.0032
CLIP ViT-B/16	0.9202 $\pm$ 0.0043	0.9515 $\pm$ 0.0029	0.9423 $\pm$ 0.0023	0.9297 $\pm$ 0.0021	0.0275 $\pm$ 0.0009	0.0474 $\pm$ 0.0018	0.0790 $\pm$ 0.0025
CLIP ViT-B/32	0.9184 $\pm$ 0.0060	0.9482 $\pm$ 0.0028	0.9408 $\pm$ 0.0041	0.9282 $\pm$ 0.0037	0.0272 $\pm$ 0.0011	0.0474 $\pm$ 0.0023	0.0785 $\pm$ 0.0019
ResNet-50	0.9132 $\pm$ 0.0057	0.9457 $\pm$ 0.0024	0.9364 $\pm$ 0.0020	0.9246 $\pm$ 0.0021	0.0262 $\pm$ 0.0020	0.0452 $\pm$ 0.0025	0.0766 $\pm$ 0.0023
EfficientNet-B0	0.9077 $\pm$ 0.0076	0.9439 $\pm$ 0.0041	0.9342 $\pm$ 0.0046	0.9205 $\pm$ 0.0042	0.0257 $\pm$ 0.0016	0.0443 $\pm$ 0.0019	0.0748 $\pm$ 0.0022

Under this broadest criterion, all six models achieve high precision (above 0.90 at P@10). Fashion-CLIP leads with mAP of 0.9336, followed by DINOv2 ViT-S/14 (0.9299) and the two CLIP ViT-B variants (0.9184–0.9202). The low recall values (R@20 of 0.07–0.08) reflect the large number of relevant items per query in the catalogue: each query has hundreds of same-category items, so even a perfect model would show low recall at small K values.

A.2 CATEGORY + COLOUR GROUND TRUTH

Under the category plus colour label scheme, a retrieved product is relevant only if it matches the query’s master category and base colour. This finer-grained scheme better isolates the models’ ability to capture visual similarity, as opposed to merely coarse category classification.

Table 61: Retrieval Accuracy, Category + Colour Ground Truth (3-Fold CV, Mean $\pm$ SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	0.2439 $\pm$ 0.0074	0.4414 $\pm$ 0.0106	0.4041 $\pm$ 0.0100	0.3619 $\pm$ 0.0082	0.0430 $\pm$ 0.0017	0.0717 $\pm$ 0.0031	0.1130 $\pm$ 0.0027
CLIP ViT-B/16	0.2253 $\pm$ 0.0046	0.4068 $\pm$ 0.0069	0.3763 $\pm$ 0.0063	0.3411 $\pm$ 0.0061	0.0392 $\pm$ 0.0012	0.0669 $\pm$ 0.0006	0.1089 $\pm$ 0.0018
CLIP ViT-B/32	0.2250 $\pm$ 0.0034	0.4099 $\pm$ 0.0057	0.3761 $\pm$ 0.0049	0.3414 $\pm$ 0.0046	0.0390 $\pm$ 0.0033	0.0666 $\pm$ 0.0019	0.1066 $\pm$ 0.0026
EfficientNet-B0	0.2248 $\pm$ 0.0018	0.4050 $\pm$ 0.0042	0.3767 $\pm$ 0.0021	0.3435 $\pm$ 0.0025	0.0350 $\pm$ 0.0014	0.0595 $\pm$ 0.0019	0.0976 $\pm$ 0.0019
ResNet-50	0.2028 $\pm$ 0.0065	0.3813 $\pm$ 0.0105	0.3516 $\pm$ 0.0069	0.3208 $\pm$ 0.0059	0.0350 $\pm$ 0.0017	0.0582 $\pm$ 0.0030	0.0955 $\pm$ 0.0029
DINOv2 ViT-S/14	0.1899 $\pm$ 0.0034	0.3478 $\pm$ 0.0059	0.3305 $\pm$ 0.0045	0.3057 $\pm$ 0.0041	0.0336 $\pm$ 0.0025	0.0586 $\pm$ 0.0031	0.0975 $\pm$ 0.0044

Under this stricter label scheme, absolute mAP drops to the 0.19-0.25 range. Fashion-CLIP maintains the clear lead (0.2439); the CLIP ViT-B/16 and CLIP ViT-B/32 variants cluster at 0.2253 and 0.2250, with EfficientNet-B0 close behind at 0.2248. Notably, DINOv2 ViT-S/14 collapses to 0.1899, the lowest of all six models, indicating that its self-supervised representation, while strong at category level, encodes fashion-specific colour attributes less precisely than the CLIP-family models. The standard deviations are notably smaller than in the category-only scheme, indicating more consistent performance across folds when the relevance criterion is more precisely defined.

A.3 CATEGORY + COLOUR + PATTERN GROUND TRUTH

Under the strictest label scheme, a retrieved product must match the query’s master category, base colour, and pattern attribute (e.g., Solid, Striped, Checked, Floral). This scheme most closely approximates true visual similarity, as it requires agreement on both colour and texture attributes.

Table 62: Retrieval Accuracy, Category + Colour + Pattern Ground Truth (3-Fold CV, Mean $\pm$ SD)

Model	mAP	P@5	P@10	P@20	R@5	R@10	R@20
Fashion-CLIP	0.2071 $\pm$ 0.0074	0.3762 $\pm$ 0.0097	0.3377 $\pm$ 0.0090	0.2952 $\pm$ 0.0065	0.0628 $\pm$ 0.0024	0.1032 $\pm$ 0.0044	0.1586 $\pm$ 0.0042
CLIP ViT-B/16	0.1861 $\pm$ 0.0057	0.3350 $\pm$ 0.0069	0.3025 $\pm$ 0.0058	0.2668 $\pm$ 0.0052	0.0562 $\pm$ 0.0024	0.0923 $\pm$ 0.0015	0.1460 $\pm$ 0.0037
CLIP ViT-B/32	0.1859 $\pm$ 0.0052	0.3399 $\pm$ 0.0061	0.3053 $\pm$ 0.0052	0.2685 $\pm$ 0.0052	0.0564 $\pm$ 0.0037	0.0928 $\pm$ 0.0026	0.1437 $\pm$ 0.0025
EfficientNet-B0	0.1842 $\pm$ 0.0022	0.3369 $\pm$ 0.0027	0.3071 $\pm$ 0.0040	0.2748 $\pm$ 0.0038	0.0519 $\pm$ 0.0010	0.0861 $\pm$ 0.0027	0.1370 $\pm$ 0.0032
DINOv2 ViT-S/14	0.1651 $\pm$ 0.0040	0.2984 $\pm$ 0.0045	0.2771 $\pm$ 0.0041	0.2511 $\pm$ 0.0037	0.0478 $\pm$ 0.0020	0.0810 $\pm$ 0.0032	0.1320 $\pm$ 0.0045
ResNet-50	0.1629 $\pm$ 0.0065	0.3105 $\pm$ 0.0112	0.2805 $\pm$ 0.0074	0.2517 $\pm$ 0.0063	0.0512 $\pm$ 0.0025	0.0834 $\pm$ 0.0033	0.1335 $\pm$ 0.0046

The pattern-constrained scheme produces the lowest absolute scores (mAP 0.16–0.21), which is expected given the narrowest definition of relevance. Fashion-CLIP again leads (0.2071), followed by the CLIP ViT-B/16 (0.1861) and CLIP ViT-B/32 (0.1859) variants and EfficientNet-B0 (0.1842); DINOv2 ViT-S/14 (0.1651) and ResNet-50 (0.1629) occupy the lower tier. The model ranking is therefore highly dependent on the relevance definition: DINOv2’s strong category-level showing does not extend to fine-grained colour and pattern attributes, whereas the CLIP family remains consistently stable.

A.4 OPERATIONAL EFFICIENCY (3-FOLD CV)

Table 63: Operational Efficiency, All Models (3-Fold CV, Mean $\pm$ SD)

Model	Latency (ms)	Through-put	Load (ms)	Storage (MB)	RAM (MB)*	Dim
EfficientNet-B0	42.6 $\pm$ 5.6	21.4 $\pm$ 1.0	118.3	8.1	100	1280
ResNet-50	96.6 $\pm$ 7.4	10.2 $\pm$ 0.0	385.6	13.0	150	2048
DINOv2 ViT-S/14	126.3 $\pm$ 5.1	10.2 $\pm$ 0.2	1,223.4	2.4	250	384
CLIP ViT-B/16	235.5 $\pm$ 7.3	4.0 $\pm$ 0.1	6,518.5	3.3	600	512

Model	Latency (ms)	Throughput	Load (ms)	Storage (MB)	RAM (MB)*	Dim
CLIP ViT-B/32	140.5 $\pm$ 7.8	11.9 $\pm$ 0.2	1,868.8	3.3	600	512
Fashion-CLIP	113.6 $\pm$ 3.7	14.2 $\pm$ 0.5	5,109.9	3.3	600	512

3

EfficientNet-B0 achieves the lowest inference latency (42.6 ms) and highest throughput (21.4 images per second), making it the most computationally efficient model. CLIP-based models (Fashion-CLIP, CLIP ViT-B/16, CLIP ViT-B/32) and DINOv2 incur the highest model load times (1.2–6.5 seconds) due to their transformer architectures and larger weight files. ResNet-50 requires the most storage (13.0 MB per 3,334 gallery vectors) owing to its high embedding dimensionality of 2,048. In this benchmark, ResNet-50 embeddings were not indexed with pgvector IVFFlat due to their large dimensionality; only the 512-dim and 1,280-dim models were indexed for the pgvector production measurements (Section 3.7). HNSW or dimensionality reduction would be needed for approximate indexing of 2,048-dim vectors at production scale. RAM values are estimates derived from each model’s parameter count plus PyTorch runtime overhead, because direct process-level measurement via psutil proved unreliable on the Linux host used for these experiments. These figures are indicative rather than instrumented; actual values vary with batch size and runtime overhead, scaling from approximately 100 MB (EfficientNet-B0) to over 600 MB (CLIP-based).

A.5 PER-FOLD VARIABILITY

The per-fold results for the primary label scheme (category only) are presented below to provide transparency on the cross-validation procedure and to permit independent verification of aggregate statistics.

Table 64: Per-Fold Breakdown, Category-Only Ground Truth

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean $\pm$ SD
Fashion-CLIP	0.9274	0.9340	0.9394	0.9336 $\pm$ 0.0060
DINOv2 ViT-S/14	0.9254	0.9278	0.9365	0.9299 $\pm$ 0.0058
CLIP ViT-B/16	0.9180	0.9174	0.9251	0.9202 $\pm$ 0.0043

³* Estimated from model parameter count plus PyTorch runtime overhead; direct measurement unreliable on the test kernel.

Model	Fold 0 mAP	Fold 1 mAP	Fold 2 mAP	Mean $\pm$ SD
CLIP ViT-B/32	0.9131	0.9171	0.9249	0.9184 $\pm$ 0.0060
ResNet-50	0.9099	0.9099	0.9198	0.9132 $\pm$ 0.0057
EfficientNet-B0	0.9031	0.9036	0.9165	0.9077 $\pm$ 0.0076

All six models exhibit low fold-to-fold variability (standard deviation 0.0043–0.0076); the 3-fold stratified split therefore preserves the dataset’s category distribution effectively, and model performance stays stable across different partitionings of the data.

B DATASET COMPOSITION

This appendix details the composition of the benchmark dataset used in the analysis presented in Chapter 3.

B.1 SOURCE AND SELECTION

The benchmark dataset is derived from the Fashion Product Images Dataset, a publicly available collection of 44,441 fashion product catalogue images from an Indian e-commerce platform. For the thesis benchmark, a controlled subset of 5,000 images was selected to balance benchmark rigour with computational tractability. The subset was sampled via stratified random sampling to preserve the natural category distribution.

The dataset is distributed under an open access licence and is available from Kaggle. Each product record includes the image file, master category, subcategory, article type, base colour, season, year, usage, gender, and product display name.

B.2 CATEGORY DISTRIBUTION

The 5,000-image subset is stratified across five master categories. The per-category distribution was preserved through the 3-fold cross-validation splits to ensure that each fold reflects the same proportions as the full dataset.

Table 65: Dataset Category Distribution

Category	Images	Percentage	Fold Size (Train / Test)
Apparel	2,500	50.0%	1,667 / 833
Accessories	1,250	25.0%	833 / 417
Footwear	750	15.0%	500 / 250
Personal Care	350	7.0%	233 / 117

Category	Images	Percentage	Fold Size (Train / Test)
Sporting Goods	150	3.0%	100 / 50
Total	5,000	100.0%	3,333 / 1,667

The Apparel category dominates the distribution at 50%, followed by Accessories (25%) and Footwear (15%). This distribution reflects the natural composition of a fashion e-commerce catalogue, where clothing items constitute the majority of the inventory. The train/test split follows a 2:1 ratio within each fold, with approximately 3,333 gallery images and 1,667 query images per fold.

B.3 GROUND-TRUTH LABEL SCHEMES

Three label schemes of increasing granularity were used for assessing retrieval relevance:

Category-only labels use the master category field (e.g., Apparel, Accessories, Footwear). Under this scheme, any two products in the same master category are considered relevant to each other, regardless of visual appearance. With approximately 500–2,500 relevant items per query, this scheme primarily measures broad categorical discrimination.

Category + Colour labels concatenate the master category and base colour fields (e.g., “Apparel/Blue”). This is the primary relevance criterion used in Chapter 3. With an average of 8.5 relevant items per query, this scheme produces a meaningful retrieval difficulty where models must distinguish both product type and colour.

Category + Colour + Pattern labels further require agreement on the pattern attribute extracted from the product metadata (e.g., “Apparel/Blue/Striped”). With an average of 3.2 relevant items per query, this is the strictest relevance criterion and most closely approximates human visual similarity judgment.

B.4 IMAGE PREPROCESSING

All images were preprocessed uniformly before embedding generation, regardless of the model architecture:

- **Resize:** Images were resized to 224 by 224 pixels using bilinear interpolation, matching the standard input resolution of all benchmarked models.
- **Normalisation:** Pixel values were normalised using the ImageNet channel statistics: mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225) for the RGB channels respectively. Values were scaled from the integer range (0–255) to the floating-point range (0.0–1.0) before normalisation.
- **Colour space:** Images were maintained in RGB colour space. No grayscale conversion or colour augmentation was applied.

- **Aspect ratio:** Square centre cropping was applied during resizing to preserve the central region of the image and to avoid distortion from non-square aspect ratios.

C HARDWARE SPECIFICATIONS

All benchmark results reported in Chapter 3 and Appendix A were collected on a single workstation.

C.1 COMPUTE HARDWARE

Table 66: Benchmark Workstation Configuration

Component	Specification
CPU	Intel (11th Gen Core i7-1165G7, 4 cores / 8 threads, 2.80 GHz base / 4.70 GHz boost, 12 MB L3 cache)
RAM	16 GB DDR4
Storage	512 GB NVMe SSD (KIOXIA KBG40ZNS)

All benchmarks were executed on CPU (no GPU). Pretrained model weights from HuggingFace and PyTorch Hub were used without fine-tuning. Inference timing includes preprocessing and postprocessing.

C.2 SOFTWARE STACK

Table 67: Benchmark Software Environment

Component	Version
Operating System	Linux (Ubuntu 24.04 LTS, kernel 6.8)
Python	3.12.x (CPython)
PyTorch	2.13.0
TorchVision	0.28.0
HuggingFace Transformers	5.14.1
OpenCLIP	3.3.0
NumPy	2.5.1

All models were loaded from pre-trained weights on HuggingFace or PyTorch Hub; no fine-tuning was performed.

C.3 PRECISION AND REPRODUCIBILITY

All computations used float32 precision; mixed precision was not used due to numerical instability. Reproducibility settings: random seed 42, *model.eval()*

with `torch.no_grad()`. Results may not be bitwise-identical on different hardware due to CPU performance variance, memory capacity differences, and disk I/O speed. All benchmark scripts and raw result files are available in the project's benchmark repository.

